

合肥工业大学

课程设计

设计题目 PHEV 整车能量管理与热管理协同控制

学生姓名 张容

学 号 2023213357

专业班级 智能车辆工程 23-02

指导教师 姚明尧

2026 年 7 月

附件三 合肥工业大学课程设计任务书及成绩评定表

姓名	张容 谢涵 赵佳申	学号	2023213357 2023213356 2023213376	成绩	
设计题目	PHEV 整车能量管理与热管理协同控制				
课程设计主要内容	1. 根据车辆参数和 CLTC-P 工况，建立整车纵向动力学及需求功率模型。 2. 建立发动机燃油消耗、电机效率和电池 SOC 更新模型。 3. 分别设计 CD-CS 基准策略、普通 ECMS 策略和热约束 ECMS 策略，实现发动机与动力电池之间的功率分配。 4. 扫描等效因子、SOC 修正系数或温度惩罚权重，寻找综合评价结果较优的参数组合。 5. 对比不同策略下的 SOC、电池温度、冷却能耗、燃油消耗和综合评价指标。 6. 采用动态规划方法进行全局优化分析，为热约束 ECMS 策略提供理论基准。评价提供参考。				
指导教师评语	1、工作态度：☐认真 ☐较认真 ☐不认真 2、工作量：☐饱满 ☐合格 ☐基本合格 ☐完成 3、设计计算：☐正确 ☐错误较少 ☐错误较多 4、设计：☐ ☐基本无错误 ☐非原则性错误较少 ☐非原则性错误较多 ☐存在原则性错误 ☐原则性错误较多 5、报告： ☐条理清晰、论述充分、文字通顺、图表规范、符合设计报告要求 ☐条理清晰、论述充分、文字通顺、图表较规范、符合设计报告要求 ☐条理基本清晰、论述基本充分、文字通顺、图表基本规范、符合设计报告要求 ☐设计报告条理不够清晰、论述不够充分但没有原则性错误、文字基本通顺、图表不够规范、符合设计报告要求 ☐设计报告条理不清晰、论述不充分，存在原则性错误 6、工作能力与团队合作： ☐独立工作能力强、有独到见解，有良好的团队协作精神 ☐有一定的独立工作能力，并具有较好的团队协作精神 ☐独立工作能力较差，有一定的团队协作精神 ☐独立工作能力差，不能与其他同学合作 7、答辩： ☐答辩过程回答问题正确，叙述清晰 ☐答辩过程回答问题正确，叙述较清晰 ☐答辩过程回答问题基本正确，叙述较清晰 ☐答辩过程回答问题基本正确，叙述不清晰 8、其他：				
指导教师签名：			年 月 日		

合肥工业大学

Hefei University of Technology

本科课程设计报告

Undergraduate Course Design Specification



题 目: PHEV 整车能量管理与热管理协同控制

专业名称: 智能车辆工程

入校年份: 2023 级

学生姓名: 张容

学 号: 2023213357

指导教师: 姚明尧

学院名称: 汽车与交通工程学院

完成时间: 2026 年 7 月

目录

1 绪论	1
1.1 研究背景	1
1.2 研究意义	1
1.3 研究目标	1
1.4 研究方法	2
2 车辆参数与动力系统构型	3
2.1 车辆及动力系统基础参数.....	3
2.2 能量管理与电池热管理控制参数.....	4
2.3 串联式 PHEV 动力系统构型	4
3 系统建模	6
3.1 已知乘用车 CLTC-P 工况	6
3.2 整车需求功率模型.....	7
3.3 发动机燃油消耗模型.....	8
3.4 电机/发电机效率模型	9
3.4.1 驱动电机.....	9
3.4.2 发电机.....	9
3.5 电池 SOC 更新模型。	10
3.5.1 等效电路模型.....	10
3.5.2 电池容量模型.....	10
3.6 建立电池发热、一阶温升和冷却功率模型。	11
3.6.1 电池发热模型.....	11
3.6.2 一阶温升模型.....	11
3.6.3 一阶冷却回路模型.....	12
4 CD-CS 与普通 ECMS 策略仿真分析	13
4.1 PHEV 能量管理与热管理协同控制系统结构.....	13
4.2 统一仿真条件与评价指标.....	13
4.3 CD-CS 基准策略原理	14
4.3.1 模式切换逻辑.....	14
4.3.2 功率分配规则.....	14
4.3.3 热力学模型(绝热假设).....	15
4.3.4 SOC 管理	15
4.3.5 燃油消耗计算.....	15
4.3.6 策略特点.....	15

4.3.7 策略小结.....	16
4.4 CD-CS 策略工况适应性分析	16
4.4.1 25 °C环境下的能量分配与状态变化.....	16
4.4.2 35 °C环境下的能量分配与状态变化.....	19
4.5 普通 ECMS 策略原理	22
4.5.1 核心目标与成本函数.....	22
4.5.2 功率分配策略.....	22
4.5.3 约束条件.....	23
4.5.4 温度模型(绝热假设).....	23
4.5.5 策略小结.....	23
4.6 普通 ECMS 策略参数影响分析	24
4.6.1 25 °C环境下等效因子 1~5 影响分析	24
4.6.2 35 °C环境下等效因子 1~5 的影响分析	27
4.6.3 35 °C环境下细化等效因子影响分析	30
4.6.4 等效因子与 SOC 修正系数联合寻优	33
4.7 本章小结	34
5 热约束 ECMS 策略仿真分析	35
5.1 热约束 ECMS 策略原理.....	35
5.1.1 温度惩罚函数定义.....	35
5.1.2 惩罚项在 ECMS 中的集成	35
5.1.3 模型核心逻辑.....	35
5.1.4 与冷却系统的协同.....	36
5.1.5 策略小结.....	36
5.2 热约束 ECMS 温度控制特性分析	36
5.2.1 25 °C环境下温度惩罚权重影响分析	36
5.2.2 35 °C环境下温度惩罚权重影响分析	39
5.2.3 温度惩罚权重与 SOC 修正系数联合寻优	42
5.3 三种能量管理策略综合性能对比.....	43
5.4 本章小结	45
6 动态规划策略及仿真对比分析	46
6.1 动态规划方法介绍.....	46
6.1.1 基本原理.....	46
6.1.2 方法特点与适用范围.....	46
6.2 动态规划能量管理策略设计.....	46
6.2.1 优化目标与问题定义.....	46

6.2.2 状态变量定义与离散化.....	47
6.2.3 控制变量定义与可行域.....	47
6.2.4 状态转移与约束处理.....	47
6.2.5 阶段代价与终端代价设计.....	48
6.2.6 倒推求解与最优策略生成.....	48
6.3 仿真结果与策略对比.....	49
6.3.1 前向仿真结果.....	49
6.3.2 与热约束 ECMS 的对比分析	49
6.4 本章小结	50
7 总结	52
参考文献	53
致谢	54
附录 1 环境 25℃ 下 CD-CS 策略 Python 程序.....	55
附录 2 环境 25℃ 下普通 ECMS 策略 Python 程序.....	66
附录 3 环境 25℃ 下热约束 ECMS 策略 Python 程序.....	78
附录 4 细化等效因子普通 ECMS 策略 Python 程序	89
附录 5 三种能量管理策略综合性能对比 Python 程序	94
附录 6 动态规划策略 Python 程序	102
附录 7 成员贡献度.....	111

插图清单

图 2.1 串联式 PHEV 动力系统构型	5
图 3.1 CLTC-P 行驶工况速度曲线	6
图 3.2 发动机输出功率—燃油消耗率及燃油效率特性曲线	8
图 3.3 动力电池一阶内阻等效电路模型	10
图 3.4 不同温度下动力电池电气参数随 SOC 的变化特性	11
图 4.1 PHEV 能量管理与热管理协同控制系统方案	13
图 4.4.1 25 °C 环境温度下 CD-CS 策略电池功率变化曲线	17
图 4.4.2 25 °C 环境温度下 CD-CS 策略驱动电机功率变化曲线	18
图 4.4.3 25 °C 环境温度下 CD-CS 策略发动机功率变化曲线	18
图 4.4.4 25 °C 环境温度下 CD-CS 策略 SOC 变化曲线	18
图 4.4.5 25 °C 环境温度下 CD-CS 策略电池温度变化曲线	19
图 4.4.6 35 °C 环境温度下 CD-CS 策略电池功率变化曲线	20
图 4.4.7 35 °C 环境温度下 CD-CS 策略驱动电机功率变化曲线	20
图 4.4.8 35 °C 环境温度下 CD-CS 策略发动机功率变化曲线	21
图 4.4.9 35 °C 环境温度下 CD-CS 策略 SOC 变化曲线	21
图 4.4.10 35 °C 环境温度下 CD-CS 策略电池温度变化曲线	21
图 4.6.1 25 °C 下不同初始 SOC 和等效因子的 SOC 变化对比	25
图 4.6.2 25 °C 下不同初始 SOC 和等效因子的电池功率对比	26
图 4.6.3 25 °C 下不同初始 SOC 和等效因子的驱动电机功率对比	26
图 4.6.4 25 °C 下不同初始 SOC 和等效因子的发动机功率对比	27
图 4.6.5 25 °C 下不同初始 SOC 和等效因子的电池温度对比	27
图 4.6.6 35 °C 下等效因子 1~5 时普通 ECMS 的 SOC 变化对比	28
图 4.6.7 35 °C 下等效因子 1~5 时普通 ECMS 的电池功率对比	29
图 4.6.8 35 °C 下等效因子 1~5 时普通 ECMS 的驱动电机功率对比	29
图 4.6.9 35 °C 下等效因子 1~5 时普通 ECMS 的发动机功率对比	30
图 4.6.10 35 °C 下等效因子 1~5 时普通 ECMS 的电池温度对比	30
图 4.6.11 35 °C 下不同细化等效因子的 SOC 变化对比	31
图 4.6.12 35 °C 下不同细化等效因子的电池功率对比	32
图 4.6.13 35 °C 下不同细化等效因子的驱动电机功率对比	32

图 4.6.14 35 °C下不同细化等效因子的发动机功率对比.....	33
图 4.6.15 35 °C下不同细化等效因子的电池温度对比.....	33
图 5.2.1 25 °C下不同温度惩罚权重的 SOC 变化对比	37
图 5.2.2 25 °C下不同温度惩罚权重的电池功率对比.....	38
图 5.2.3 25 °C下不同温度惩罚权重的驱动电机功率对比.....	38
图 5.2.4 25 °C下不同温度惩罚权重的发动机功率对比.....	39
图 5.2.5 25 °C下不同温度惩罚权重的电池温度对比.....	39
图 5.2.6 35 °C下不同温度惩罚权重的 SOC 变化对比	40
图 5.2.7 35 °C下不同温度惩罚权重的电池功率对比.....	41
图 5.2.8 35 °C下不同温度惩罚权重的驱动电机功率对比.....	41
图 5.2.9 35 °C下不同温度惩罚权重的发动机功率对比.....	42
图 5.2.10 35 °C下不同温度惩罚权重的电池温度对比.....	42
图 5.3 三种能量管理策略综合性能对比.....	44
图 6.1 动态规划与热约束 ECMS 的状态轨迹及发动机功率对比	50

表格清单

表 2.1 整车及动力系统主要参数.....	3
表 2.2 能量管理与热管理主要控制参数.....	4
表 3.1 CLTC-P 行驶工况主要特征参数	6
表 3.2 发动机输出功率与燃油消耗率数据.....	9
表 4.1 25 °C环境温度下 CD-CS 策略综合评价结果	16
表 4.2 35 °C环境温度下 CD-CS 策略综合评价结果	19
表 4.3 25 °C下等效因子 1~5 的普通 ECMS 评价结果	24
表 4.4 35 °C下等效因子 1~5 的普通 ECMS 评价结果	28
表 4.5 35 °C下细化等效因子的普通 ECMS 评价结果	31
表 4.6 不同基础等效因子对应的较优 SOC 修正系数	34
表 5.1 25 °C下不同温度惩罚权重的热约束 ECMS 评价结果	37
表 5.2 35 °C下不同温度惩罚权重的热约束 ECMS 评价结果	40
表 5.3 不同等效因子对应的较优温度惩罚权重.....	43
表 5.4 三种能量管理策略综合性能对比.....	44
表 6.1 动态规划与热约束 ECMS 综合油耗对比	49

1 绪论

1.1 研究背景

随着能源消耗和环境污染问题日益突出，新能源汽车已成为汽车产业的重要发展方向。插电式混合动力汽车兼具传统燃油汽车续航里程长、补能方便以及纯电动汽车节能环保的特点，能够根据车辆运行状态灵活使用燃油和电能，在降低燃油消耗、减少污染物排放等方面具有明显优势。

增程式电动汽车属于串联式插电混合动力汽车的一种。其发动机不直接参与车轮驱动，而是带动发电机发电，车辆主要由驱动电机提供动力。由于系统中同时存在发动机、发电机、驱动电机和动力电池等多个能量部件，因此需要通过合理的能量管理策略，对发动机功率和电池功率进行协调分配。

1.2 研究意义

动力电池在充放电过程中会因内阻产生热量，其温度受到环境温度、充放电功率和冷却系统运行状态等因素影响。电池温度过高可能降低电池性能并增加运行风险，而冷却系统工作又会产生额外能耗。

将电池温度约束和冷却能耗引入整车能量管理策略，可以在满足车辆动力需求和 SOC 约束的同时，综合考虑燃油消耗、电能消耗与电池热状态，有助于提高整车综合经济性，改善动力电池工作环境，并为 PHEV 能量管理与热管理协同控制提供参考。

1.3 研究目标

本课程设计以固定构型的串联式 PHEV 为研究对象，以 CLTC-P 行驶工况作为输入，建立整车需求功率、发动机燃油消耗、动力电池 SOC 更新及电池温升和冷却模型。

在此基础上，分别设计 CD-CS 策略、普通 ECMS 策略和热约束 ECMS 策略，并通过等效因子、SOC 修正系数和温度惩罚权重的参数扫描，研究不同控制参数对整车能耗、终端 SOC 和电池温度的影响，寻找综合评价结果较优的参数组合。

1.4 研究方法

本课程设计采用理论建模与 Python 仿真相结合的方法，使用 0D/1D 整车能量模型和电池一阶热模型，对车辆运行过程进行计算分析。

首先，根据 CLTC-P 工况和车辆参数计算整车需求功率；其次，建立发动机、电机、动力电池及电池热管理模型；然后，分别实现 CD-CS、普通 ECMS 和热约束 ECMS 三种控制策略，在不同环境温度和初始 SOC 条件下进行参数扫描，对比燃油消耗、电能消耗、冷却能耗、终端 SOC 偏差、电池最高温度和温度越界时间等指标；最后，采用动态规划开展离线全局优化，并将其作为热约束 ECMS 性能评价的理论基准。

2 车辆参数与动力系统构型

2.1 车辆及动力系统基础参数

本研究对象的整车参数介于传统乘用车与商用车之间，车辆类型可归为多用途乘用车（MPV）。综合考虑车辆用途及质量等级，循环工况选用中国轻型汽车行驶工况中的乘用车工况 CLTC-P 作为标准测试工况。整车、发动机、发电机、驱动电机和动力电池的主要参数如表 2.1 所示。表中参数作为车辆需求功率、部件功率限制、SOC 更新及电池温升计算的基础输入。

表 2.1 整车及动力系统主要参数

部件分类 (Component)	参数名称 (Parameter)	数值 (Value)	单位 (Unit)
整车	整备质量	3120	kg
	旋转质量换算系数	1.1	—
	空气阻力系数	0.6	—
	等效迎风面积	3.25	m^2
	滚动阻力系数	0.009	—
	传动系统效率	0.95	—
	主减速比	16.32	—
	油箱	不限制	L
发动机	峰值功率 / 对应转速	130/5200	kW
	峰值扭矩 / 对应转速	300/3500	$N \cdot m$
发电机	峰值功率	165	kW
	最高转速	13000	rpm
	峰值扭矩	350	$N \cdot m$
驱动电机	峰值功率	195	kW
	最高转速	14000	rpm
	峰值扭矩	350	$N \cdot m$
动力电池包	额定容量	155	$A \cdot h$
	额定电量	54.3	$kW \cdot h$

	额定电压	350.4	V
	比热容	1100	$J/(kg \cdot ^\circ C)$
	电池包总质量	267	kg

2.2 能量管理与电池热管理控制参数

能量管理与热管理模型采用的主要控制参数及推荐范围如表 2.2 所示。表中 SOC 工作窗口用于保证电池安全运行，等效因子和 SOC 修正系数用于调节能量分配，温度阈值及冷却功率用于约束电池热状态。

表 2.2 能量管理与热管理主要控制参数

参数	推荐范围	作用
SOC 工作窗口	20%–90%	保证电池可用能量
ECMS 等效因子	1–5	平衡燃油和电能消耗
SOC 修正系数	自定义范围（50%–70%）	维持终端 SOC
电池温度阈值	35–45 $^\circ C$	热管理约束
温度惩罚权重	自定义范围(1–5)	限制高温运行
冷却功率	0–2kW	估算附加能耗
电池热容/热阻	按参数表设定(热容 1100J/(kg· $^\circ C$)，只考虑电池到冷却液的热阻，等效传热系数 500 W/(m ² ·K)	建立热模型

2.3 串联式 PHEV 动力系统构型

车辆动力系统构型如图 2.1 所示。由图 2.1 可知，发动机仅驱动发电机发电，动力电池和发电机经逆变器向驱动电机供电，发动机不直接参与车轮驱动。

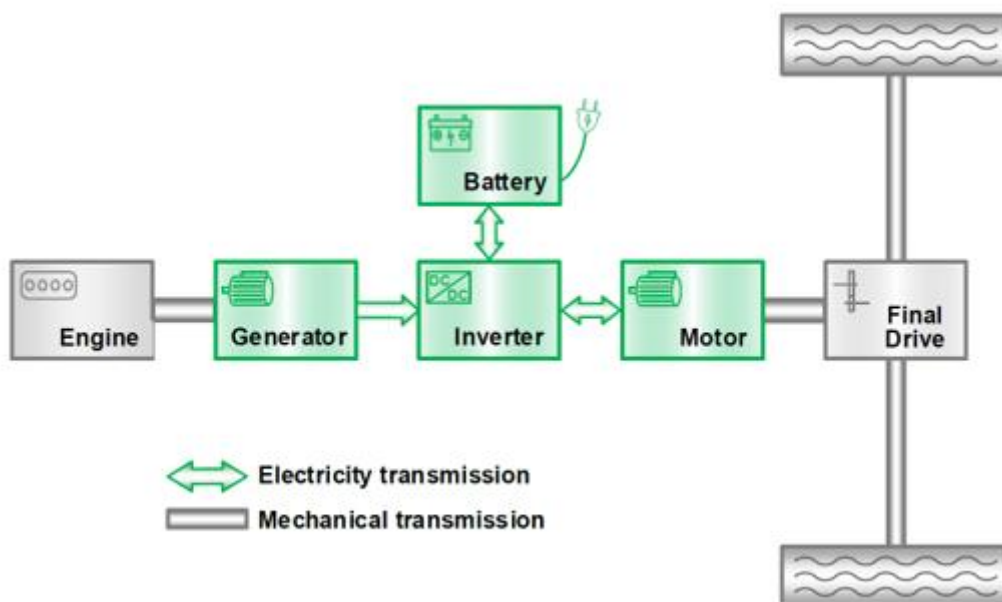


图 2.1 串联式 PHEV 动力系统构型

3 系统建模

3.1 已知乘用车 CLTC-P 工况

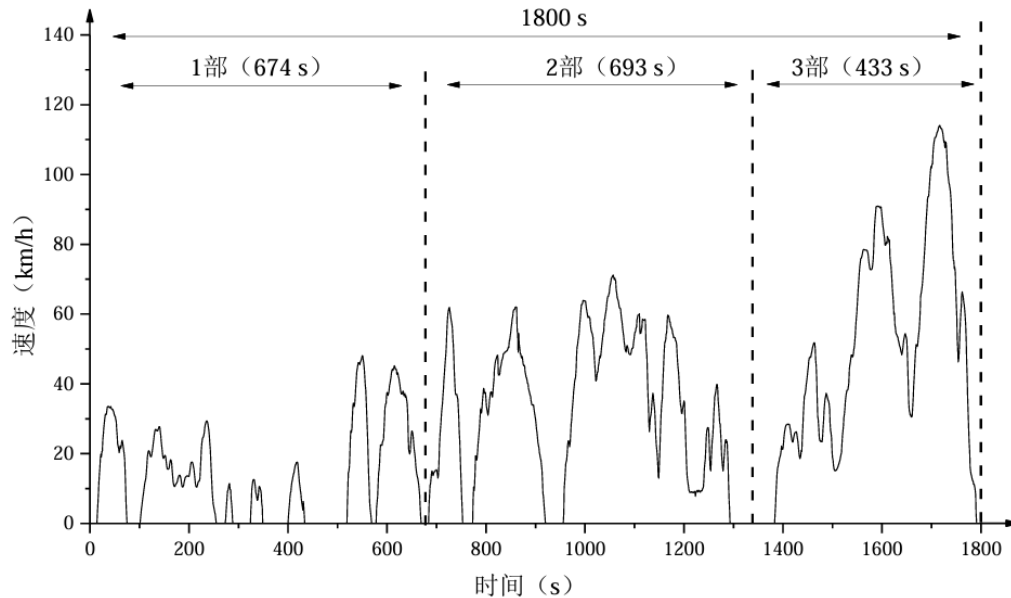


图 3.2 CLTC-P 行驶工况速度曲线

CLTC-P 工况共包低速、中速和高速 3 个速度区间，工况时长共计 1800 秒，其中低速区间时间比例为 37.4%，中速区间时间比例为 38.5%，高速区间时间比例为 24.1%，平均车速为 29.0km/h，最大车速为 114.0km/h，怠速比例为 22.1%。CLTC-P 工况逐秒数据见附录 A，CLTC-P 工况曲线如图 3.1 所示，该工况由低速、中速和高速三个阶段组成，能够反映城市低速、一般道路及高速行驶条件下的速度变化特征。CLTC-P 工况曲线统计特征参数如表 3.1 所示。

表 3.1 CLTC-P 行驶工况主要特征参数

特征	总体	1 部	2 部	3 部
运行时间(s)	1800	674	693	433
里程(km)	14.48	2.45	5.91	6.12
最大速度(km/h)	114.00	48.10	71.20	114.00
最大加速度(m/s^2)	1.47	1.47	1.44	1.06
最大减速度(m/s^2)	-1.47	-1.42	-1.47	-1.46

平均速度 (km/h)	28.96	13.09	30.68	50.90
运行平均速度 (km/h)	37.18	20.19	38.24	53.89
加速段平均加速度 (m/s^2)	0.45	0.42	0.46	0.46
减速段平均减速度 (m/s^2)	-0.49	-0.45	-0.50	-0.54
相对正加速度 RPA (m/s^2)	0.17	0.14	0.16	0.18
加速比例 (%)	28.61	22.26	30.45	35.57
减速比例 (%)	26.44	21.51	28.43	30.95
匀速比例 (%)	22.83	21.07	21.36	27.94
怠速比例 (%)	22.11	35.16	19.77	5.54

3.2 整车需求功率模型

车辆需求功率用于为驱动电机、空调系统和辅助系统提供能量。因此，PHEV 的功率平衡方程可表示为：

$$P_{eng} + P_{bat} = P_{motor} + P_{AC} + P_{aux} \quad (3.1)$$

式 (3.1) 中， P_{eng} 和 P_{bat} 分别表示发动机和电池的输出功率； P_{motor} 、 P_{AC} 和 P_{aux} 分别为驱动电机、空调系统和辅助系统的需求功率。

行驶过程中的纵向需求功率占需求功率的大部分，故忽略横向需求功率，驱动电机的纵向行驶需求功率计算如式 (3.2)。

$$P_{motor} = \begin{cases} (P_f + P_i + P_w + P_j) / \eta_T / \eta_m & drivemode \\ \eta_T \eta_m \times (P_f + P_i + P_w + P_j) & brakemode \end{cases} \quad (3.2)$$

式 (3.2) 中， η_t 和 η_m 分别为传动系统和驱动电机的效率； P_f 、 P_i 、 P_w 和 P_j 分别为克服滚动阻力、坡道阻力、空气阻力和加速阻力的需求功率，可写为式 (3.3)。

$$\begin{cases} P_f = m g f \cos \theta \cdot v \\ P_i = m g \sin \theta \cdot v \\ P_w = \frac{1}{2} C_D A \rho \cdot v^3 \\ P_j = \delta m \cdot a \cdot v \end{cases} \quad (3.3)$$

式 (3.3) 中, m 为整车质量, δ 为旋转质量换算系数 (含平移质量和旋转质量), C_D 为空气阻力系数, A 为等效迎风面积, f 为滚动阻力系数, g 为重力加速度, θ 为道路坡度, ρ 为空气密度, v 和 a 分别为车辆瞬时速度和加速度。

整备质量 3120kg, 旋转质量换算系数 1.1, 空气阻力系数 0.6, 等效迎风面积 3.25 平方米, 滚动阻力系数 0.009, 传动系统效率 0.95, 主减速比 16.32。

3.3 发动机燃油消耗模型

为建立发动机燃油消耗模型, 发动机燃油消耗率及燃油效率随输出功率的变化关系如图 3.2 所示。该图表明发动机燃油效率随功率先提高后降低, 可据此提取不同输出功率下的燃油消耗率。

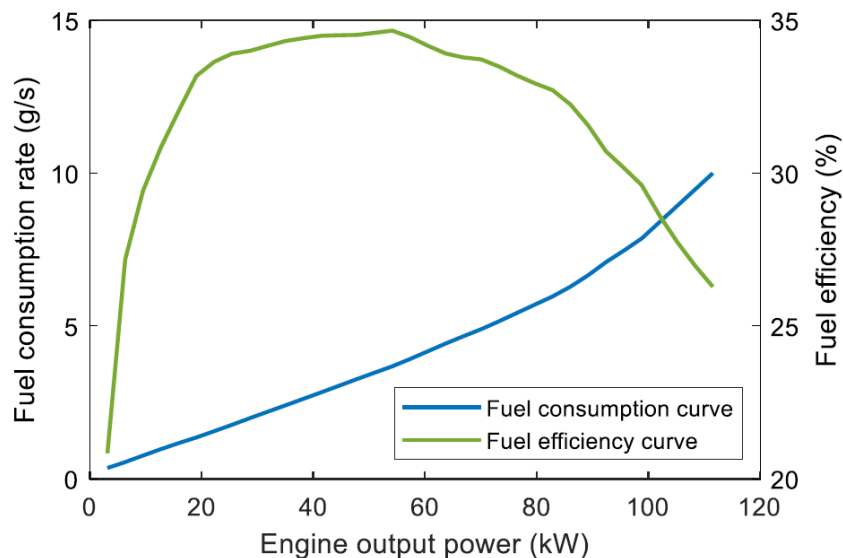


图 3.2 发动机输出功率—燃油消耗率及燃油效率特性曲线

根据上图提取简化的发动机经济性数据:

表 3.2 发动机输出功率与燃油消耗率数据

发动机输出功率 P_{eng} (kW)	燃油消耗率 $m \cdot \text{fuel}$ (g/s)	备注
-------------------------------	-----------------------------------	----

	(基于图中蓝线提取)	
0	0	停机状态
10	1.8	起步加速区
20	3.6	经济区起点
30	5.2	接近最佳经济区
40	6.5	最佳经济区中心
50	7.5	最佳经济区边缘
60	8.2	中等负荷
70	8.8	负荷增加
80	9.2	高负荷
90	9.6	接近满负荷
100	10.0	满负荷边缘
110	10.3	过载区
120	10.0	极限功率点

3.4 电机/发电机效率模型

3.4.1 驱动电机

驱动电机模型采用恒定效率法进行简化处理，电机效率 η_m 作为固定系数，用于表征电能与机械能转换过程中的综合损耗，未进一步建立基于转速和转矩的效率 MAP 。驱动电机峰值功率为 $195kW$ ，最高转速为 $14000r/min$ ，峰值扭矩为 $350N \cdot m$ 。

3.4.2 发电机

发电机未单独建立电压、电流及效率特性模型，其能量转换效率和损耗特性通过发动机—发电机组的综合燃油消耗特性进行等效表征，并纳入发动机最优燃油消耗曲线中。发电机峰值功率为 $165kW$ ，最高转速为 $13000r/min$ ，峰值扭矩为 $350N \cdot m$ 。

3.5 电池 SOC 更新模型。

3.5.1 等效电路模型

采用内阻等效电路模型描述电池的电特性，如图 3.3 所示。该模型由开路电压和欧姆内阻组成，可根据电池功率计算端电压、电流及内阻损耗。式（3.4）中， U_t 为电池端电压， U_{oc} 和 R_0 为电池单体开路电压和内阻，是电池关键参数。

$$I_b = \frac{U_{oc} - U_t}{R_0} = \frac{U_{oc} - \sqrt{U_{oc}^2 - 4 \cdot R_0 \cdot P_{bat}}}{2 \cdot R_0} \quad (3.4)$$

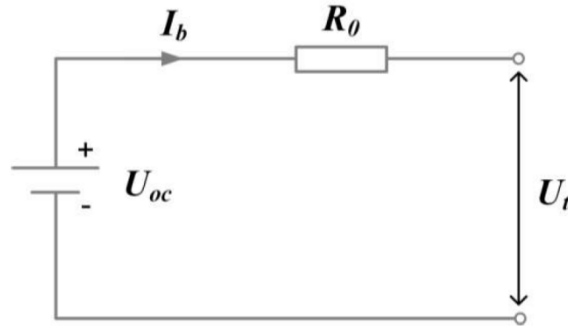


图 3.3 动力电池一阶内阻等效电路模型

3.5.2 电池容量模型

电池 SOC: State of Charge, 即荷电状态, 通常指电池剩余电量与总容量的比值。根据安时积分法, 电池 SOC 可表示为式（3.5）, 其中 Cap_{bat} 为电池容量。SOC 和电池温度的函数, 动力电池开路电压和内阻随 SOC 及温度变化的规律如图 3.4 所示。其中, 图 3.4(a) 为开路电压特性, 图 3.4(b) 和图 3.4(c) 分别表示不同工况下的内阻变化, 为电池电流、SOC 及发热量计算提供参数依据。

$$\frac{dSOC(t)}{dt} = \frac{-I_b(t)}{Cap_{bat}} \quad (3.5)$$

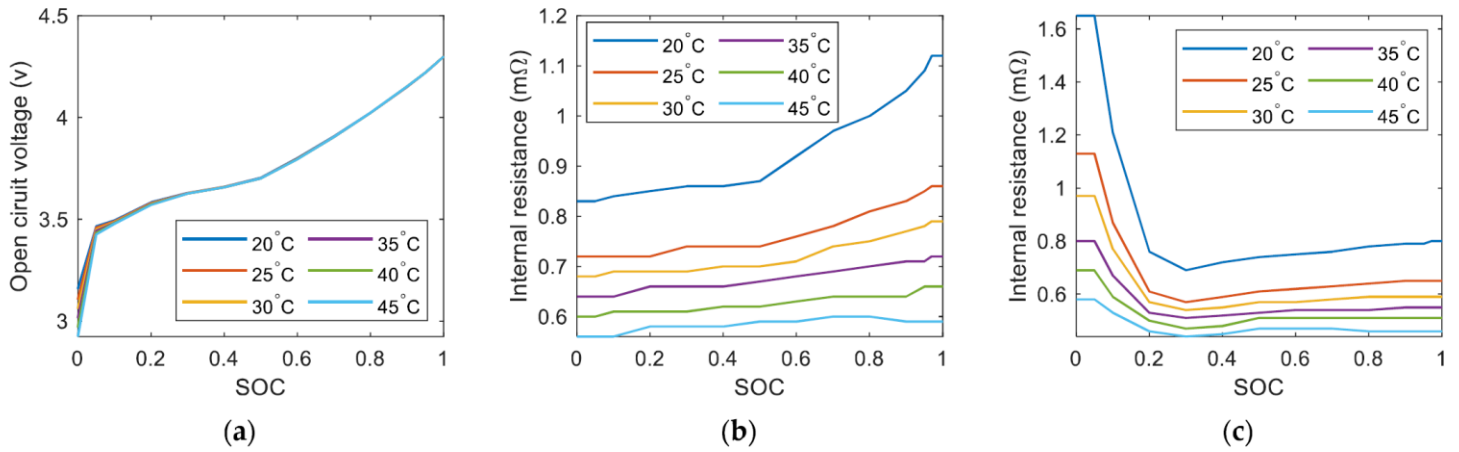


图 3.4 不同温度下动力电池电气参数随 SOC 的变化特性

3.6 建立电池发热、一阶温升和冷却功率模型。

3.6.1 电池发热模型

电池热模型描述电池的生热、换热和温度变化过程。根据 **Bernardi** 方程，电池生热 $\dot{Q}_{bat}$ 表示为式 (3.6)，第一项为由电流热效应引起的生热，第二项为可逆热，占总生热比例较小，可忽略：

$$\dot{Q}_{bat} = I_b^2 R_0 + I_b T_{bat} \frac{\partial U_{oc}}{\partial T_{bat}} \quad (3.6)$$

$$Q_{bat} = I_b^2 R_0 \quad (3.7)$$

3.6.2 一阶温升模型

电池换热过程主要包括电池与外界环境的换热量，以及电池与热管理系统之间的换热量。确定电池生热功率和换热功率后，其温度可描述为式 (3.8)

$$c_{p,bat} m_{bat} \frac{dT_{bat}}{dt} = I_b^2 R_0 + \frac{T_{amb} - T_{bat}}{R_{th,bat}} - \dot{Q}_{cool}^{bat} \quad (3.8)$$

这就是 0D 集总参数模型——把整个电池包当成一个均匀温度节点，不考虑内部梯度。

电池包的热容 $C_{th} = c_{p,bat} \cdot m_{bat}$ (单位: J/K), 则温升速率:

$$C_{th} \frac{dT_{bat}}{dt} = Q_{gen} - Q_{cool} - Q_{loss} \quad (3.9)$$

式 (3.9) 中:

Q_{cool} : 冷却系统从电池带走的热量 (W)

Q_{loss} : 电池包向环境自然散热 (W), $Q_{loss} = K_{loss}(T_{bat} - T_{amb})$

K_{loss} : 等效散热系数 (W/K), 包含壳体导热 + 表面对流

T_{amb} : 环境温度 (K)

写成标准形式 (3.10) (一阶线性 ODE):

$$\frac{dT_{bat}}{dt} = \frac{1}{C_{th}}(I^2 R_0 - Q_{cool} - K_{loss}(T_{bat} - T_{amb})) \quad (3.10)$$

3.6.3 一阶冷却回路模型

把冷却液温度也作为一个状态变量:

$$\begin{cases} C_{bat} \dot{T}_{bat} = Q_{gen} - hA(T_{bat} - T_{cool}) \\ C_{cool} \dot{T}_{cool} = hA(T_{bat} - T_{cool}) - \dot{m}_c c_p (T_{cool} - T_{rad_{out}}) \\ \dot{m}_c = f(P_{pump}) \end{cases} \quad (3.11)$$

其中:

hA : 电池到冷却液的等效传热系数 (W/K)

C_{cool} : 冷却液热容 (J/K)

$\dot{m}_c$: 冷却液质量流量 (kg/s), 由水泵功率 P_{pump} 决定

$T_{rad_{out}}$: 散热器出口温度, 取决于环境温度和风扇功率

4 CD-CS 与普通 ECMS 策略仿真分析

4.1 PHEV 能量管理与热管理协同控制系统结构

PHEV 能量管理与热管理协同控制系统结构如图 4.1 所示。该系统以行驶工况、车辆状态和环境温度为输入，通过 CD-CS、普通 ECMS 或热约束 ECMS 进行功率分配，同时结合电池热模型和冷却系统实现能量与温度的协同控制。

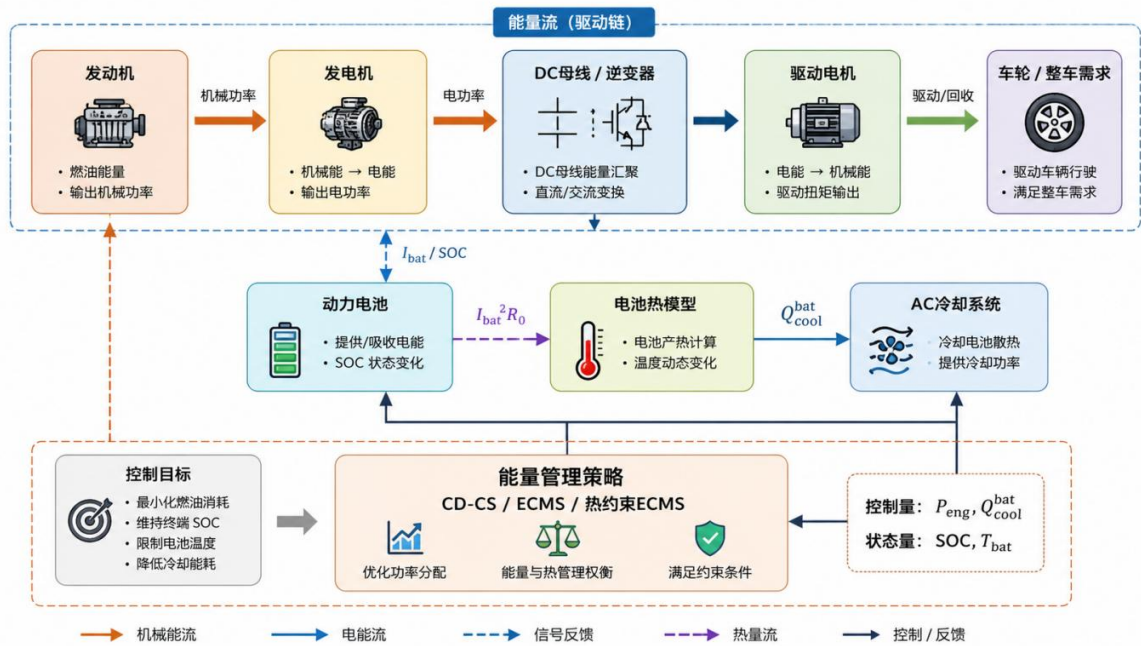


图 4.1 PHEV 能量管理与热管理协同系统方案

4.2 统一仿真条件与评价指标

行驶工况： CLTC 标准速度工况，循环数量30 – 60个；

环境温度： 25℃、35℃；

初始 SOC： 40%与 60%；

主要参数：SOC 工作窗口 20% – 90%； ECMS 等效因子，一组 1-5；另一组 1, 1.2, 1.4, 1.8, 2；SOC 修正系数 1-5，维持终端 SOC50% – 70% ； 电池温度阈值 35-45 ℃；温度惩罚权重 0.5-5；冷却功率0 – 2kW。

燃油消耗、电机效率、电池热容和冷却功率等参数来源：任务书参考文献一；热阻（等效传热系数）约定 $500 W/(m^2 \cdot K)$ 。

4.3 CD-CS 基准策略原理

CD-CS (Charge - Depleting / Charge - Sustaining) 策略模型通过基于 SOC 阈值的模式切换，结合纯电动 (CD 模式) 和发动机恒定功率辅助 (CS 模式)，实现能量管理，同时考虑绝热条件下的电池温度变化。

4.3.1 模式切换逻辑

以电池 SOC 是否超过下限阈值 ($SOC_{min} = 0.2$) 为切换依据：

CD 模式 (Charge-Depleting, 电量消耗模式)：当 $SOC > SOC_{min}$ 时，发动机关闭 ($P_{eng} = 0$)，仅由电池提供动力 ($P_{bat} = P_{req}$)。

CS 模式 (Charge-Sustaining, 电量维持模式)：当 $SOC \leq SOC_{min}$ 时，发动机以恒定功率 ($p_{eng_{cs}} = 50kW$) 工作，电池补充剩余功率 ($P_{bat} = P_{req} - P_{eng}$)，且电池充电功率限制为 $-30kW$ (防止过度充电)。

4.3.2 功率分配规则

根据模式动态分配发动机与电池功率，满足车辆需求功率 P_{req} ：

$$P_{req} = P_{eng} + P_{bat} \quad (4.1)$$

CD 模式：

$$P_{eng} = 0, P_{bat} = P_{req} \quad (\text{电池单独驱动}) \quad (4.2)$$

CS 模式：

$$\text{若 } P_{req} \leq P_{eng_{cs}}$$

$$\text{则 } P_{eng} = P_{req}, P_{bat} = 0 \quad (\text{发动机单独驱动}) \quad (4.3)$$

若 $P_{req} > P_{eng_{cs}}$

则 $P_{eng} = P_{eng_{cs}}$, $P_{bat} = P_{req} - P_{eng}$ (发动机+电池联合驱动) (4.4)

4.3.3 热力学模型(绝热假设)

电池温度仅由内阻发热引起(无冷却系统), 温度变化公式为式(4.5)

$$T_{t+1} = T_t + \frac{Q_{gen} \cdot \Delta t}{C_{th}} \quad (4.5)$$

$Q_{gen} = (I_{bat}^2) \cdot R_0$: 电池内阻发热功率 (I_{bat} 为电池电流, R_0 为内阻)。(4.6)

$C_{th} = m_{bat} \cdot c_{p,bat}$: 电池热容 (m_{bat} 为电池质量, $c_{p,bat}$ 为比热容)。(4.7)

4.3.4 SOC 管理

电池 SOC 根据电流变化更新为式(4.8):

$$SOC_{t+1} = SOC_t - \frac{I_{bat} \cdot \Delta t}{3600 \cdot Cap_{Ah}} \quad (4.8)$$

$I_{bat} = \frac{P_{bat} \cdot 1000}{V_{nom}}$: 电池电流 (V_{nom} 为额定电压)。(4.9)

SOC 约束: 限制在 $[SOC_{min} = 0.2, SOC_{max} = 0.9]$ 范围内(通过 `np.clip` 强制约束)。

4.3.5 燃油消耗计算

仅 CS 模式下 ($p_{eng} > 0$) 计算燃油消耗:

发动机油耗通过油耗插值表 (`eng_fc_interp`) 根据当前功率 P_{eng} 查表得到每秒油耗, 累加得到总油耗。

4.3.6 策略特点

模式切换驱动：以 SOC 下限为阈值，自动在“纯电动”和“发动机恒定功率辅助”间切换，确保 SOC 不低于安全下限。

绝热温度管理：假设电池无散热，温度仅由内阻发热决定，反映电池在无冷却条件下的热行为。

恒定 CS 功率：CS 模式下发动机以固定功率（50kW）工作，简化控制逻辑，同时保证 SOC 维持。

4.3.7 策略小结

CD-CS 策略通过 SOC 阈值触发模式切换，结合纯电动（CD）和发动机恒定功率（CS）模式，实现能量分配；在绝热假设下，温度仅由内阻发热决定，重点保障 SOC 在安全范围内，同时监控温度变化。其核心是“电量消耗 - 电量维持”的二元切换逻辑，适用于对 SOC 控制要求较高的场景。

CD 阶段 SOC 高于目标值 20%时优先由电池承担驱动；SOC 接近20%后进入 CS 阶段，发动机启动并以 50kW（燃油效率最高点）输出给电池充电，当 SOC 高于90%切换到 CD 阶段。CS 阶段，当整车驱动功率大于50kW时，发动机增加功率，满足电机驱动时的用电需求。

4.4 CD-CS 策略工况适应性分析

4.4.1 25℃环境下的能量分配与状态变化

完整 CLTC-P 工况 30 次循环，在初始温度25.0℃，初始SOC = 40%，绝热无冷却条件下，CD-CS 策略的仿真输出如表 4.1 所示。表中列出了终端 SOC、最高温度、燃油消耗、电能消耗和等效油耗，可用于评价该策略的电量消耗及经济性。

表 4.1 CD-CS 策略综合评价表

指标名称	数值
初始 SOC	0.4
环境温度(℃)	25.0

CS 功率 (kW)	50.0
循环次数	30
行驶里程 (km)	434.39
终止 SOC	0.202
终止温度 (°C)	34.5
最高温度 (°C)	34.5
燃油消耗 (L)	52.94
电能净消耗 (kWh)	25.6
百公里油耗 (L/100km)	12.19
百公里电耗 (kWh/100km)	5.9
等效油耗 (L/100km)	15.1

为分析 25 °C环境温度下 CD-CS 策略的功率分配、SOC 维持及电池温升特性，图 4.4.1-4.4.5 分别给出了电池功率、驱动电机功率、发动机功率、SOC 和电池温度的变化曲线。由图可见，车辆前期主要消耗电池电量；当 SOC 下降至控制阈值附近后，发动机开始介入，SOC 逐渐稳定，而电池温度在绝热条件下随运行时间持续升高。

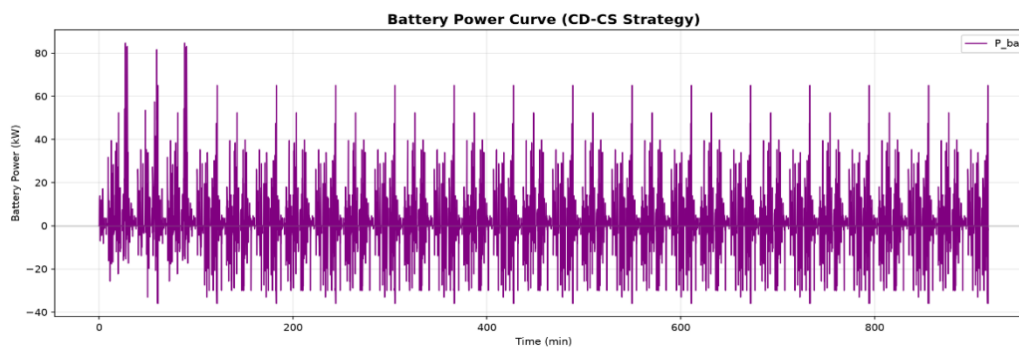


图 4.4.1 电池功率曲线

该曲线反映 CD-CS 策略下动力电池在整个循环中的充放电功率变化。车辆加速和持续驱动时电池主要输出功率，制动减速时出现负功率以回收能量；进入 CS 阶段后，发动机介入分担需求，电池平均放电程度降低，并围绕 SOC 维持目标承担瞬时功率补偿。

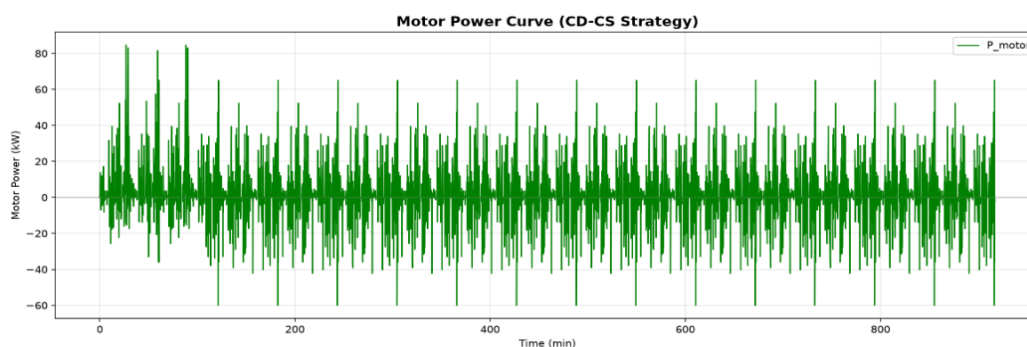


图 4.4.2 驱动电机功率曲线

该曲线表示驱动电机为满足车辆行驶工况所需的瞬时功率。正功率对应牵引驱动，负功率对应再生制动，功率峰值主要出现在加速阶段；由于电机直接承担车轮驱动，其变化规律主要由 CLTC-P 速度和加速度决定。

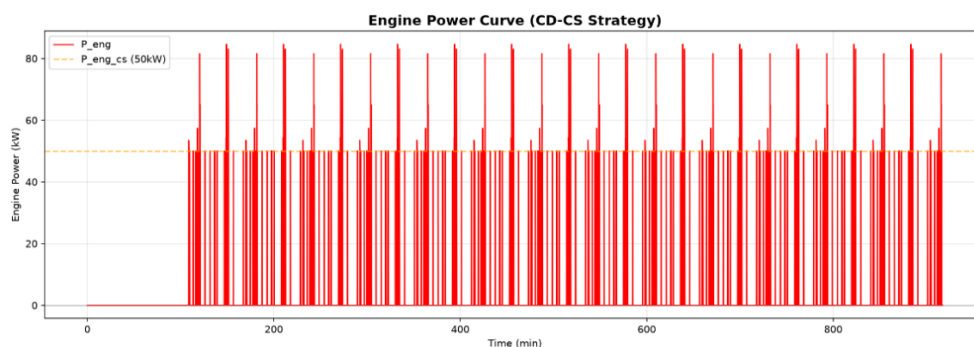


图 4.4.3 发动机功率曲线

该曲线体现发动机在 CD-CS 策略中的启停及输出功率特征。CD 阶段发动机保持关闭，SOC 下降至切换阈值后进入 CS 阶段，发动机在较高效率区域工作并为驱动需求或电池补能提供功率，高负荷时可适当提高输出。

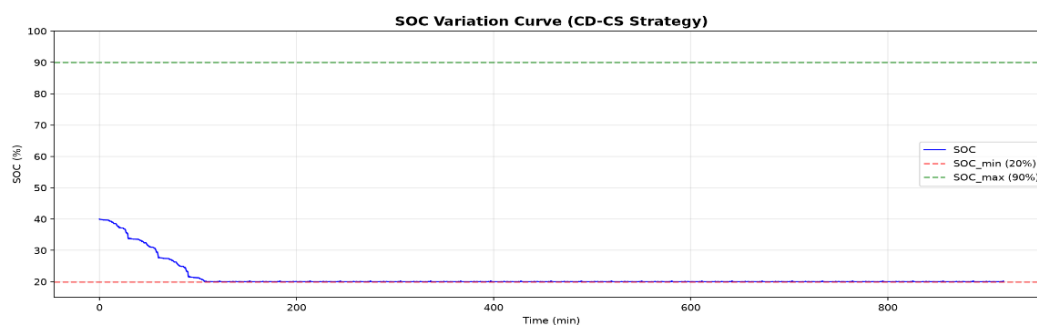


图 4.4.4 SOC 变化曲线

该曲线展示电池 SOC 由电量消耗阶段向电量维持阶段的变化过程。前期车辆优先使用电池，SOC 持续下降；达到设定阈值后发动机介入，SOC 在目标值附近小幅波动，体现 CD-CS 策略“先消耗、后维持”的基本特征。

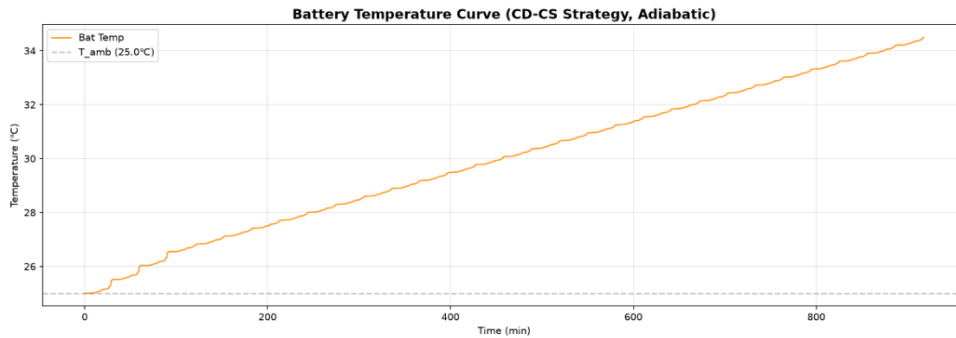


图 4.4.5 电池温度曲线

该曲线反映动力电池在 25 °C 环境、绝热无主动冷却条件下的温升过程。电池内阻损耗随充放电电流不断产生热量，因此温度总体呈累积上升趋势，功率波动较大的阶段温升速度相对更快。

4.4.2 35 °C 环境下的能量分配与状态变化

完整 CLTC-P 工况 30 次循环, 在初始温度 35.0°C, 初始 SOC = 40, 绝热无冷却条件下, CD-CS 策略的仿真输出如表 4.2 所示。表中列出了终端 SOC、最高温度、燃油消耗、电能消耗和等效油耗, 可用于评价该策略的电量消耗及经济性。

表 4.2 CD-CS 策略综合评价表

指标名称	数值
初始 SOC	0.4
环境温度 (°C)	35.0
CS 功率 (kW)	50.0
循环次数	30
行驶里程 (km)	434.39
终止 SOC	0.202
终止温度 (°C)	44.5

最高温度(℃)	44.5
燃油消耗(L)	52.94
电能净消耗(kWh)	25.6
百公里油耗(L/100km)	12.19
百公里电耗(kWh/100km)	5.9
等效油耗(L/100km)	15.13

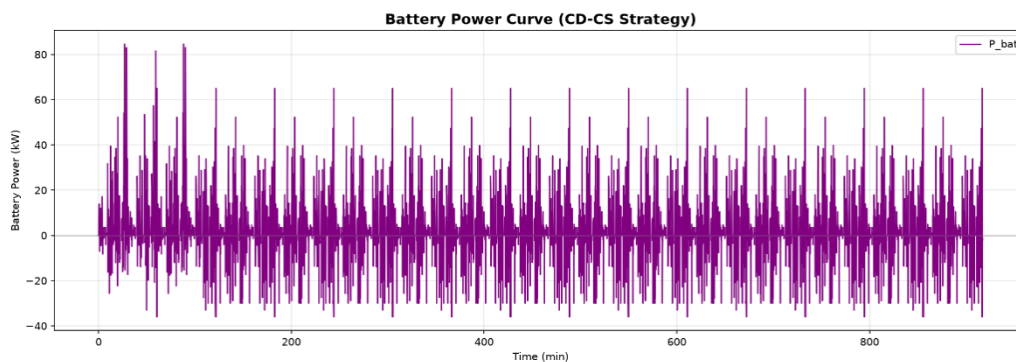


图 4.4.6 电池功率曲线

该曲线表示 35 ℃环境下电池承担的充放电功率。其正负变化仍与驱动、能量回收及发动机介入相对应；环境温度主要影响电池热状态，并不改变车辆需求功率的基本规律，但较高温度会提高限制大电流工作的必要性。

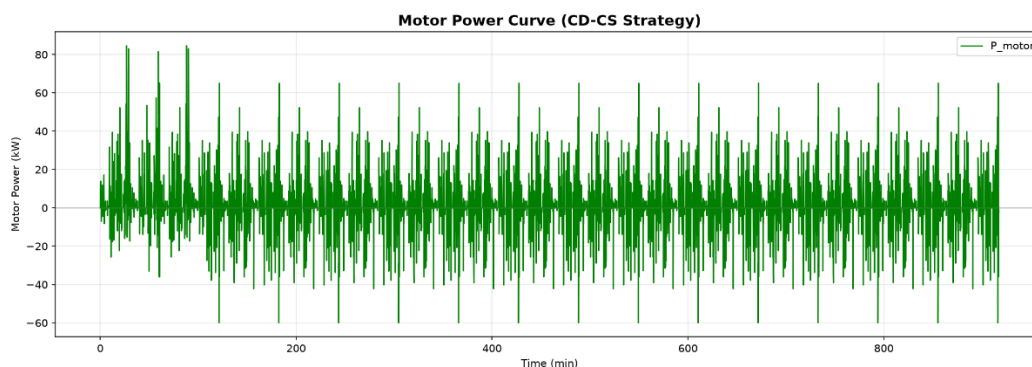


图 4.4.7 电机功率曲线

该曲线反映 35 ℃环境下驱动电机对车辆需求功率的跟随情况。加速时电机输出正功率，减速时回收制动能量，停车或低负荷阶段功率接近零，说明动力系统能够按照工况要求完成牵引与再生制动。

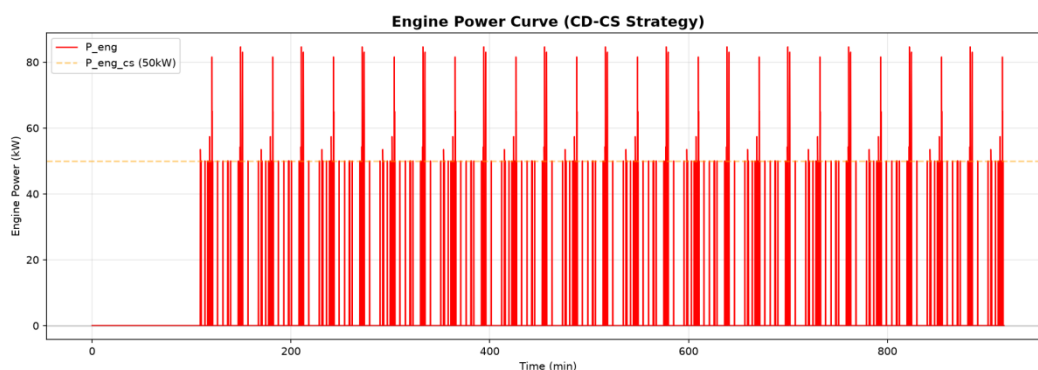


图 4.4.8 发动机功率曲线

该曲线展示 35 °C 环境下发动机在 SOC 阈值触发后的工作过程。发动机在 CD 阶段不参与供能，进入 CS 阶段后以相对稳定的经济功率为主，并在需求功率较大或需要补充电量时提高输出，以减轻电池持续放电。

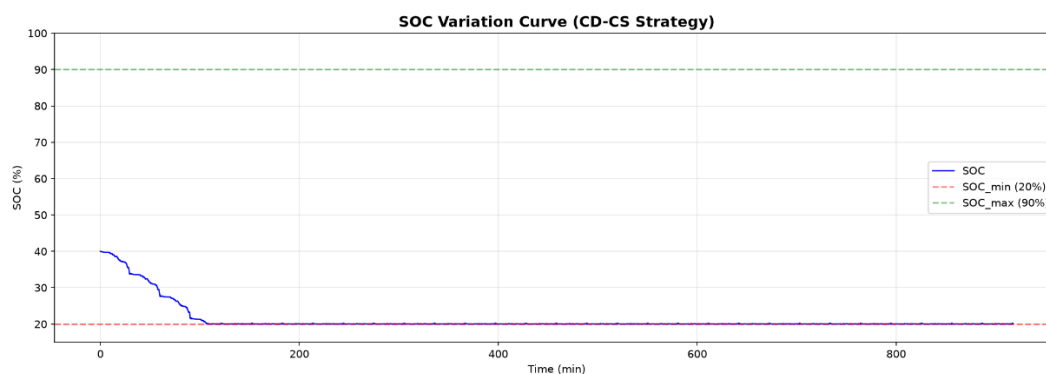


图 4.4.9 SOC 变化曲线

该曲线体现 35 °C 条件下 SOC 的消耗与维持过程。SOC 在前期逐步下降，达到控制下限附近后由发动机参与供能，使其在目标范围内保持稳定，表明环境温度变化不会改变 CD-CS 的核心 SOC 切换逻辑。

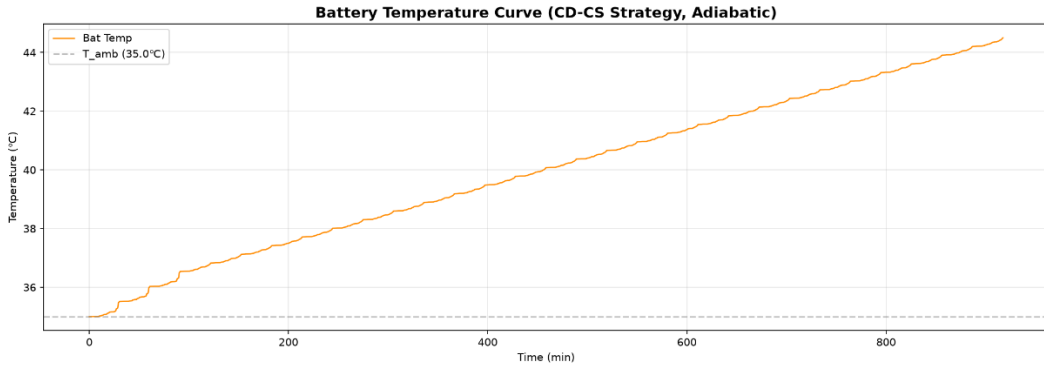


图 4.4.10 电池温度曲线

该曲线表示较高环境温度下电池温度随循环运行的变化。由于初始温度更接近允许上限，电池在相同电流负荷下更早进入高温区域；在无主动冷却的假设下，热量持续积累，温度约束的重要性更加突出。

4.5 普通 ECMS 策略原理

普通 ECMS（等效燃油消耗最小化策略）模型的核心是通过优化发动机与电池的功率分配，最小化等效燃油消耗，同时满足车辆动力需求和电池 SOC 约束。

4.5.1 核心目标与成本函数

ECMS 的目标是最小化等效燃油消耗，成本函数为式（4.10）：

$$Cost = FuelCost + s \cdot ElectricCost \quad (4.10)$$

FuelCost：发动机实际燃油消耗（由发动机功率查表得到）。

ElectricCost：电池电能消耗（折算为等效燃油消耗， s 为等效因子）。

s ：电能与燃油的等效折算系数（反映电能的“燃油价值”）。

4.5.2 功率分配策略

根据车辆需求功率 P_{req} ，在发动机 P_{eng} 和电池 P_{bat} 之间分配功率，满足式（4.11）

$$P_{req} = P_{eng} + P_{bat} \quad (4.11)$$

当 $P_{req} < 0$ （制动能量回收）：发动机关闭（ $P_{eng} = 0$ ），电池充电（ $P_{bat} = \max(P_{req}, -30.0)$ ）。

当 $P_{req} \geq 0$ （驱动需求）：通过扫描发动机功率范围（ $0 \sim P_{eng_{max}}$ ），计算每个组合的成本，选择成本最小的 P_{eng} 和 P_{bat} 。

4.5.3 约束条件

SOC 约束：电池 SOC 需保持在 $[0.5, 0.7]$ 范围内（初始 SOC 为 0.6）。功率约束：发动机功率 $P_{eng} \leq 120kW$ ，电池功率 $P_{bat} \in [-30, 60]kW$ 。

4.5.4 温度模型(绝热假设)

电池温度仅由内阻发热引起（无冷却系统），温度变化公式为式（4.12）：

$$T_{t+1} = T_t + \frac{Q_{gen} \cdot \Delta t}{C_{th}} \quad (4.12)$$

$Q_{gen} = (I_{bat}^2) \cdot R_0$ ：电池内阻发热功率（ I_{bat} 为电池电流， R_0 为内阻）。
(4.13)

$C_{th} = m_{bat} \cdot c_{p,bat}$ ：电池热容（ m_{bat} 为电池质量， $c_{p,bat}$ 为比热容）。
(4.14)

4.5.5 策略小结

普通 ECMS 模型通过等效油耗最小化的目标函数，结合功率分配约束和绝热温度模型，实现了发动机与电池的协同控制，重点优化燃油经济性，同时监控电池 SOC 和温度状态。其核心是成本函数的设计和功率分配的优化逻辑。

普通 ECMS 策略计算每个时刻发动机功率的最小瞬时燃油成本和等效电能成本之和。本包括扫描 1-5 的等效因子，维持终端 SOC 在 0.5 – 0.7 之间，使策略兼顾燃油消耗、电耗和终端 SOC。普通 ECMS 不做电池冷却。

$s = 1$ 时等效油耗最低（13.01L），因为电能被赋予很低的成本，策略优先消耗电池电量，燃油消耗极少（40.84L）。

$s \geq 2$ 后，电能成本权重增大，ECMS 策略转为“发动机主导+主动充电”模式，大量燃油用于发电（净充入149.7kWh电能），导致燃油消耗暴增至180.71L，等效油耗升至24.37。

$s = 2/3/4/5$ 结果完全一致，说明当等效因子 ≥ 2 时，最优解已锁定为“全程发动机驱动+充电至 SOC 上限”，继续增大 s 不再改变决策。

温升方面， $s = 1$ 时电池充放电较均衡，温升8.9℃； $s \geq 2$ 时发动机持续大功率充电，电流更大，温升达12.3℃。

4.6 普通 ECMS 策略参数影响分析

分别完成环境温度 25℃ 条件下，ECMS 等效因子 1-5 和环境温度 35℃ 条件下 ECMS 等效因子 1, 1.2, 1.4, 1.6, 1.8, 2

4.6.1 25 ℃环境下等效因子 1~5 影响分析

CLTC-P 60 次循环， $T=25.0^{\circ}\text{C}$ ，初始 SOC 分 0.4 和 0.6 的普通 ECMS 仿真结果如表 4.3 所示。该表用于比较等效因子对终端 SOC、燃油消耗、电能消耗和电池温度的影响。

表 4.3 ECMS 策略评价表

初始 SOC	等效因子	行驶里程	终止 SOC	终止温度(℃)	最高温度(℃)	燃油消耗(L)	电能净消耗	等效油耗(L/100km)
0.4	1	868.78	0.502	42.2	42.2	91.59	-5.5	10.22
0.4	2	868.78	0.700	44.5	44.5	340.19	-16.3	38.22
0.4	3	868.78	0.700	44.5	44.5	340.19	-16.3	38.22
0.4	4	868.78	0.700	44.5	44.5	340.19	-16.3	38.22
0.4	5	868.78	0.700	44.5	44.5	340.19	-16.3	38.22
0.6	1	868.78	0.502	42.3	42.3	89.11	5.3	10.56

0.6	2	868.78	0.700	44.2	44.2	337.86	-5.4	38.58
0.6	3	868.78	0.700	44.2	44.2	337.86	-5.4	38.58
0.6	4	868.78	0.700	44.2	44.2	337.86	-5.4	38.58
0.6	5	868.78	0.700	44.2	44.2	337.86	-5.4	38.58

为分析普通 ECMS 等效因子对功率分配和状态变化的影响，图 4.6.1-4.6.5 对比了 25℃环境温度、不同初始 SOC 和等效因子条件下的仿真结果。不同等效因子改变了电能目标函数中的等效成本，进而影响发动机介入程度、电池充放电功率、终端 SOC 和电池温升。

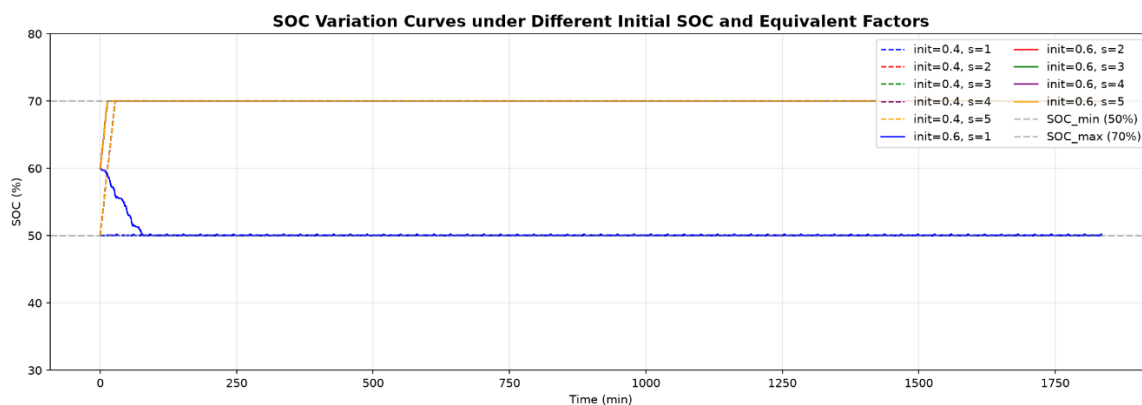


图 4.6.1 SOC 变化对比曲线

该曲线用于比较不同初始 SOC 和等效因子对普通 ECMS 电量使用方式的影响。等效因子较小时策略倾向于使用电池，SOC 下降较快；等效因子增大后电能成本提高，发动机介入增加，SOC 维持能力增强。初始 SOC 决定曲线起点，而参数选择决定长期变化趋势。

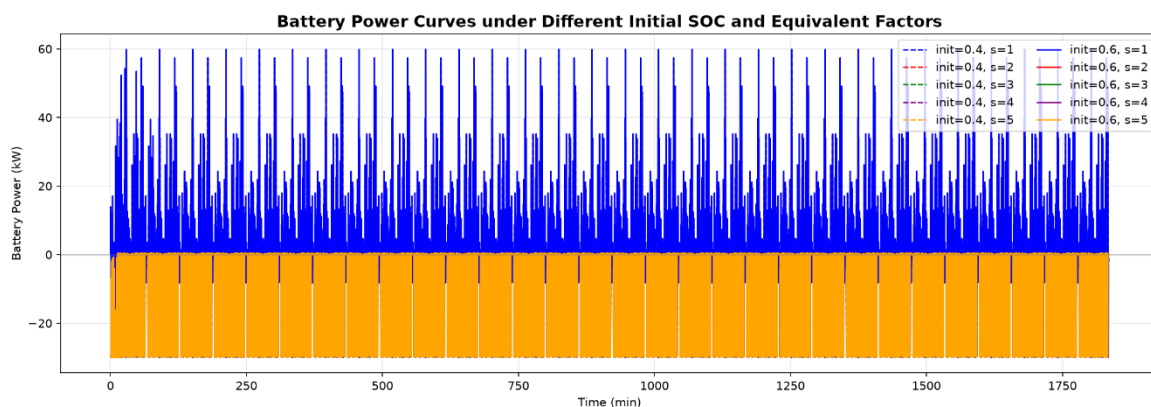


图 4.6.2 电池功率对比曲线

该曲线反映不同等效因子下电池功率分配的差异。较小等效因子通常对应更积极的电池放电，较大等效因子会抑制放电并增加发动机发电或电池充电过程，说明等效因子直接决定燃油与电能之间的使用偏好。

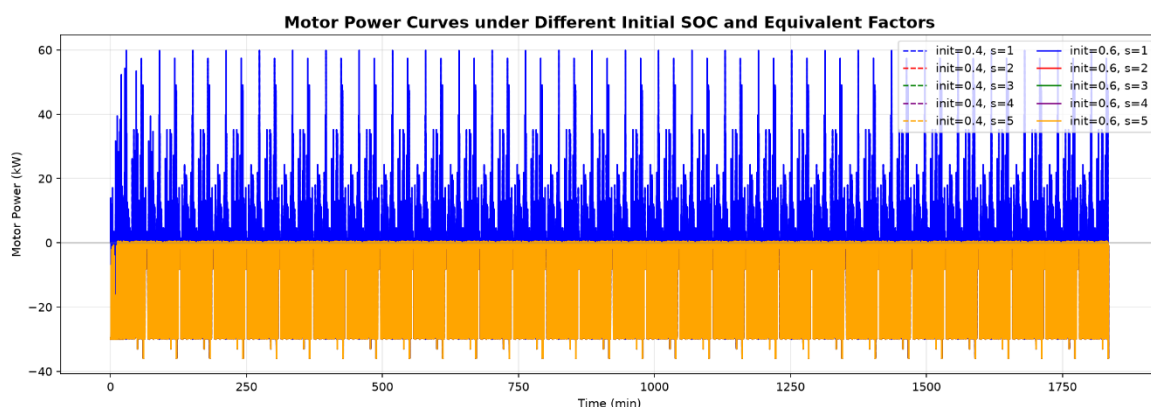


图 4.6.3 电机功率对比曲线

该曲线表示不同参数组合下驱动电机的需求功率。由于各方案采用相同的行驶工况，电机功率整体应保持一致；不同等效因子主要改变电机所需电能由发动机发电还是由动力电池提供，而不改变车轮端的基本动力需求。

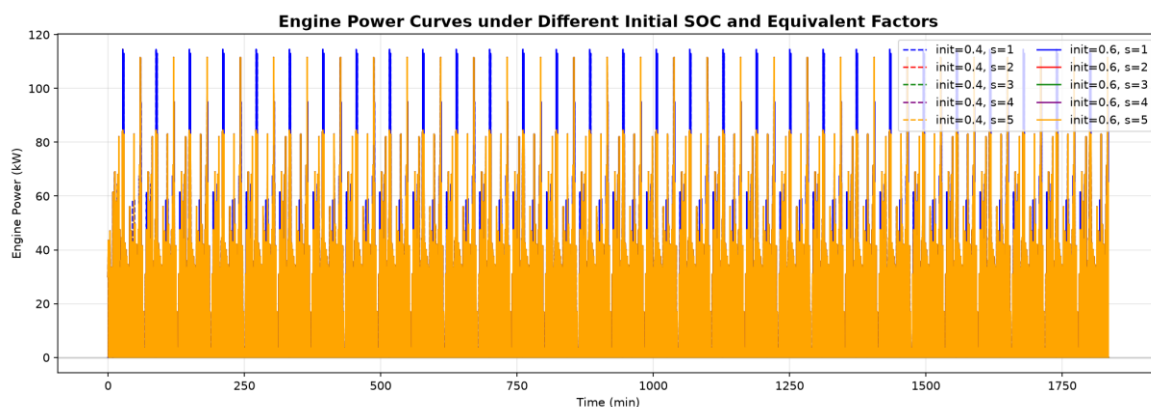


图 4.6.4 发动机功率对比曲线

该曲线展示等效因子对发动机启停频率和输出水平的影响。等效因子较小时发动机使用较少，车辆更多依赖电池；随着等效因子增大，发动机更早、更频繁地进入工作，并可能输出高于瞬时驱动需求的功率为电池补能。

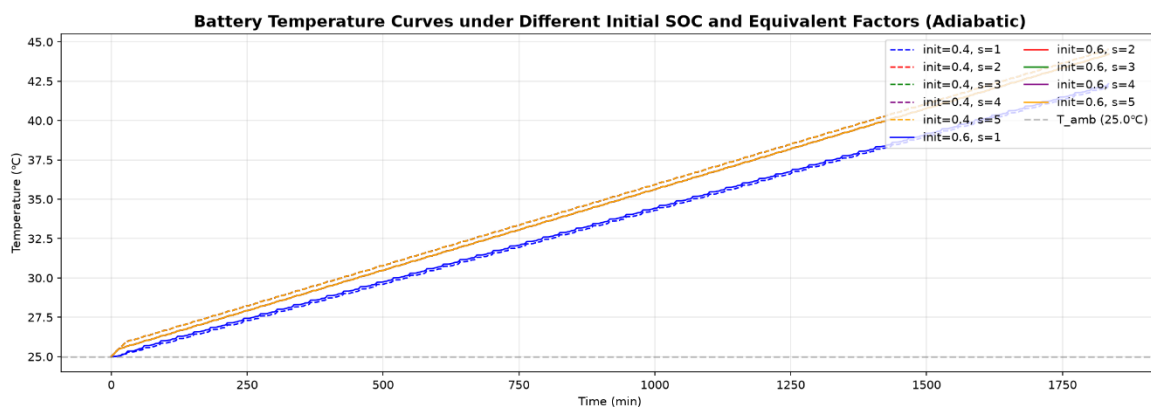


图 4.6.5 电池温度对比曲线

该曲线反映不同功率分配方案造成的电池温升差异。温升主要取决于充放电电流及其持续时间，过度放电或持续大功率充电均会增加内阻发热；合理的等效因子能够避免电池功率长期偏大，从而减缓温度上升。

4.6.2 35 °C环境下等效因子 1~5 的影响分析

CLTC-P 60 次循环， $T=35.0^{\circ}\text{C}$ ，初始 SOC 分 0.4 和 0.6 的普通 ECMS 评价结果如表 4.4 所示。表中结果可用于分析高环境温度下不同能量分配方案的经济性和热状态。

表 4.4 ECMS 策略评价表

初始 SOC	等效因子	行驶里程	终止 SOC	终止温度($^{\circ}\text{C}$)	最高温度($^{\circ}\text{C}$)	燃油消耗(L)	电能净消耗	等效油耗(L/100km)
0.4	1	868.78	0.502	52.2	52.2	91.59	-5.5	10.22
0.4	2	868.78	0.700	54.5	54.5	340.19	-16.3	38.22
0.4	3	868.78	0.700	54.5	54.5	340.19	-16.3	38.22
0.4	4	868.78	0.700	54.5	54.5	340.19	-16.3	38.22
0.4	5	868.78	0.700	54.5	54.5	340.19	-16.3	38.22
0.6	1	868.78	0.502	52.3	52.3	89.11	5.3	10.56
0.6	2	868.78	0.700	54.2	54.2	337.86	-5.4	38.58
0.6	3	868.78	0.700	54.2	54.2	337.86	-5.4	38.58
0.6	4	868.78	0.700	54.2	54.2	337.86	-5.4	38.58
0.6	5	868.78	0.700	54.2	54.2	337.86	-5.4	38.58

图 4.6.6-4.6.10 给出了 35°C 环境温度下普通 ECMS 在等效因子 1~5 范围内的状态与功率对比。该组结果用于分析高环境温度对不同功率分配方案下电池温升的影响。由图可见，环境温度升高不会改变基本功率分配逻辑，但会整体提高电池温度水平。

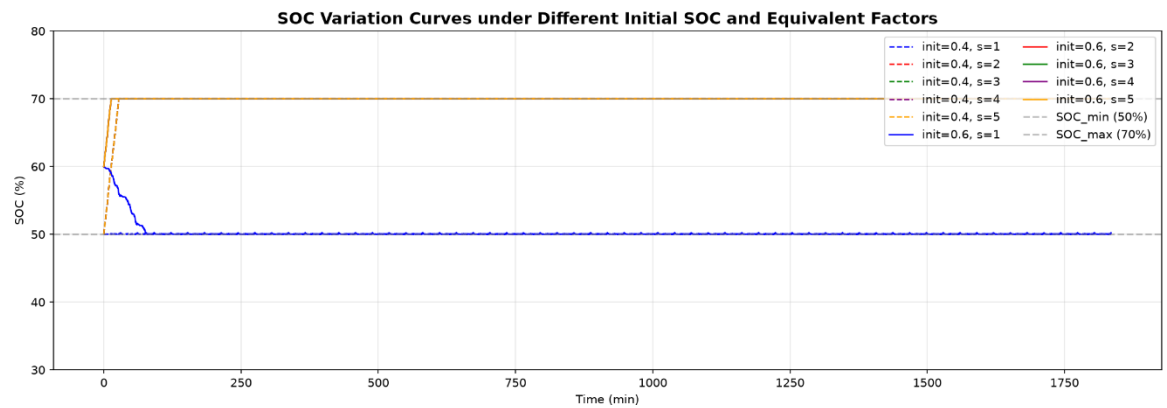


图 4.6.6 SOC 变化对比曲线

该曲线比较 35℃ 环境下等效因子 1~5 对应的 SOC 变化。较低等效因子使策略优先消耗电能，SOC 下降明显；较高等效因子提高发动机供能比例，使 SOC 下降速度减缓，过高时还可能出现持续充电并向 SOC 上限靠近的趋势。

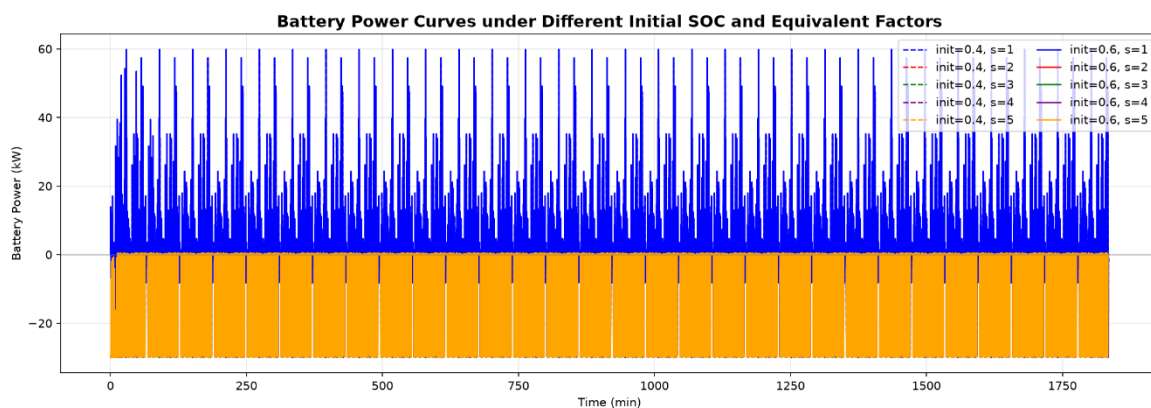


图 4.6.7 电池功率对比曲线

该曲线反映高环境温度下不同等效因子的电池功率分配。等效因子增大时，电池正向放电功率总体受到抑制，负向充电功率出现频率增加；这表明 ECMS 通过改变电能等效成本在电池驱动和发动机发电之间进行权衡。

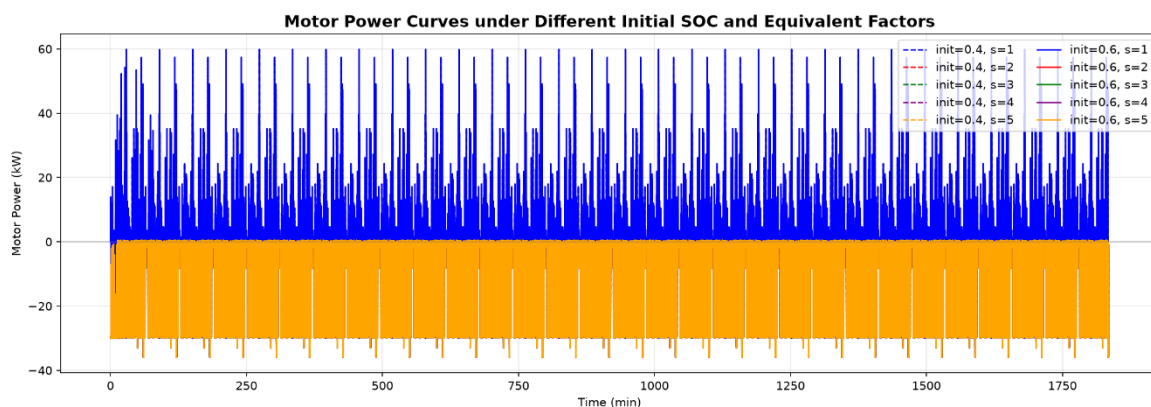


图 4.6.8 电机功率对比曲线

该曲线表示各等效因子方案下驱动电机对 CLTC-P 需求功率的响应。各曲线的整体轮廓主要由车辆工况决定，参数变化不应改变电机完成牵引和再生制动的任务，只改变其上游能量来源。

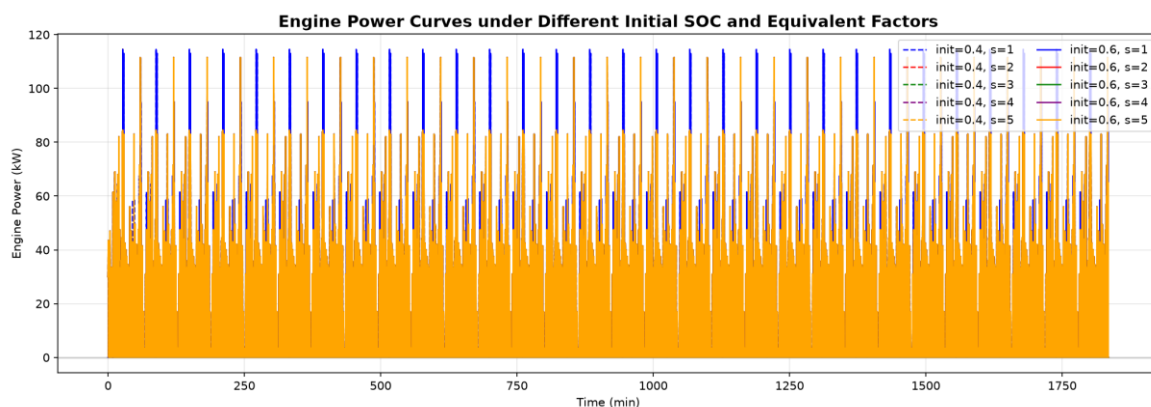


图 4.6.9 发动机功率对比曲线

该曲线展示 35 °C 环境下等效因子对发动机功率的调节作用。等效因子越大，电池能量在成本函数中越昂贵，发动机工作时间和平均输出通常越高；当参数超过一定范围后，最优决策可能趋于稳定，继续增大参数对功率分配的影响减弱。

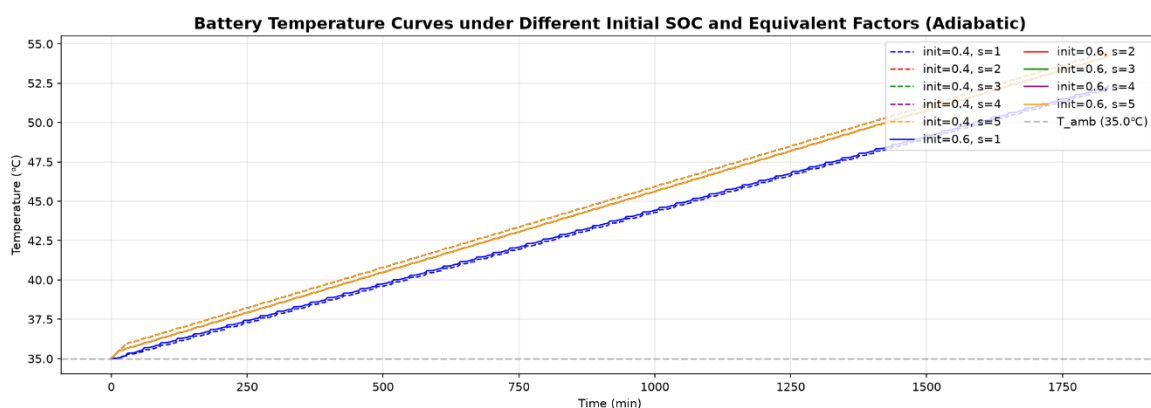


图 4.6.10 电池温度对比曲线

该曲线反映不同等效因子在 35 °C 环境下引起的电池温度变化。较高初始环境温度使所有方案的温度水平整体升高，而不同充放电电流决定附加温升；因此仅优化燃油和电能成本仍可能出现较高温度，需要进一步引入热约束。

4.6.3 35 °C 环境下细化等效因子影响分析

为进一步确定普通 ECMS 的参数变化区间，对等效因子进行细化扫描，评价结果如表 4.5 所示。该表反映了等效因子由 1.0 增大至 2.0 时终端 SOC、油耗和温度的变化规律。

表 4.5 ECMS 策略评价表

初始 SOC	等效因子	行驶里程	终止 SOC	终止温度(℃)	最高温度(℃)	燃油消耗(L)	电能净消耗	等效油耗(L/100km)
0.4	1.0	868.78	0.502	52.2	52.2	91.59	-5.5	10.22
0.4	1.2	868.78	0.509	51.1	51.1	99.43	-5.9	11.10
0.4	1.4	868.78	0.700	50.6	50.6	141.76	-16.3	15.38
0.4	1.8	868.78	0.700	54.5	54.5	340.19	-16.3	38.22
0.4	2.0	868.78	0.700	54.5	54.5	340.19	-16.3	38.22
0.6	1.0	868.78	0.502	52.3	52.3	89.11	5.3	10.56
0.6	1.2	868.78	0.509	51.2	51.2	96.79	5.0	11.43
0.6	1.4	868.78	0.700	50.5	50.5	140.33	-5.4	15.84
0.6	1.8	868.78	0.700	54.2	54.2	337.86	-5.4	38.58
0.6	2.0	868.78	0.700	54.2	54.2	337.86	-5.4	38.58

为进一步观察普通 ECMS 在策略切换区间内的变化规律，将等效因子细化为 1.0、1.2、1.4、1.6、1.8 和 2.0，仿真结果如图 4.6.11~图 4.6.15 所示。细化扫描能够更清楚地反映等效因子变化对 SOC 目标、发动机介入程度及电池温升的影响，并为选择较优等效因子提供依据。

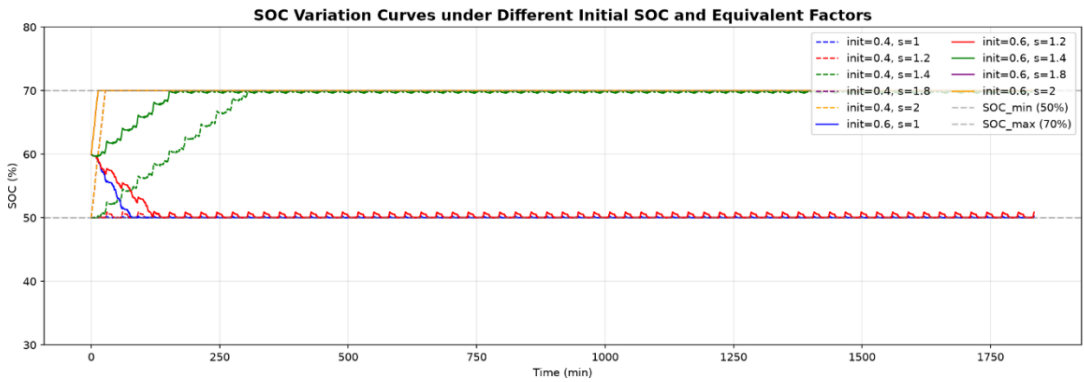


图 4.6.11SOC 变化对比曲线

该曲线通过细化等效因子间隔，展示 SOC 变化由“电池优先”向“发动机优先”逐步过渡的过程。细化扫描有助于识别 SOC 终值对参数最敏感的区间，并选择既不过度放电、也不过度充电的折中等效因子。

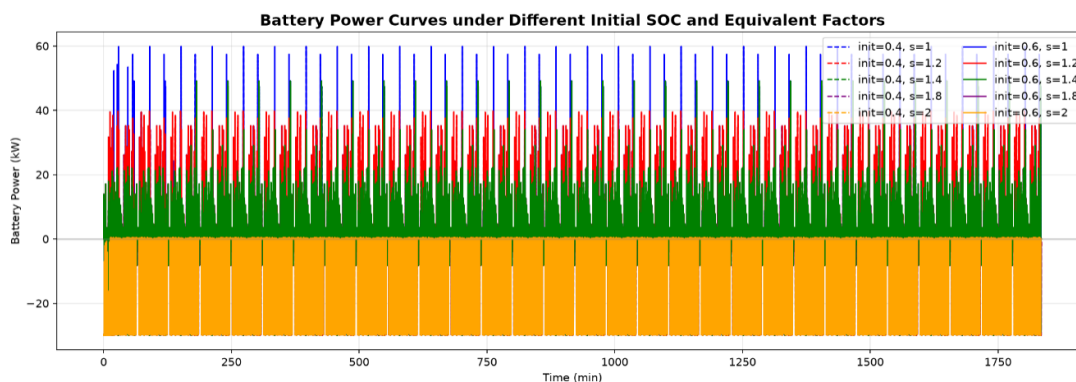


图 4.6.12 电池功率对比曲线

该曲线反映细化等效因子对电池功率均值和峰值分布的影响。随着等效因子逐步增加，电池放电比例下降、发动机供能比例上升，功率分配呈连续变化，可用于判断参数调整对电池负荷的边际影响。

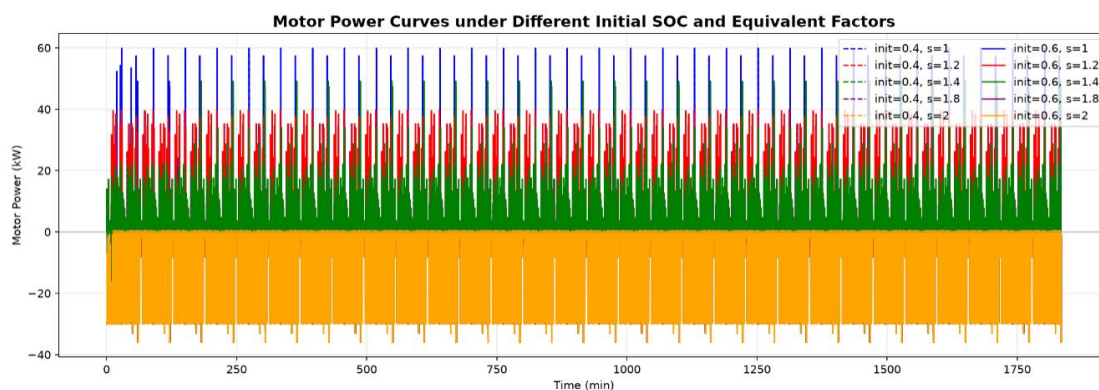


图 4.6.13 电机功率对比曲线

该曲线表示细化参数扫描下的驱动电机功率。由于车辆速度工况和纵向动力学参数保持不变，电机功率需求基本不随等效因子改变，从而说明不同方案的经济性差异来源于发动机与电池之间的功率分配。

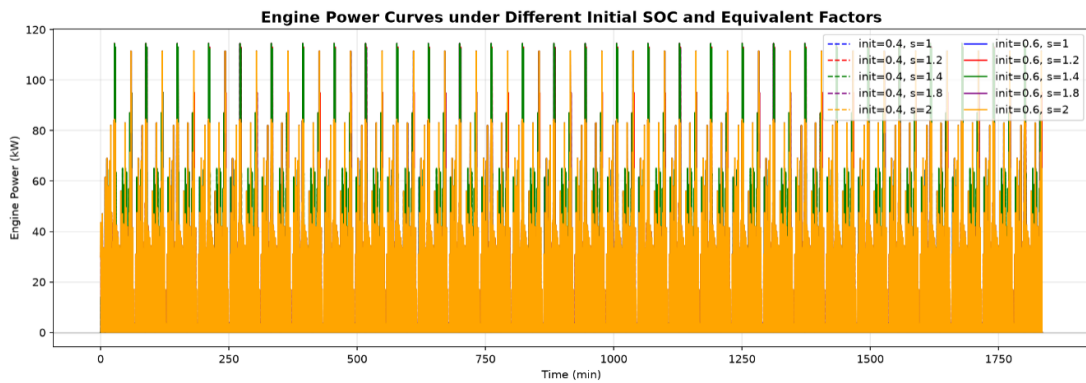


图 4.6.14 发动机功率对比曲线

该曲线展示细化等效因子范围内发动机功率控制的渐变特征。参数增大后发动机介入程度逐步提高，输出功率更倾向于经济工作区并承担电池补能任务；该变化可与终端 SOC 和综合油耗共同用于确定较优参数。

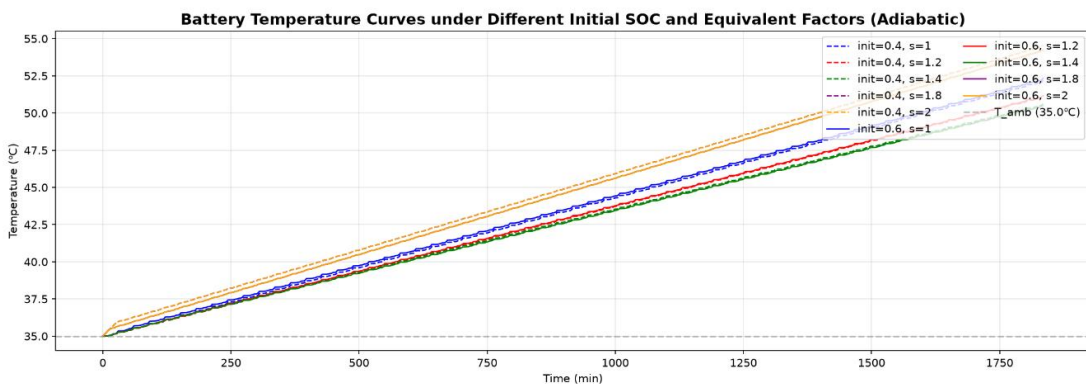


图 4.6.15 电池温度对比曲线

该曲线反映细化等效因子对电池温度的影响。不同参数通过改变电池电流的方向、幅值和持续时间影响内阻发热，温度变化通常不会与燃油消耗完全同步，因此参数选择还需兼顾热状态而不能只追求最低瞬时等效油耗。

4.6.4 等效因子与 SOC 修正系数联合寻优

等效因子与 SOC 修正系数的参数扫描结果如表 4.6 所示。该表按照终端 SOC 偏差优先、等效油耗次优的原则，给出了不同基础等效因子对应的较优修正系数。每个等效因子 s_0 的最优 SOC 修正系数 Kp 。

筛选条件：优先 SOC 偏差 < 0.02 且等效油耗最低。

表 4.6 不同基础等效因子对应的较优 SOC 修正系数

基础等效因子(S_0)	修正系数(Kp)	终端 SOC	SOC 偏差	等效油耗($L/100 km$)
1.0	0.0	0.5896	0.0104	7.05
1.2	0.0	0.5896	0.0104	7.05
1.4	0.0	0.5896	0.0104	7.05
1.8	5.0	0.6308	0.0308	22.67
2.0	5.0	0.6471	0.0471	29.59

4.7 本章小结

本章在统一的 CLTC-P 循环和车辆参数条件下，对 CD-CS 基准策略与普通 ECMS 策略进行了仿真。CD-CS 策略依靠 SOC 阈值实现模式切换，控制逻辑清晰，但发动机工作点和电池功率分配较为固定；普通 ECMS 通过等效因子在燃油消耗与电能消耗之间进行权衡，对参数选取更为敏感。参数扫描结果表明，等效因子及 SOC 修正系数会显著影响终端 SOC、发动机介入程度和综合油耗，因此需要结合终端 SOC 约束选择较优参数组合。

5 热约束 ECMS 策略仿真分析

5.1 热约束 ECMS 策略原理

5.1.1 温度惩罚函数定义

温度惩罚基于温度偏离安全区间（35°C~45°C）的程度，采用平方惩罚机制，公式为：

$$\Phi(T) = \begin{cases} wT \cdot (T - Thigh)^2, T > Thigh & \text{(高温惩罚)} \\ wT \cdot (Tlow - T)^2, T < Tlow & \text{(低温惩罚)} \\ 0, Tlow \leq T \leq Thigh & \text{(安全区间无惩罚)} \end{cases} \quad (5.1)$$

式（5.1）中：

wT ：温度惩罚权重参数（自定义，扫描范围为[0.0,0.5,1.0,2.0,5.0]）

$Tlow = 35^\circ\text{C}$ ：温度下限。

$Thigh = 45^\circ\text{C}$ ：温度上限。

5.1.2 惩罚项在 ECMS 中的集成

惩罚项 $\Phi(T)$ 被加入到 ECMS 的成本函数中，与电池功率的绝对值相乘，影响功率分配决策：当温度越界（ $T > 45^\circ\text{C}$ 或 $T < 35^\circ\text{C}$ ）时， $\Phi(T) > 0$ ，电池功率越大，惩罚成本越高。惩罚成本越高，策略越倾向于减少电池大功率充放电，转而使用发动机，从而控制温度。

5.1.3 模型核心逻辑

高温控制：当 $T > 45^\circ\text{C}$ ，惩罚项随温度偏离程度平方增长，促使策略优先用发动机（减少电池发热）。

低温控制：当 $T < 35^\circ\text{C}$ ，惩罚项随温度偏低程度平方增长，促使策略通过发动机加热或电池充电（避免过度放电）。

权重作用： wT 越大，惩罚强度越高，温度控制越严格（如 $wT = 5.0$ 时，策略对温度越界更敏感）。

5.1.4 与冷却系统的协同

冷却功率（ $0.2kW$ ）作为附加负载加载到整车需求，同时冷却过程带走电池热量，与温度惩罚共同实现温度管理：

冷却启动条件： $T \geq 45^{\circ}\text{C}$ （滞回控制）。

冷却能耗计入总电耗，影响策略的经济性权衡。

5.1.5 策略小结

温度惩罚权重模型通过平方惩罚函数量化温度偏离成本，结合 ECMS 的功率分配决策，实现燃油经济性与电池温度稳定性的平衡。权重 wT 控制惩罚强度，扫描不同 wT 分析其对温度控制效果（如越界时间）和能耗（油耗、电耗）的影响。

5.2 热约束 ECMS 温度控制特性分析

为突出温度约束对功率分配的影响，热约束仿真中将动力电池等效内阻调整为 $0.15\ \Omega$ ，以增强高功率充放电过程中的发热特征。该参数仅用于策略对比与敏感性分析，不代表实际电池包的固定内阻。

5.2.1 $25\ ^{\circ}\text{C}$ 环境下温度惩罚权重影响分析

等效因子 $s=1.6$ ，CLTC-P 30 次， $T=25.0^{\circ}\text{C}$ ，温度范围 $35\text{--}45^{\circ}\text{C}$ ，不同初始 SOC 和温度惩罚权重的热约束 ECMS 评价结果如表 5.1 所示。该表综合比较了油耗、电耗、冷却能耗、终端 SOC 偏差和温度越界时间。

表 5.1 25 ℃下不同温度惩罚权重的热约束 ECMS 评价结果

初始 SOC	热惩罚权重 (w_T)	油耗 ($L/100\ km$)	电耗 ($kW\ h/100\ km$)	冷却能耗 ($kW\ h/100\ km$)	终端 SOC 偏差	温度越界时间 (min)
0.4	0.0	27.81	-17.58	0.73	0.3	149.95
0.4	0.5	22.76	-6.31	0.00	0.3	915.00
0.4	1.0	22.76	-6.31	0.00	0.3	915.00
0.4	2.0	22.76	-6.31	0.00	0.3	915.00
0.4	5.0	22.76	-6.31	0.00	0.3	915.00
0.6	0.0	27.31	-16.86	0.72	0.1	169.20
0.6	0.5	22.75	-6.30	0.00	0.1	915.00
0.6	1.0	22.75	-6.30	0.00	0.1	915.00
0.6	2.0	22.75	-6.30	0.00	0.1	915.00
0.6	5.0	22.75	-6.30	0.00	0.1	915.00

为研究温度惩罚权重对热约束 ECMS 控制效果的影响，图 5.2.1-5.2.5 给出了 25 ℃环境温度、不同初始 SOC 及温度惩罚权重下的仿真结果。该组曲线反映了温度惩罚项对 SOC 调节、电池充放电功率、发动机输出及电池温升的综合影响。

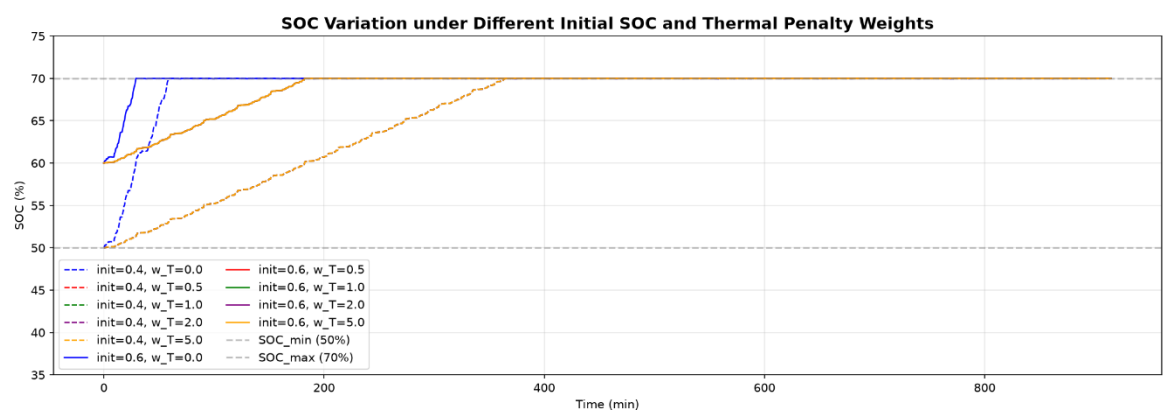


图 5.2.1 SOC 变化对比曲线

该曲线比较 25 ℃环境下不同温度惩罚权重对应的 SOC 变化。权重较小时策略接近普通 ECMS，电池使用较积极；权重增大后，大功率充放电的附加成本提

高，发动机分担比例增加，SOC 下降速度通常减缓，从而为温度控制保留更多电量调节空间。

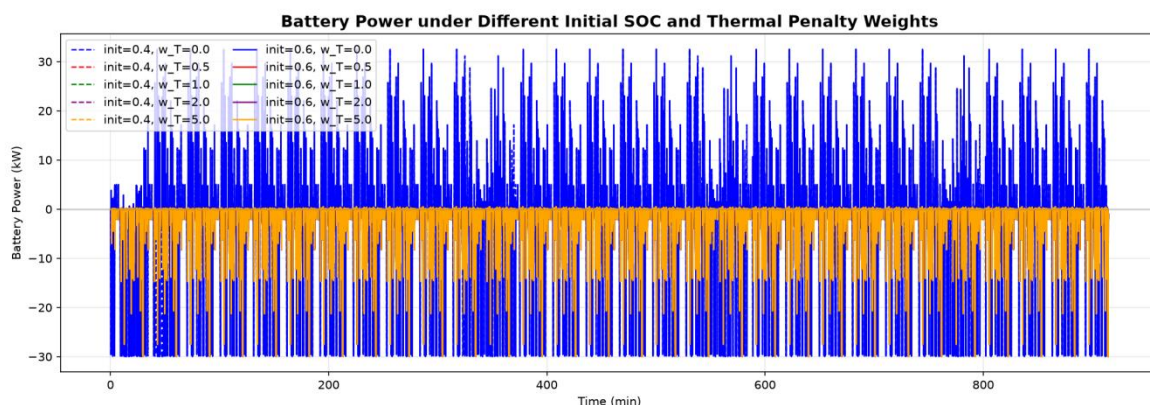


图 5.2.2 电池功率对比曲线

该曲线反映温度惩罚权重对电池功率的直接约束作用。随着权重提高，策略会在电池接近温度限制时减少大幅度充放电，将部分需求转移给发动机；权重过大则可能导致发动机使用增加，因此需要在热安全与经济性之间折中。

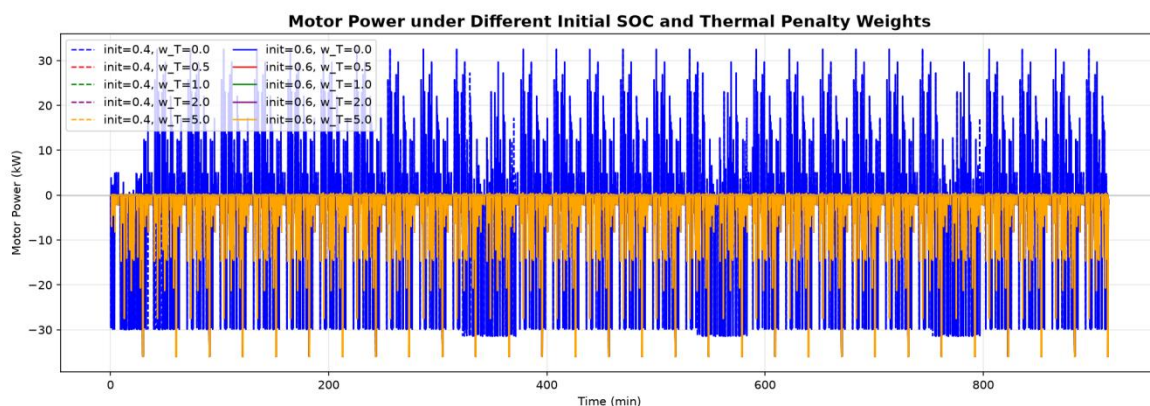


图 5.2.3 电机功率对比曲线

该曲线表示不同温度权重下驱动电机对车辆需求功率的响应。电机仍需完整跟随 CLTC-P 工况完成牵引和再生制动，温度惩罚并不降低车辆动力需求，而是改变驱动电能从发动机发电与电池之间的来源。

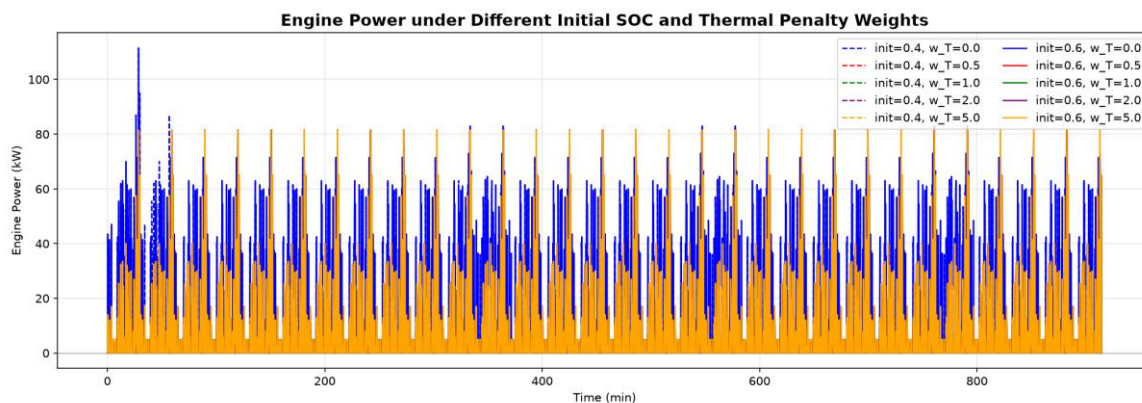


图 5.2.4 发动机功率对比曲线

该曲线展示温度惩罚对发动机运行时序和输出功率的影响。电池温度升高或权重增大时，发动机更倾向于提前介入并承担较多功率，以降低电池电流和发热；在温度较低时，策略仍可利用电池提高整体经济性。

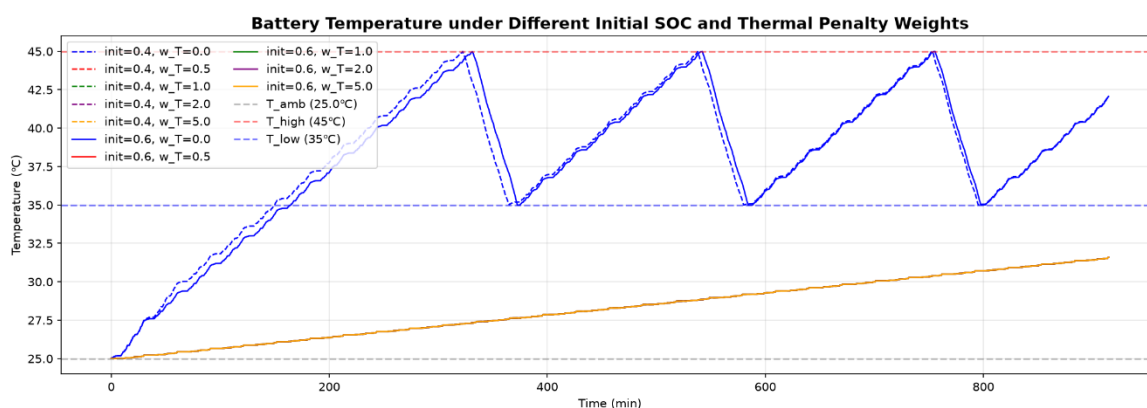


图 5.2.5 电池温度对比曲线

该曲线反映不同温度惩罚权重下电池温度的受控变化。增大权重能够提前抑制高电流工况，并配合冷却系统降低温升和高温持续时间；当温度已得到有效限制后，继续提高权重的降温收益会逐渐减小，同时可能增加燃油或冷却能耗。

5.2.2 35 °C 环境下温度惩罚权重影响分析

等效因子 $s=1.6$ ，CLTC-P 30 次， $T=35.0^{\circ}\text{C}$ ，温度范围 $35\text{--}45^{\circ}\text{C}$ ，热约束 ECMS 的评价结果如表 5.2 所示。该表重点反映高温环境下温度惩罚权重对冷却能耗和温度越界时间的影响。

表 5.2 35 °C 下不同温度惩罚权重的热约束 ECMS 评价结果

初始 SOC	热惩罚权重 (w_T)	油耗 (L/100km)	电耗 (kWh/100km)	冷却能耗 (kWh/100km)	终端 SOC 偏差	温度越界时间 (min)
0.4	0.0	27.45	-17.60	0.20	0.3	4.88
0.4	0.5	27.45	-17.60	0.20	0.3	4.88
0.4	1.0	27.45	-17.60	0.20	0.3	4.88
0.4	2.0	27.45	-17.60	0.20	0.3	4.88
0.4	5.0	27.45	-17.60	0.20	0.3	4.88
0.6	0.0	26.94	-16.86	0.19	0.1	0.33
0.6	0.5	26.94	-16.86	0.19	0.1	0.33
0.6	1.0	26.94	-16.86	0.19	0.1	0.33
0.6	2.0	26.94	-16.86	0.19	0.1	0.33
0.6	5.0	26.94	-16.86	0.19	0.1	0.33

图 5.2.6-5.2.10 给出了 35 °C 环境温度下热约束 ECMS 的状态及功率变化。由于环境温度更接近冷却开启阈值，电池温度更容易进入受限区域，冷却系统及温度惩罚项对功率分配的影响更加明显。

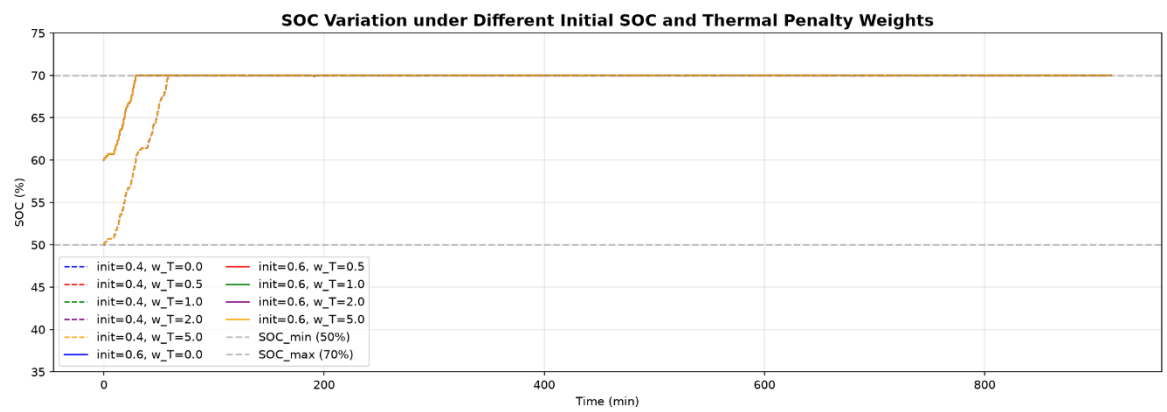


图 5.2.6 SOC 变化对比曲线

该曲线比较 35 °C 环境下不同温度惩罚权重对应的 SOC 轨迹。由于初始温度更接近控制阈值，热约束更早参与功率分配；较大权重通常减少电池持续放电，使 SOC 下降更缓，并通过发动机供能维持电池的热调节能力。

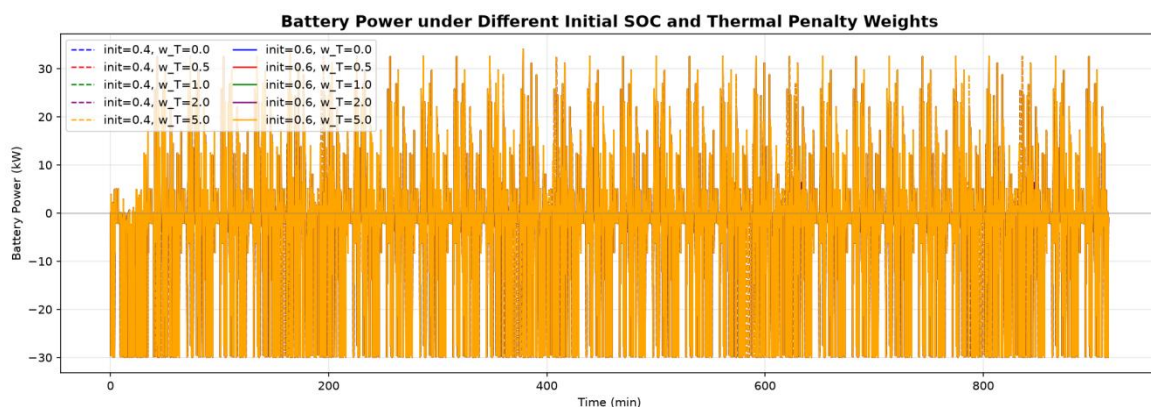


图 5.2.7 电池功率对比曲线

该曲线表示高环境温度下电池功率受温度权重调节的情况。温度接近上限时，较大的惩罚权重会明显压缩电池充放电幅值和持续时间，必要时由发动机承担更多驱动及冷却附加功率，以避免电池热量继续快速积累。

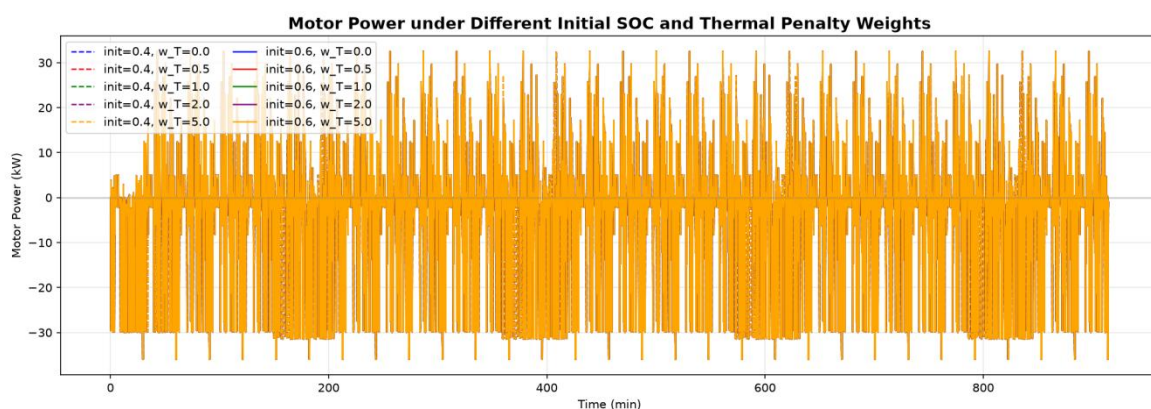


图 5.2.8 电机功率对比曲线

该曲线反映 35 °C 环境下驱动电机功率随工况的变化。不同温度惩罚权重不会改变车辆所需的牵引和制动功率，因此电机功率的总体变化规律一致，策略差异主要体现在发动机和电池对电机供能的比例。

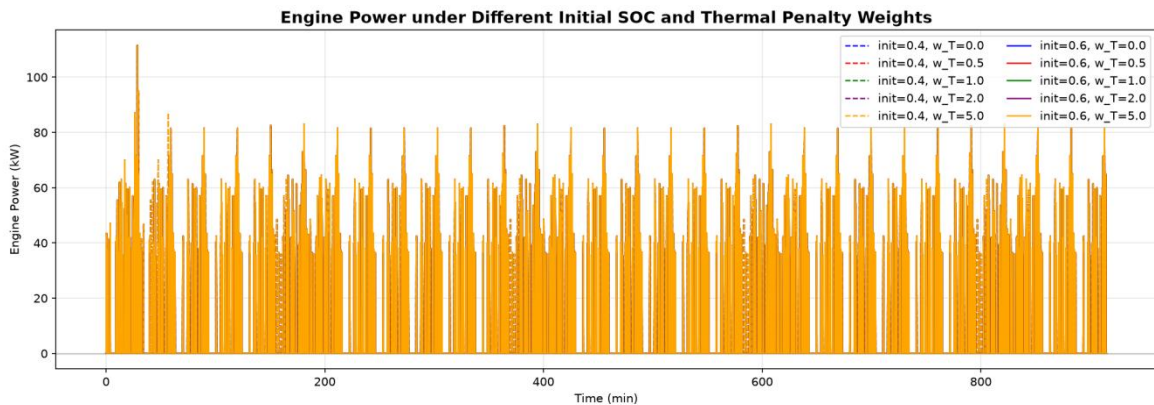


图 5.2.9 发动机功率对比曲线

该曲线展示高温条件下发动机对热约束的响应。权重增大后，发动机在电池温度接近阈值时更频繁地工作，并通过提高供能比例减少电池电流；这种主动介入能够改善热状态，但会带来一定燃油消耗，需要结合综合指标评价。

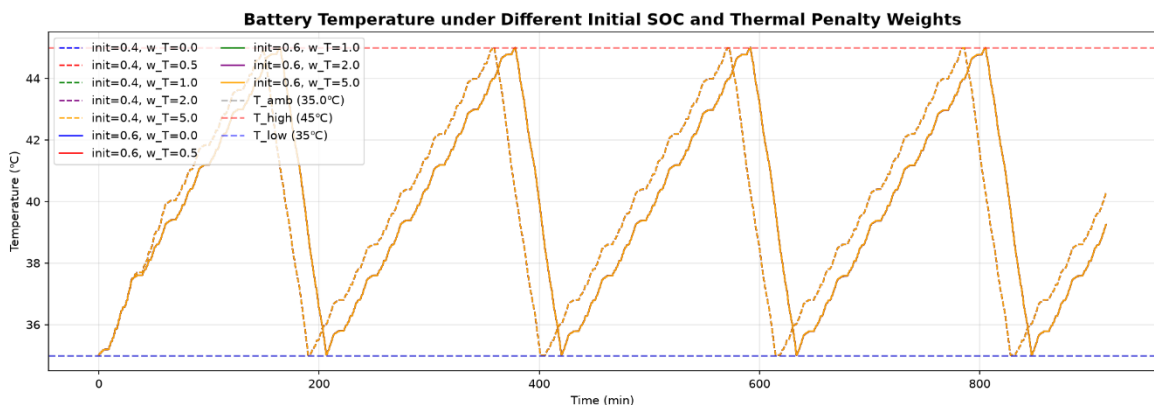


图 5.2.10 电池温度对比曲线

该曲线表示 35 °C 环境下温度惩罚与主动冷却共同作用后的电池温度变化。较高权重可降低峰值温度并缩短温度越界时间，冷却系统则在达到开启条件后带走热量；合理参数应在满足温度约束的同时控制冷却附加能耗。

5.2.3 温度惩罚权重与 SOC 修正系数联合寻优

不同等效因子和温度惩罚权重的联合扫描结果如表 5.3 所示。该表按照温度越界时间优先、综合油耗次优的原则，选取各等效因子对应的较优温度惩罚权重。评价标准是温度越界时间最短对应的综合油耗最低

表 5.3 不同等效因子对应的较优温度惩罚权重

等效因子	温度惩罚 w_T	综合油耗 (L/100km)	燃油 (L/100km)	电耗 (kWh/100km)	冷却能耗 (kWh/100km)	终端 SOC 偏差	温度越界时间
1.0	0.0	7.05	0.00	14.09	0.0	0.0312	2340.0
1.2	0.0	7.05	0.00	14.09	0.0	0.0312	2340.0
1.4	0.0	7.05	0.00	14.09	0.0	0.0312	2340.0
1.6	0.5	14.09	16.72	-5.26	0.0	0.0117	2340.0
1.8	0.5	14.09	16.72	-5.26	0.0	0.0117	2340.0
2.0	0.5	14.09	16.72	-5.26	0.0	0.0117	2340.0

参数调整逻辑说明：

$R_0 = 0.1$ ：在这个内阻下，电池在30kW放电时产热约722W，在60kW放电时产热约 2890W。这意味着小功率时冷却系统能压住温度，大功率时温度会快速上升。

$T_{high} = 45.0, T_{low} = 40.0$ ：保持温度上限为 45 度，下限设为 40 度。冷却系统在 45 度开启，40 度关闭。

$P_{cool_max} = 1.51500W$ 的冷却功率。当电池大功率发热 ($> 1500W$) 时，即使冷却全开，温度依然会超过 45 度。此时如果不加惩罚 ($w_T=0$)，温度会长时间停留在 45 度以上。当加入惩罚后，策略会主动降低电池功率，发热量骤减，冷却系统就能把温度拉回 45 度以下，从而缩短越界时间。这样就能找到非零的 w_T 最优解。

5.3 三种能量管理策略综合性能对比

对比不同策略下的 SOC、电池温度、冷却能耗、燃油消耗和综合评价指标。在相同工况和参数条件下，CD-CS、普通 ECMS 及热约束 ECMS 的综合性能如表 5.4 所示。该表从燃油消耗、电能消耗、冷却能耗、终端 SOC、最高温度和温度越界时间等方面比较三种策略的控制效果。

表 5.4 三种能量管理策略综合性能对比

策略	行驶里程	燃油消耗 (L)	燃油 (L/100 km)	电耗 (kW h/100 km)	冷却能耗 (kW h/100 km)	综合油耗 (L/100 km)	终端 SOC	终端温度	最高温度	温度越界时间
CD_CS	12.04	0.00	0.00	14.09	0.0	7.05	0.569	35.3	35.3	2340.0
ECMS	12.04	5.24	43.56	-42.48	0.0	22.32	0.694	36.5	36.5	2340.0
Thermal_ECMS	12.04	2.01	16.72	-5.26	0.0	14.09	0.612	35.0	35.0	2340.0

为直观比较 CD-CS、普通 ECMS 和热约束 ECMS 的控制差异，三种策略下的 SOC、电池温度、发动机功率和电池功率如图 5.3 所示。由图可分析不同策略在电量使用、发动机介入及温度控制方面的差异，其中热约束 ECMS 通过调整发动机与电池的功率分配，降低电池在高温区间内的大功率运行程度。

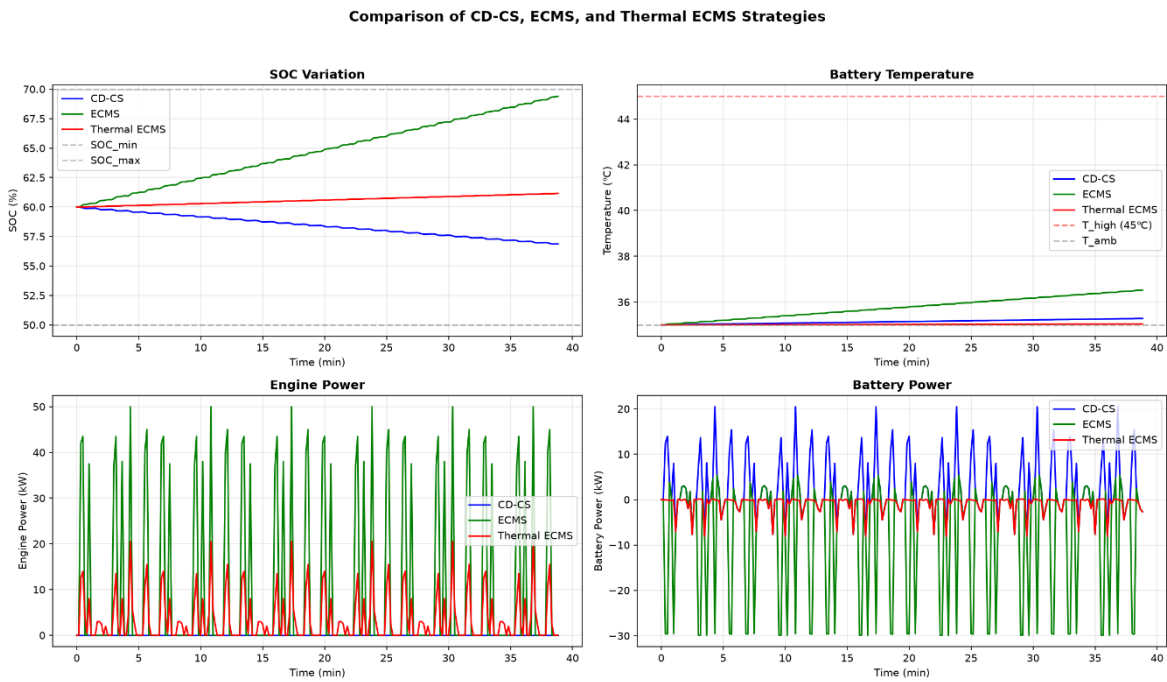


图 5.3 三种能量管理策略综合性能对比

该图用于从燃油消耗、电能消耗、终端 SOC、电池最高温度和冷却能耗等方面综合比较三种策略。CD-CS 控制简单但功率分配灵活性有限，普通 ECMS 侧重

能量经济性，热约束 ECMS 进一步兼顾电池温度和冷却代价，因此评价时应关注多项指标的整体平衡，而不能仅依据单一油耗指标判断优劣。

策略逻辑与对比说明：

CD-CS：前期纯电消耗 SOC 至 0.55，随后发动机强制介入提供主驱动功率，电池仅做削峰填谷，SOC 维持在低水平。

普通 ECMS：使用固定等效因子 $s = 1.6$ 进行全局优化分配，不考虑温度惩罚 ($w_T = 0$)，但同样受物理冷却系统的约束。

热约束 ECMS：在普通 ECMS 基础上引入温度惩罚 $w_T = 5.0$ ，策略会主动限制大电流以降低发热。

综合评价：**综合油耗(L) = 燃油(L) + (电耗 + 冷却能耗) * 0.5折算系数**，确保能源消耗在同一维度比较。

5.4 本章小结

本章在普通 ECMS 基础上引入温度惩罚和冷却控制，并在 25 °C 与 35 °C 环境下完成参数扫描和策略对比。结果表明，温度惩罚权重会改变发动机与电池的功率分配，使策略在电池接近温度上限时主动降低大电流充放电；冷却系统则通过附加能耗换取温度约束的满足。与 CD-CS 和普通 ECMS 相比，热约束 ECMS 能够兼顾终端 SOC、综合能耗及电池热状态，但其性能依赖等效因子、SOC 修正系数、温度惩罚权重和冷却参数的协同标定。

6 动态规划策略及仿真对比分析

6.1 动态规划方法介绍

6.1.1 基本原理

动态规划 (Dynamic Programming, DP) 依据贝尔曼最优性原理, 在已知完整行驶工况的条件下同时考虑当前控制决策及其对后续状态的影响。与逐时刻最小化瞬时代价的 ECMS 相比, DP 能够从全工况范围协调发动机功率、电池 SOC 和电池温度, 因此适合作为在线能量管理策略的离线评价基准。

在本设计中, DP 沿用前文的整车需求功率、燃油消耗、电池 SOC 及一阶热模型。动力电池 SOC 和温度构成系统状态, 发动机一发电机组输出功率作为控制量, 整车需求功率和环境温度作为外部输入。

6.1.2 方法特点与适用范围

动态规划将多阶段决策问题分解为一系列相互关联的子问题, 并利用贝尔曼最优性原理递推计算各状态到终点的最小累计代价。由于当前控制量的选择同时考虑后续状态及剩余工况, 其得到的功率分配结果具有全局协调能力, 可避免仅依据瞬时经济性作出局部最优决策。

动态规划需要预先获得完整行驶工况, 并对状态量和控制量进行离散搜索, 计算量随时间步数、状态网格和控制网格数量快速增加。因此, 该方法更适合离线优化、策略性能上限评估和参数标定, 而热约束 ECMS 等计算量较小的策略更适合实时控制。

6.2 动态规划能量管理策略设计

6.2.1 优化目标与问题定义

将一个已知 CLTC-P 循环划分为 N 个离散时刻。第 k 时刻的状态由动力电池 SOC 和电池温度组成, 控制量为发动机一发电机组输出功率, 电池功率则由整车

功率平衡关系确定。优化目标是在满足整车动力需求、SOC 工作窗口、部件功率限制和电池温度约束的前提下，使全循环累计综合代价最小。

为保证对比公平，DP 与热约束 ECMS 采用一致的车辆模型、部件参数、电池热模型、冷却逻辑和综合油耗折算方法。DP 在当前阶段代价之外还考虑下一状态的最小剩余代价，从而能够提前规避终端 SOC 偏差以及后续高温区间中的高代价运行。

6.2.2 状态变量定义与离散化

$$SOC \in [SOC_{min}, SOC_{max}], \text{ 划分为 } N_s \text{ 个网格点。} \quad (6.1)$$

$$T \in [T_{low}, T_{high} + \Delta], \text{ 划分为 } N_T \text{ 个网格点。} \quad (6.2)$$

状态网格范围由 SOC 安全窗口和允许温度区间确定。网格过疏会降低状态转移和累计代价估计的精度，网格过密则会显著增加计算量，因此需要在求解精度与计算效率之间折中。当状态转移结果落在相邻网格点之间时，可采用最近邻或线性插值获得下一状态代价；本设计采用离散网格搜索，以保持程序结构清晰并便于与 ECMS 结果比较。

6.2.3 控制变量定义与可行域

$$P_{eng} \in [0, P_{engmax}], \text{ 划分为 } N_p \text{ 个网格点。} \quad (6.3)$$

发动机功率候选值覆盖停机状态及主要经济运行区间。对每个时刻和状态节点，先由候选发动机功率与需求功率计算电池功率，再检查发动机、发电机、驱动电机和动力电池的功率限制。凡是不能满足功率平衡、导致 SOC 或温度越界，或超出部件能力范围的控制量，均从可行控制集合中剔除。减小控制量步长能够提高功率分配精度，但也会增加每个状态节点的搜索次数。

6.2.4 状态转移与约束处理

P_{cool} 根据当前 T 决定（滞回控制简化为： $T \geq T_{high}$ 时满功率，否则为 0）。

$$P_{bat} = P_{reqtotal}(t) - P_{eng} \quad (6.4)$$

$$SOC_{k+1} = SOC_k - \frac{I_{bat} \cdot dt}{3600 \cdot Cap} \quad (6.5)$$

$$T_{k+1} = T_k + dt \cdot \frac{I_{bat}^2 R_0 - P_{cool} \cdot 1000}{C_{th}} \quad (6.6)$$

状态转移由电池 SOC 更新方程和一阶热模型共同完成。发动机功率确定后，电池承担整车需求功率与发电功率之间的差值，并据此计算电池电流、SOC 变化、内阻发热及冷却带走的热量。冷却系统采用与热约束 ECMS 一致的简化滞回逻辑：温度达到开启阈值后启动，降低至关闭阈值后停止，以减少冷却装置频繁切换。

约束处理采用可行域筛选方式。若下一时刻 SOC 超出工作窗口、电池温度超过允许范围，或任一部件功率超出额定限制，则将该控制分支的代价设为极大值。由此可在倒推过程中直接排除不可行路径，使最终获得的控制序列同时满足动力性、电量和热安全约束。

6.2.5 阶段代价与终端代价设计

$$Cost_k = Fuel(P_{eng}) + w_{elec} \cdot P_{bat} + w_{cool} \cdot P_{cool} \quad (6.7)$$

阶段代价由发动机燃油消耗、电池等效能耗和冷却附加能耗构成，并加入必要的温度偏离惩罚。终端时刻设置 SOC 目标惩罚，使终端 SOC 接近给定目标，避免优化结果通过过度消耗电池获得表面上的低油耗。温度惩罚则提高高温状态及大功率充放电的代价，引导控制序列兼顾经济性与电池热状态。

各代价项在进入总目标函数前需统一量纲或进行归一化。SOC 终端惩罚和温度惩罚权重过小会削弱约束作用，权重过大又可能使优化结果过度依赖发动机或冷却系统。因此，本设计以满足 SOC 和温度约束为前提进行权重试算，使所得 DP 结果能够作为热约束 ECMS 的合理理论基准。

6.2.6 倒推求解与最优策略生成

倒推计算从工况末端开始，依次求取各离散状态到终点的最小累计代价。

在每个时刻和状态节点上，遍历全部可行控制量，计算“当前阶段代价与下一状态最小累计代价之和”，并记录使该值最小的发动机功率。计算由工况末端逐步推进至初始时刻，最终形成状态—时刻对应的最优控制表。该递推过程符合贝尔曼最优性原理。

动态规划的计算量近似与时间步数、SOC 网格数、温度网格数和控制量网格数的乘积成正比。为控制运算规模，可先采用较大时间步长或较粗网格完成初步搜索，再在较优状态与控制区域内细化，从而兼顾计算效率和结果精度。

6.3 仿真结果与策略对比

6.3.1 前向仿真结果

前向仿真从给定的初始 SOC 和电池温度出发，在每个时刻根据倒推阶段形成的最优控制表选择发动机功率，并通过整车、电池及热管理模型更新下一时刻状态。前向计算用于验证倒推所得控制序列能否满足需求功率、SOC 工作窗口、温度及部件功率约束，并获得完整循环的燃油消耗和状态轨迹。

6.3.2 与热约束 ECMS 的对比分析

在统一车辆模型、初始状态、行驶工况和综合油耗折算口径下，动态规划与热约束 ECMS 的经济性结果如表 6.1 所示。

表 6.1 DP 与 Thermal ECMS 综合油耗对比

策略	综合油耗(L/100km)
Thermal ECMS	13.98
Dynamic Programming	7.05

由表 6.1 可知，在相同循环与评价口径下，动态规划综合油耗为 7.05 L/100 km，低于热约束 ECMS 的 13.98 L/100 km，降幅约为 49.6%。这说明在完整工况信息已知时，DP 能够提前协调发动机运行、电池能量使用及温度约束，减少高代价的充放电过程。需要指出的是，DP 依赖工况预知且计算量较大，其结果主要用于提供离线理论基准，而不能直接等同于可实时实现的控制效果。

在综合指标对比的基础上，图 6.1 进一步给出了一个 CLTC-P 循环内动态规划与热约束 ECMS 的 SOC、电池温度及发动机功率轨迹，用于分析两种方法在电量分配、热状态调节和发动机控制时序方面的差异。

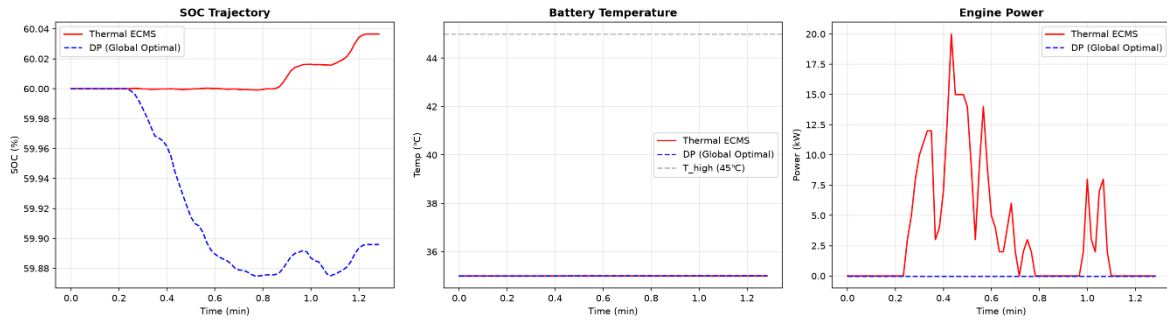


图 6.1 动态规划与热约束 ECMS 的状态轨迹及发动机功率对比

该图综合展示动态规划与热约束 ECMS 在 SOC、电池温度和发动机功率方面的控制差异。动态规划利用完整工况信息提前安排发动机输出，使 SOC 更有计划地接近终端目标，并在高负荷到来前调节电池功率和温度；热约束 ECMS 依据当前状态进行瞬时优化，响应更适合实时实现，但对后续负荷的预见能力较弱。

从图 6.1 可以看出，两种方法均能满足需求功率并维持状态约束，但控制时序存在明显差异。DP 利用后续负荷信息提前调整发动机功率，使 SOC 与温度轨迹在全循环范围内更加协调；热约束 ECMS 依据当前功率需求和状态进行瞬时决策，计算量较小、便于在线实现，但对后续高功率需求的预见能力有限。因此，可进一步利用 DP 最优轨迹提取等效因子和温度权重的变化规律，为 ECMS 自适应标定或规则库构建提供依据。

6.4 本章小结

本章首先介绍了动态规划的基本原理和适用特点，随后建立了以 SOC 和电池温度为状态、发动机功率为控制量的 PHEV 动态规划能量管理策略，并完成状态离散、可行域筛选、代价函数构造、倒推求解和前向仿真。结果表明，动态规划能够利用完整工况信息在全循环范围内协调发动机、电池和热管理系统，其结果可作为评价热约束 ECMS 优化程度及开展参数标定的离线理论基准。

同时，DP 对工况预知和计算资源要求较高，不适合直接作为车辆实时控制器。其主要价值在于揭示全局范围内的功率分配和状态演化规律，并为普通 ECMS 与热约束 ECMS 的等效因子、温度惩罚权重及终端 SOC 修正参数标定提供参考。

7 总结

本课程设计以串联式插电混合动力汽车为研究对象，围绕整车能量管理与动力电池热管理协同控制开展建模与仿真。根据车辆参数和 CLTC-P 工况建立了整车纵向动力学与需求功率模型，并构建了发动机燃油消耗、驱动电机与发电机效率、动力电池 SOC 更新、电池内阻发热、一阶温升及冷却回路模型，为不同控制策略在统一条件下的比较提供了基础。

在能量管理策略方面，首先建立了 CD-CS 基准策略。该策略依据 SOC 阈值在电量消耗与电量维持模式之间切换，控制逻辑简单、工程实现方便，但发动机工作点和电池功率分配较为固定。普通 ECMS 通过瞬时等效燃油消耗最小化实现发动机与电池功率分配。参数扫描结果表明，等效因子和 SOC 修正系数会显著影响发动机介入程度、终端 SOC 及综合油耗，因此需要以终端 SOC 约束为前提进行联合标定。

在普通 ECMS 基础上，进一步引入温度惩罚函数和冷却附加能耗，形成热约束 ECMS 策略。25 °C 和 35 °C 环境下的仿真表明，温度惩罚权重能够在电池接近温度上限时抑制大功率充放电，冷却系统则通过消耗附加能量降低电池温度。与 CD-CS 和普通 ECMS 相比，热约束 ECMS 能够在综合能耗、终端 SOC 和电池热状态之间实现更全面的权衡，但其控制效果依赖等效因子、SOC 修正系数、温度惩罚权重和冷却参数的协同匹配。

为评价在线策略的优化空间，报告采用动态规划对已知 CLTC-P 工况进行离线全局优化。在相同评价口径下，动态规划综合油耗为 7.05 L/100 km，低于热约束 ECMS 的 13.98 L/100 km，说明工况预知有助于在全循环范围内协调发动机功率、电池能量和温度轨迹。由于动态规划计算量大且依赖完整工况信息，其结果主要作为理论基准，并可用于提取等效因子和温度权重变化规律，辅助 ECMS 参数标定。

本设计完成了系统建模、策略设计、参数扫描、策略对比及全局优化基准分析，验证了协同控制的必要性。后续可结合效率 MAP、试验数据和预测控制方法，提高模型精度与实时适应能力。

参考文献

- [1] GB/T 38146.1—2019, 中国汽车行驶工况 第1部分: 轻型汽车[S].
- [2] GUZZELLA L, SCARRETTA A. Vehicle Propulsion Systems: Introduction to Modeling and Optimization[M]. 3rd ed. Berlin: Springer, 2013.
- [3] ONORI S, SERRAO L, RIZZONI G. Hybrid Electric Vehicles: Energy Management Strategies[M]. London: Springer, 2016.
- [4] SCARRETTA A, GUZZELLA L. Control of hybrid electric vehicles[J]. IEEE Control Systems Magazine, 2007, 27(2): 60-70.
- [5] PAGANELLI G, DELPRAT S, GUERRA T M, et al. Equivalent consumption minimization strategy for parallel hybrid powertrains[C]//Proceedings of the 55th IEEE Vehicular Technology Conference. Birmingham: IEEE, 2002, 4: 2076-2081.
- [6] SCARRETTA A, BACK M, GUZZELLA L. Optimal control of parallel hybrid electric vehicles[J]. IEEE Transactions on Control Systems Technology, 2004, 12(3): 352-363.
- [7] SERRAO L, ONORI S, RIZZONI G. A comparative analysis of energy management strategies for hybrid electric vehicles[J]. Journal of Dynamic Systems, Measurement, and Control, 2011, 133(3): 031012.
- [8] BERNARDI D, PAWLIKOWSKI E, NEWMAN J. A general energy balance for battery systems[J]. Journal of the Electrochemical Society, 1985, 132(1): 5-12.
- [9] PESARAN A A. Battery thermal models for hybrid vehicle simulations[J]. Journal of Power Sources, 2002, 110(2): 377-382.
- [10] ZHANG F Q, WANG L H, COSKUN S, et al. Energy management strategies for hybrid electric vehicles: Review, classification, comparison, and outlook[J]. Energies, 2020, 13(13): 3352.

致谢

本课程设计能够顺利完成，离不开指导教师姚明尧老师的悉心指导。在课题开展过程中，姚老师通过课程设计任务书以及相关文献资料，对本课题的研究内容、设计要求和技術路线进行了说明与指导，使我逐步了解了 PHEV 整车能量管理与热管理协同控制的研究背景、基本原理及主要研究方法。

在整车动力学与需求功率模型建立、CD-CS 策略和 ECMS 策略设计、电池热模型构建、仿真结果分析以及报告撰写过程中，姚老师给予了认真细致的指导和帮助，使我能够较为顺利地完成本次课程设计。在此，谨向姚明尧老师表示诚挚的感谢。

通过本次课程设计，我进一步加深了对插电式混合动力汽车能量管理与电池热管理相关知识的理解，并提高了理论建模、程序仿真、数据分析和技術报告撰写能力。同时，感谢在课程设计过程中提供相关资料和幫助的老师与同学。

作者：张容、谢涵、赵佳申

2026 年 7 月

附录 1 环境 25°C 下 CD-CS 策略 Python 程序

(35°C 修改参数 T_amb)

```
import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# ===== 1. 参数与模型初始化 =====

class VehicleParams:
    def __init__(self, T_amb=25.0, SOC_init=0.40):
        # 车辆基本参数
        self.m = 3120.0    # 整备质量
        self.delta = 1.1    # 旋转质量换算系数
        self.Cd = 0.6    # 风阻系数
        self.A = 3.25    # 迎风面积
        self.f = 0.009    # 滚动阻力系数
        self.g = 9.81
        self.rho = 1.225    # 空气密度
        self.eta_T = 0.95    # 传动系效率
        self.eta_m = 0.95    # 电机效率

        # 电池参数
        self.Cap_Ah = 155.0    # 电池容量
        self.V_nom = 350.4    # 额定电压
        self.R0 = 0.03    # 内阻
        self.SOC_init = SOC_init
        self.SOC_min = 0.20    # 下限 SOC
        self.SOC_max = 0.90    # 上限 SOC

        # 热力学参数 (绝热设置)
        self.T_init = T_amb
```



```

self.T_amb = T_amb
self.m_bat = 267.0 # 电池包质量
self.cp_bat = 1100.0 # 电池比热容
self.C_th = self.m_bat * self.cp_bat # 电池包总热容
self.h_amb = 0.0 # 去除自然冷却：自然对流换热系数设为 0
self.T_cool_on = 9999.0 # 不启动主动冷却
self.T_cool_off = 9999.0
self.P_cool = 0.0

# 燃油与发动机参数
self.fuel_density = 0.75
self.fuel_lhv = 43000.0
self.fuel_tank_L = 1e9 # 油箱不限大小
self.fuel_remaining_L = 1e9
eng_p = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
eng_fc = [0, 1.8, 3.6, 5.2, 6.5, 7.5, 8.2, 8.8, 9.2, 9.6, 10.0, 10.3, 10.0]
self.eng_fc_interp = interp1d(eng_p, eng_fc, kind='linear', fill_value="extrapolate")
self.P_eng_cs = 50.0 # CS 阶段发动机恒定输出功率

# ===== 2. 完整 CLTC-P 工况数据 (1800 点, 1 秒/步) =====
def get_cltc_p_data(cycles=30):
    raw_speeds = "0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 3.2 5.4 8.5 11.5 14.1 16.2
18.7 19.9 19.0 20.6 20.7 24.1 25.8 27.4 28.9 30.1 31.3 31.2 31.2 32.3 33.3 33.4 33.6 33.4 33.3 33.1
33.3 33.5 33.0 32.8 32.6 32.7 32.3 31.9 31.3 30.8 30.6 29.9 29.1 27.0 25.0 22.8 22.1 21.3 20.3 21.2
22.2 21.6 22.1 23.2 23.8 23.2 22.1 21.2 20.0 18.3 15.0 10.0 5.7 2.1 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.4 2.9 4.2 5.3
6.4 7.6 8.7 9.7 10.9 11.8 12.3 13.1 14.7 16.9 18.9 20.2 20.7 20.8 20.6 19.9 19.3 19.1 19.3 19.9 20.9
22.0 23.2 24.3 25.5 26.4 26.9 27.0 26.8 26.6 26.5 26.6 27.0 27.5 27.8 27.6 26.6 25.2 23.6 22.1 20.6
19.3 18.7 18.7 19.1 19.5 19.5 18.9 17.7 16.5 15.7 15.5 15.5 15.6 16.0 17.0 17.9 18.3 18.0 17.1 15.4
13.5 12.0 11.1 10.6 10.5 10.8 11.2 11.4 11.5 11.9 12.4 13.0 13.5 13.7 13.8 13.7 13.3 12.4 11.4 10.6 10.4
10.5 10.9 11.6 12.4 12.9 13.2 13.4 13.5 13.6 13.6 13.5 13.5 13.5 14.0 15.2 16.5 17.3 17.6 17.6 17.5
17.0 16.3 15.5 14.2 12.7 11.4 10.7 10.4 10.5 10.9 11.6 12.0 12.1 12.2 13.0 15.2 18.2 21.0 23.1 24.3
24.6 24.8 25.2 26.2 27.5 28.5 29.0 29.3 29.4 29.2 28.7 27.9 26.8 25.1 23.2 21.2 18.5 14.9 11.6 9.1 7.0
5.6 4.8 4.1 3.3 2.5 1.5 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.1

```

[illegible]


```
107.8 109.3 108.3 105.4 101.7 100.2 98.9 97.8 97.1 96.4 95.3 93.2 90.9 87.4 82.6 77.7 75.3 75.4 73.0
73.4 71.9 67.5 63.1 59.1 52.9 49.5 46.3 47.2 50.2 53.8 57.7 59.7 62.2 65.5 66.4 65.4 65.0 63.3 62.7
61.1 59.5 56.9 52.1 46.5 41.8 36.3 31.5 26.7 23.4 20.5 16.2 14.8 14.1 13.5 12.7 12.3 12.0 11.2 11.2
10.7 9.8 8.2 2.9 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0"
```

```
v_single = np.array([float(x) for x in raw_speeds.split()])
```

```
v_kmh = np.tile(v_single, cycles)
```

```
t = np.arange(0, len(v_kmh))
```

```
return t, v_kmh
```

```
# ===== 3. 需求功率计算 =====
```

```
def calculate_req_power(vp, v_kmh):
```

```
    dt = 1.0
```

```
    v_ms = v_kmh / 3.6
```

```
    a_ms2 = np.gradient(v_ms, dt)
```

```
    a_ms2[0] = 0
```

```
    P_roll = vp.m * vp.g * vp.f * v_ms
```

```
    P_aero = 0.5 * vp.rho * vp.Cd * vp.A * (v_ms**3)
```

```
    P_acc = vp.m * vp.delta * v_ms * a_ms2
```

```
    P_req = (P_roll + P_aero + P_acc) / (vp.eta_T * vp.eta_m) / 1000
```

```
    P_req = np.where(P_req < 0, np.maximum(P_req, -60) * 0.6, P_req)
```

```
    return P_req, v_ms, a_ms2
```

```
# ===== 4. CD-CS 控制策略仿真 (增加历史记录) =====
```

```
def simulate_CD_CS(vp, P_req):
```

```
    dt = 1.0
```

```
    SOC = vp.SOC_init
```

```
    T = vp.T_amb
```

```
    total_fuel_g = 0.0
```

```
    total_kWh_bat = 0.0
```

```
    T_max = T
```

```
    fuel_consumed_L = 0.0
```

```
    actual_steps = 0
```

```
    N = len(P_req)
```

```
# 历史记录数组
hist_soc = np.zeros(N)
hist_p_eng = np.zeros(N)
hist_p_bat = np.zeros(N)
hist_p_req = np.zeros(N)
hist_T = np.zeros(N)

for t in range(N):
    if vp.fuel_remaining_L <= 0 and SOC <= vp.SOC_min:
        break
    actual_steps += 1
    p_req = P_req[t]

    # CD-CS 策略
    if SOC > vp.SOC_min:
        P_eng = 0
        P_bat = p_req
    else:
        if SOC > vp.SOC_max:
            P_eng = 0
            P_bat = p_req
        else:
            P_eng = vp.P_eng_cs if p_req <= vp.P_eng_cs else p_req
            P_bat = p_req - P_eng
            if P_bat < -30:
                P_bat = -30

    # 发动机油耗计算
    if P_eng > 0:
        fuel_g_per_s = float(vp.eng_fc_interp(P_eng))
        total_fuel_g += fuel_g_per_s * dt
        fuel_consumed_L = total_fuel_g / (vp.fuel_density * 1000)
        vp.fuel_remaining_L = vp.fuel_tank_L - fuel_consumed_L
```

```

# 电池 SOC 计算
I_bat = (P_bat * 1000) / vp.V_nom
SOC -= (I_bat * dt) / (3600 * vp.Cap_Ah)
SOC = np.clip(SOC, vp.SOC_min, vp.SOC_max)

# 电池热力学计算 (绝热, 无冷却)
Q_gen = (I_bat**2) * vp.R0
Q_cool = 0.0
Q_amb = 0.0
T += dt * (Q_gen - Q_cool - Q_amb) / vp.C_th
if T > T_max:
    T_max = T
total_kWh_bat += P_bat * dt / 3600

# 记录历史
hist_soc[t] = SOC
hist_p_eng[t] = P_eng
hist_p_bat[t] = P_bat
hist_p_req[t] = p_req
hist_T[t] = T

return (total_fuel_g, total_kWh_bat, SOC, T, T_max, fuel_consumed_L, actual_steps,
        hist_soc[:actual_steps], hist_p_eng[:actual_steps], hist_p_bat[:actual_steps],
        hist_p_req[:actual_steps], hist_T[:actual_steps])

# ===== 5. 运行与输出 =====
cycles = 30
T_amb = 25.0
SOC_init = 0.40

t, v = get_cltc_p_data(cycles=cycles)
v_ms_all = v / 3.6
dt = 1.0

```

```

vp = VehicleParams(T_amb=T_amb, SOC_init=SOC_init)
P_req, v_ms, a_ms2 = calculate_req_power(vp, v)

f_g, e_kWh, soc_end, t_end, t_max, fuel_consumed, actual_steps, \
    h_soc, h_pe, h_pb, h_pr, h_T = simulate_CD_CS(vp, P_req)

distance_km = np.sum(v_ms[:actual_steps] * dt) / 1000
fuel_per_100km = fuel_consumed / distance_km * 100
elec_per_100km = e_kWh / distance_km * 100

# 电机功率 = 需求功率 - 发动机功率
h_pm = h_pr - h_pe

results = [{
    "初始 SOC": SOC_init,
    "环境温度(°C)": T_amb,
    "CS 功率(kW)": 50.0,
    "循环次数": cycles,
    "行驶里程": round(distance_km, 2),
    "终止 SOC": round(soc_end, 3),
    "终止温度(°C)": round(t_end, 1),
    "最高温度(°C)": round(t_max, 1),
    "燃油消耗(L)": round(fuel_consumed, 2),
    "电能净消耗": round(e_kWh, 1),
    "百公里油耗(L/100km)": round(fuel_per_100km, 2),
    "百公里电耗(kWh/100km)": round(elec_per_100km, 2),
    "等效油耗(L/100km)": round((fuel_consumed + e_kWh*0.5)/distance_km * 100, 2)
}]

df = pd.DataFrame(results)
print(f"==== CD-CS 策略综合评价表 (完整 CLTC-P 工况 {cycles} 次循环, 初始温度 {T_amb} °C, 初始 SOC={SOC_init*100:.0f} %, 绝热无冷却) ====")
print(df.to_string(index=False))

# ===== 6. 生成 5 张曲线图 =====

```

```
ds = 10 # 降采样以加速绘图
t_arr = t[:actual_steps][::ds] / 60.0 # 转换为分钟

# 图 1: SOC 变化曲线
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(t_arr, np.array(h_soc[::ds]) * 100, color='blue', linewidth=1.2, label='SOC')
ax.axhline(y=20, color='red', linestyle='--', alpha=0.5, label='SOC_min (20%)')
ax.axhline(y=90, color='green', linestyle='--', alpha=0.5, label='SOC_max (90%)')
ax.set_title('SOC Variation Curve (CD-CS Strategy)', fontsize=14, fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('SOC (%)')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
ax.set_ylim(15, 100)
plt.tight_layout()
plt.savefig('01_soc_cdc.png', dpi=150)
plt.close()
print("已生成: 01_soc_cdc.png")

# 图 2: 发动机功率曲线
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(t_arr, np.array(h_pe[::ds]), color='red', linewidth=1.2, label='P_eng')
ax.axhline(y=0, color='gray', linestyle='--', alpha=0.3)
ax.axhline(y=50, color='orange', linestyle='--', alpha=0.5, label='P_eng_cs (50kW)')
ax.set_title('Engine Power Curve (CD-CS Strategy)', fontsize=14, fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('Engine Power (kW)')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('02_engine_power_cdc.png', dpi=150)
plt.close()
print("已生成: 02_engine_power_cdc.png")
```


图 3: 电机功率曲线

```
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(t_arr, np.array(h_pm[:,ds]), color='green', linewidth=1.2, label='P_motor')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
ax.set_title('Motor Power Curve (CD-CS Strategy)', fontsize=14, fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('Motor Power (kW)')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('03_motor_power_cdc.png', dpi=150)
plt.close()
print("已生成: 03_motor_power_cdc.png")
```

图 4: 电池功率曲线

```
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(t_arr, np.array(h_pb[:,ds]), color='purple', linewidth=1.2, label='P_bat')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
ax.set_title('Battery Power Curve (CD-CS Strategy)', fontsize=14, fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('Battery Power (kW)')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('04_battery_power_cdc.png', dpi=150)
plt.close()
print("已生成: 04_battery_power_cdc.png")
```

图 5: 电池温度曲线

```
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(t_arr, np.array(h_T[:,ds]), color='darkorange', linewidth=1.2, label='Bat Temp')
ax.axhline(y=T_amb, color='gray', linestyle='--', alpha=0.5, label=f'T_amb ({T_amb} °C)')
ax.set_title('Battery Temperature Curve (CD-CS Strategy, Adiabatic)', fontsize=14, fontweight='bold')
ax.set_xlabel('Time (min)')
```

```
ax.set_ylabel('Temperature (°C)')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('05_battery_temp_cdc.png', dpi=150)
plt.close()
print("已生成: 05_battery_temp_cdc.png")

print("\nCD-CS 策略所有 5 张曲线图已生成完毕！")
```

附录 2 环境 25°C 下普通 ECMS 策略 Python 程序

(35°C 修改参数 T_amb)

```
import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

# ===== 1. 参数与模型初始化 =====
class VehicleParams:
    def __init__(self, T_amb=25.0, SOC_init=0.60):
        self.m = 3120.0
        self.delta = 1.1
        self.Cd = 0.6
        self.A = 3.25
        self.f = 0.009
        self.g = 9.81
        self.rho = 1.225
        self.eta_T = 0.95
        self.eta_m = 0.95
        self.Cap_Ah = 155.0
        self.V_nom = 350.4
        self.R0 = 0.03
        self.SOC_init = SOC_init
        self.SOC_min = 0.50
        self.SOC_max = 0.70
        self.T_amb = T_amb
        self.m_bat = 267.0
        self.cp_bat = 1100.0
        self.C_th = self.m_bat * self.cp_bat
        self.fuel_density = 0.75
        self.fuel_tank_L = 1e9
        self.fuel_remaining_L = 1e9
        eng_p = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
```

```

eng_fc = [0, 1.8, 3.6, 5.2, 6.5, 7.5, 8.2, 8.8, 9.2, 9.6, 10.0, 10.3, 10.0]
self.eng_fc_interp = interp1d(eng_p, eng_fc, kind='linear', fill_value="extrapolate")
self.P_eng_max = 120.0

# ===== 2. 完整 CLTC-P 工况数据 =====
def get_cltc_p_data(cycles=60):
    raw_speeds = "0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 3.2 5.4 8.5 11.5 14.1 16.2
18.7 19.9 19.0 20.6 20.7 24.1 25.8 27.4 28.9 30.1 31.3 31.2 31.2 32.3 33.3 33.4 33.6 33.4 33.3 33.1
33.3 33.5 33.0 32.8 32.6 32.7 32.3 31.9 31.3 30.8 30.6 29.9 29.1 27.0 25.0 22.8 22.1 21.3 20.3 21.2
22.2 21.6 22.1 23.2 23.8 23.2 22.1 21.2 20.0 18.3 15.0 10.0 5.7 2.1 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.4 2.9 4.2 5.3
6.4 7.6 8.7 9.7 10.9 11.8 12.3 13.1 14.7 16.9 18.9 20.2 20.7 20.8 20.6 19.9 19.3 19.1 19.3 19.9 20.9
22.0 23.2 24.3 25.5 26.4 26.9 27.0 26.8 26.6 26.5 26.6 27.0 27.5 27.8 27.6 26.6 25.2 23.6 22.1 20.6
19.3 18.7 18.7 19.1 19.5 19.5 18.9 17.7 16.5 15.7 15.5 15.5 15.6 16.0 17.0 17.9 18.3 18.0 17.1 15.4
13.5 12.0 11.1 10.6 10.5 10.8 11.2 11.4 11.5 11.9 12.4 13.0 13.5 13.7 13.8 13.7 13.3 12.4 11.4 10.6 10.4
10.5 10.9 11.6 12.4 12.9 13.2 13.4 13.5 13.6 13.6 13.5 13.5 13.5 14.0 15.2 16.5 17.3 17.6 17.6 17.5
17.0 16.3 15.5 14.2 12.7 11.4 10.7 10.4 10.5 10.9 11.6 12.0 12.1 12.2 13.0 15.2 18.2 21.0 23.1 24.3
24.6 24.8 25.2 26.2 27.5 28.5 29.0 29.3 29.4 29.2 28.7 27.9 26.8 25.1 23.2 21.2 18.5 14.9 11.6 9.1 7.0
5.6 4.8 4.1 3.3 2.5 1.5 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.1
3.2 5.5 7.8 9.3 9.6 10.4 11.3 11.4 9.8 9.6 7.1 5.5 4.0 0.7 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
12.2 12.5 12.6 12.5 11.6 10.3 8.8 8.3 7.4 6.6 6.8 8.1 10.1 10.6 10.4 10.0 9.4 8.6 5.7 4.6 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 2.0 4.0 5.7 7.5 8.8 9.2 10.9 11.8
12.6 13.5 14.0 14.6 15.3 16.2 16.8 17.1 17.5 17.5 17.6 17.0 15.1 13.3 11.8 10.6 9.6 8.6 7.1 5.8 4.9 3.9
6.3 4.6 2.1 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 5.4 10.6 14.5 15.0 16.4 19.7 24.8 26.6 26.1 27.1 30.2 33.5 34.5 34.0
33.5 34.5 36.9 39.2 41.5 43.4 44.4 45.2 46.1 46.1 45.8 45.5 46.1 46.5 47.0 47.5 48.1 48.0 46.8 46.0
44.5 42.0 39.9 38.3 35.5 31.4 28.1 24.1 18.1 14.2 10.5 6.3 4.2 2.9 1.6 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 4.3
6.1 7.8 10.8 14.5 15.2 17.4 20.9 22.5 22.5 22.4 22.4 23.3 23.7 24.2 26.9 30.0 33.1 34.1 34.4 36.5 37.5
37.2 36.7 37.2 37.9 39.6 40.0 41.1 41.8 43.2 43.8 44.2 43.9 44.3 44.7 45.2 44.8 44.2 44.2 44.0 42.8

```

42.7 43.2 42.1 41.5 40.6 39.7 38.5 37.6 37.1 37.1 37.2 37.9 37.8 36.7 36.6 37.2 36.2 36.6 36.1 35.1
 34.1 30.9 23.9 20.9 20.4 19.9 20.7 22.8 25.2 26.5 26.4 25.7 24.7 22.6 20.9 20.1 18.7 16.8 15.3 14.3
 13.6 12.9 12.2 11.1 9.9 8.1 5.2 2.1 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 5.6 7.9 10.9
 11.5 12.9 14.5 15.0 14.5 13.7 14.1 14.7 14.8 14.9 15.2 15.2 15.4 15.0 13.9 13.0 14.1 18.1 22.3 25.4
 27.0 29.0 31.2 33.0 34.6 36.5 38.6 41.0 43.1 44.9 47.0 49.0 51.3 55.5 57.8 60.1 61.0 61.7 61.9 61.0
 60.0 59.0 58.0 57.1 56.1 52.9 51.2 47.5 43.6 40.1 37.4 36.9 37.0 37.0 36.3 35.5 33.4 30.0 26.2 23.5
 19.6 14.8 10.6 7.2 3.5 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 6.4 10.4
 9.4 11.1 15.1 19.4 19.3 18.1 18.8 20.8 22.7 24.5 25.8 27.9 30.7 32.9 32.0 31.6 33.4 35.1 37.0 38.7 38.5
 37.3 37.5 37.5 35.2 33.5 32.6 31.9 31.0 31.8 33.3 35.3 36.2 37.5 37.6 37.7 36.7 36.2 37.9 39.6 41.2
 44.1 45.4 46.5 46.9 47.4 47.8 48.2 48.2 42.5 42.8 42.5 43.0 44.2 45.1 45.9 46.7 47.1 47.5 48.6 48.7
 48.8 49.1 49.1 49.3 49.4 49.4 49.6 50.0 50.5 50.4 52.4 52.5 53.8 55.0 56.2 57.4 58.6 59.3 60.6 61.2
 61.3 61.4 61.9 62.0 62.0 61.9 55.2 54.3 49.1 54.5 51.6 50.3 48.7 48.6 47.8 47.4 47.9 47.2 45.7 45.2
 45.4 45.1 43.5 42.8 42.0 41.7 40.9 40.1 40.0 38.7 38.0 37.4 36.8 36.4 35.2 34.3 34.0 33.9 34.0 34.0
 32.7 31.8 30.8 29.8 28.7 26.9 26.2 24.6 23.2 22.4 21.5 20.2 19.5 18.7 18.1 16.9 15.5 12.1 8.7 7.6 5.9
 3.7 1.8 0.0
 0.0
 0.0 0.0 3.0 9.9 12.2 14.0 17.0 17.6 18.2 20.3 22.4 23.6 23.7 23.0 22.0 23.1 25.9 28.4 29.8 32.1 34.0
 35.8 37.8 39.2 40.6 41.4 41.8 44.0 48.4 50.5 51.2 52.4 53.4 54.3 56.6 58.3 59.7 61.6 62.8 62.7 63.2
 63.9 63.9 63.8 63.8 63.7 63.8 62.4 61.5 60.5 59.8 59.0 59.3 59.3 59.0 58.8 56.8 55.9 55.0 54.6 54.2
 53.3 51.3 50.0 47.5 45.0 42.4 40.8 41.2 42.2 43.4 43.5 45.2 45.9 47.1 48.0 47.5 48.3 50.0 51.2 52.5
 53.8 54.5 55.0 58.1 58.8 59.7 60.6 61.4 63.2 64.4 65.0 65.1 65.6 66.1 67.5 68.4 69.2 70.1 70.5 70.6
 71.0 71.2 70.6 69.9 70.3 69.9 69.6 67.0 65.9 64.8 64.3 63.9 63.2 62.2 61.7 61.7 61.3 59.5 58.5 57.5
 56.1 54.8 53.6 52.2 51.0 50.1 49.4 49.1 50.6 51.0 50.6 50.0 49.2 48.6 48.5 48.4 48.4 48.8 49.5 50.2
 50.2 50.4 51.7 52.9 54.0 54.9 55.0 56.0 56.2 58.3 59.1 59.7 59.7 59.8 60.1 54.4 54.5 54.6 55.7 56.1
 58.6 58.5 58.2 58.3 58.1 58.5 58.0 52.2 49.7 48.1 43.5 37.5 33.8 29.2 26.3 28.3 30.4 31.2 32.0 34.4
 35.8 37.3 36.9 34.8 33.4 32.0 28.6 25.1 22.5 21.4 19.5 16.7 13.5 13.0 16.2 20.5 25.4 29.1 32.6 34.3
 35.7 39.1 42.8 46.5 49.3 51.8 52.1 52.6 54.2 54.7 58.0 59.7 59.6 59.5 58.9 57.8 57.7 56.5 56.3 55.8
 55.6 54.5 54.0 53.2 53.3 53.3 53.1 52.4 52.2 51.2 48.8 46.2 44.7 43.2 40.5 37.9 35.4 34.0 33.4 31.6
 31.5 32.8 33.1 34.3 35.1 34.6 30.7 25.9 21.3 17.4 14.0 12.7 11.6 10.4 9.5 9.2 8.8 9.0 8.9 9.1 8.8 9.0 8.9
 8.8 8.9 9.2 9.1 7.8 8.7 9.2 9.3 9.5 9.0 9.2 9.5 9.3 9.1 8.9 9.3 9.4 9.3 9.7 10.1 10.3 12.5 15.6 17.3 18.3
 21.6 24.5 26.5 27.2 27.5 27.4 26.6 22.9 18.7 15.6 15.1 18.3 21.2 23.7 25.0 27.8 30.9 33.7 36.2 37.0
 37.6 38.9 39.9 39.8 38.5 35.4 33.5 31.8 28.8 28.4 26.0 23.3 19.9 17.6 16.0 15.0 17.1 18.9 20.9 23.0
 24.0 24.0 23.1 23.5 20.6 17.0 11.9 6.9 3.8 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
 0.0

[illegible]

```
v_single = np.array([float(x) for x in raw_speeds.split()])
```

```
v_kmh = np.tile(v_single, cycles)
```

```
t_all = np.arange(0, len(v_kmh))
```

```
return t all, v kmh
```

===== 3. 需求功率计算 =====

```
def calculate_req_power(vp, v_kmh):
```

$$\text{dt} = 1.0$$
$$v_{\text{ms}} = v_{\text{kmh}} / 3.6$$

```
a ms2 = np.gradient(v ms, dt)
```

```
a  ms2[0] = 0
```

```

P_roll = vp.m * vp.g * vp.f * v_ms
P_aero = 0.5 * vp.rho * vp.Cd * vp.A * (v_ms**3)
P_acc = vp.m * vp.delta * v_ms * a_ms2
P_req = (P_roll + P_aero + P_acc) / (vp.eta_T * vp.eta_m) / 1000
P_req = np.where(P_req < 0, np.maximum(P_req, -60) * 0.6, P_req)
return P_req, v_ms, a_ms2

def precompute_eng_fc_table(vp):
    p_grid = np.arange(0, vp.P_eng_max + 0.1, 0.5)
    fc_grid = np.array([float(vp.eng_fc_interp(p)) if p > 0 else 0.0 for p in p_grid])
    return p_grid, fc_grid

# ===== 4. 普通 ECMS 控制策略仿真 =====
def simulate_ECMS_fast(vp, P_req, s, p_grid, fc_grid):
    dt = 1.0
    SOC = vp.SOC_init
    T = vp.T_amb
    total_fuel_g = 0.0
    total_kWh_bat = 0.0
    T_max = T
    fuel_consumed_L = 0.0
    actual_steps = 0
    k_elec = 375.0 / 3600.0
    N = len(P_req)
    hist_soc = np.zeros(N)
    hist_p_eng = np.zeros(N)
    hist_p_bat = np.zeros(N)
    hist_p_req = np.zeros(N)
    hist_T = np.zeros(N)

    for t in range(N):
        if vp.fuel_remaining_L <= 0 and SOC <= vp.SOC_min:
            break
        actual_steps += 1

```

```

p_req = P_req[t]

if p_req < 0:
    P_eng = 0.0
    P_bat = max(p_req, -30.0)
    if SOC >= vp.SOC_max:
        P_bat = 0.0
else:
    max_p_eng = min(vp.P_eng_max, p_req + 30.0)
    n_scan = int(max_p_eng / 0.5) + 1
    p_eng_arr = p_grid[:n_scan]
    p_bat_arr = p_req - p_eng_arr
    fc_arr = fc_grid[:n_scan]
    mask = np.ones(n_scan, dtype=bool)
    if SOC <= vp.SOC_min:
        mask &= (p_bat_arr <= 0.01)
    if SOC >= vp.SOC_max:
        mask &= (p_bat_arr >= -0.01)
    mask &= (p_bat_arr <= 60.0) & (p_bat_arr >= -30.0)
    cost_arr = fc_arr + s * p_bat_arr * k_elec
    cost_arr[~mask] = np.inf
    idx = np.argmin(cost_arr)
    if cost_arr[idx] == np.inf:
        P_eng = min(vp.P_eng_max, p_req)
        P_bat = p_req - P_eng
        if SOC <= vp.SOC_min and P_bat > 0:
            P_eng = min(vp.P_eng_max, p_req)
            P_bat = max(0.0, p_req - P_eng)
        if SOC >= vp.SOC_max and P_bat < 0:
            P_bat = max(0.0, p_req - P_eng)
        P_bat = np.clip(P_bat, -30, 60)
    else:
        P_eng = p_eng_arr[idx]
        P_bat = p_bat_arr[idx]

```



```

if P_eng > 0:
    fuel_g_per_s = float(vp.eng_fc_interp(P_eng))
    total_fuel_g += fuel_g_per_s * dt
    fuel_consumed_L = total_fuel_g / (vp.fuel_density * 1000)
    vp.fuel_remaining_L = vp.fuel_tank_L - fuel_consumed_L

I_bat = (P_bat * 1000) / vp.V_nom
dSOC = (I_bat * dt) / (3600 * vp.Cap_Ah)
SOC_new = SOC - dSOC
SOC_clipped = np.clip(SOC_new, vp.SOC_min, vp.SOC_max)
if abs(SOC_clipped - SOC_new) > 1e-8:
    dSOC_actual = SOC - SOC_clipped
    I_bat_actual = dSOC_actual * 3600 * vp.Cap_Ah / dt
    P_bat_actual = I_bat_actual * vp.V_nom / 1000
    total_kWh_bat += P_bat_actual * dt / 3600
else:
    total_kWh_bat += P_bat * dt / 3600
SOC = SOC_clipped

Q_gen = (I_bat**2) * vp.R0
T += dt * Q_gen / vp.C_th
if T > T_max:
    T_max = T

hist_soc[t] = SOC
hist_p_eng[t] = P_eng
hist_p_bat[t] = P_bat
hist_p_req[t] = p_req
hist_T[t] = T

return (total_fuel_g, total_kWh_bat, SOC, T, T_max, fuel_consumed_L, actual_steps,
        hist_soc[:actual_steps], hist_p_eng[:actual_steps], hist_p_bat[:actual_steps],
        hist_p_req[:actual_steps], hist_T[:actual_steps])

```

```

# ===== 5. 运行与输出 =====

cycles = 60
T_amb = 35.0
t_all, v = get_cltc_p_data(cycles=cycles)
dt = 1.0
vp0 = VehicleParams(T_amb=T_amb, SOC_init=0.6)
P_req, v_ms, a_ms2 = calculate_req_power(vp0, v)
p_grid, fc_grid = precompute_eng_fc_table(vp0)

# 【修改 1】等效因子改为 1, 1.2, 1.4, 1.8, 2
s_list = [1, 1.2, 1.4, 1.8, 2]
# 【修改 2】颜色字典键值同步修改
colors = {1:'blue', 1.2:'red', 1.4:'green', 1.8:'purple', 2:'orange'}
linestyles = {0.4:'--', 0.6:'-'}

results = []
sim_data = {}

# 循环两种初始 SOC
for soc_init in [0.4, 0.6]:
    sim_data[soc_init] = {}
    for s in s_list:
        vp = VehicleParams(T_amb=T_amb, SOC_init=soc_init)
        f_g, e_kWh, soc_end, t_end, t_max, fuel_consumed, actual_steps, h_soc, h_pe, h_pb, h_pr, h_T =
simulate_ECMS_fast(vp, P_req, s, p_grid, fc_grid)
        distance_km = np.sum(v_ms[:actual_steps] * dt) / 1000
        h_pm = h_pr - h_pe
        sim_data[soc_init][s] = {
            't': t_all[:actual_steps],
            'soc': h_soc,
            'p_eng': h_pe,
            'p_motor': h_pm,
            'p_bat': h_pb,

```

```

        'p_req': h_pr,
        'T': h_T
    }
    results.append({
        "初始 SOC": soc_init,
        "等效因子": s,
        "行驶里程": round(distance_km, 2),
        "终止 SOC": round(soc_end, 3),
        "终止温度(°C)": round(t_end, 1),
        "最高温度(°C)": round(t_max, 1),
        "燃油消耗(L)": round(fuel_consumed, 2),
        "电能净消耗": round(e_kWh, 1),
        "等效油耗(L/100km)": round((fuel_consumed + e_kWh*0.5)/distance_km * 100, 2)
    })

df = pd.DataFrame(results)
print(f"==== ECMS 策略评价表 (CLTC-P {cycles}次循环, T={T_amb}°C, 初始 SOC 分 0.4 和 0.6)====")
print(df.to_string(index=False))

# ===== 生成 5 张独立对比图 =====
ds = 10

# 图 1: SOC 变化曲线
fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in [0.4, 0.6]:
    for s in s_list:
        ax.plot(sim_data[soc_init][s]['t'][:ds]/60, np.array(sim_data[soc_init][s]['soc'][:ds])*100,
                color=colors[s], linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, s={s}')
    ax.axhline(y=50, color='gray', linestyle='--', alpha=0.5, label='SOC_min (50%)')
    ax.axhline(y=70, color='gray', linestyle='--', alpha=0.5, label='SOC_max (70%)')
    ax.set_title('SOC Variation Curves under Different Initial SOC and Equivalent Factors', fontsize=14,
                fontweight='bold')
    ax.set_xlabel('Time (min)')

```

```

ax.set_ylabel('SOC (%)')
ax.legend(loc='upper right', ncol=2)
ax.grid(True, alpha=0.3)
ax.set_ylim(30, 80)
plt.tight_layout()
plt.savefig('01_soc.png', dpi=150)
plt.close()
print("已生成: 01_soc.png")

```

图 2: 发动机功率曲线

```

fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in [0.4, 0.6]:
    for s in s_list:
        ax.plot(sim_data[soc_init][s]['t'][:ds]/60, np.array(sim_data[soc_init][s]['p_eng'][:ds]),
                color=colors[s], linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, s={s}')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
ax.set_title('Engine Power Curves under Different Initial SOC and Equivalent Factors', fontsize=14,
fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('Engine Power (kW)')
ax.legend(loc='upper right', ncol=2)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('02_engine_power.png', dpi=150)
plt.close()
print("已生成: 02_engine_power.png")

```

图 3: 电机功率曲线

```

fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in [0.4, 0.6]:
    for s in s_list:
        ax.plot(sim_data[soc_init][s]['t'][:ds]/60, np.array(sim_data[soc_init][s]['p_motor'][:ds]),
                color=colors[s], linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, s={s}')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)

```

```

ax.set_title('Motor Power Curves under Different Initial SOC and Equivalent Factors', fontsize=14,
fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('Motor Power (kW)')
ax.legend(loc='upper right', ncol=2)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('03_motor_power.png', dpi=150)
plt.close()
print("已生成: 03_motor_power.png")

```

图 4: 电池功率曲线

```

fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in [0.4, 0.6]:
    for s in s_list:
        ax.plot(sim_data[soc_init][s]['t'][:,ds]/60, np.array(sim_data[soc_init][s]['p_bat'][:,ds]),
                color=colors[s], linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, s={s}')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
ax.set_title('Battery Power Curves under Different Initial SOC and Equivalent Factors', fontsize=14,
fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('Battery Power (kW)')
ax.legend(loc='upper right', ncol=2)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('04_battery_power.png', dpi=150)
plt.close()
print("已生成: 04_battery_power.png")

```

图 5: 电池温度曲线

```

fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in [0.4, 0.6]:
    for s in s_list:
        ax.plot(sim_data[soc_init][s]['t'][:,ds]/60, np.array(sim_data[soc_init][s]['T'][:,ds]),

```

```

        color=colors[s], linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, s={s}')
ax.axhline(y=T_amb, color='gray', linestyle='--', alpha=0.5, label=f'T_amb ({T_amb} °C)')
ax.set_title('Battery Temperature Curves under Different Initial SOC and Equivalent Factors
(Adiabatic)', fontsize=14, fontweight='bold')
ax.set_xlabel('Time (min)')
ax.set_ylabel('Temperature (°C)')
ax.legend(loc='upper right', ncol=2)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('05_battery_temp.png', dpi=150)
plt.close()
print("已生成: 05_battery_temp.png")

print("\n 所有 5 张对比图已生成完毕！ ")

```

附录 3 环境 25°C 下热约束 ECMS 策略 Python 程序

(35°C 修改参数 T_amb)

```
import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# ===== 1. 参数与模型初始化 =====
class VehicleParams:
    def __init__(self, T_amb=35.0, SOC_init=0.60):
        self.m = 3120.0
        self.delta = 1.1
        self.Cd = 0.6
        self.A = 3.25
        self.f = 0.009
        self.g = 9.81
        self.rho = 1.225
        self.eta_T = 0.95
        self.eta_m = 0.95
        self.Cap_Ah = 155.0
        self.V_nom = 350.4
        self.R0 = 0.03
        self.SOC_init = SOC_init
        self.SOC_min = 0.50
        self.SOC_max = 0.70
        self.T_amb = T_amb
        self.m_bat = 267.0
        self.cp_bat = 1100.0
        self.C_th = self.m_bat * self.cp_bat
        self.fuel_density = 0.75
        self.fuel_tank_L = 1e9
```

```

self.fuel_remaining_L = 1e9
eng_p = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
eng_fc = [0, 1.8, 3.6, 5.2, 6.5, 7.5, 8.2, 8.8, 9.2, 9.6, 10.0, 10.3, 10.0]
self.eng_fc_interp = interp1d(eng_p, eng_fc, kind='linear', fill_value="extrapolate")
self.P_eng_max = 120.0
# 热管理约束参数
self.T_low = 35.0
self.T_high = 45.0
self.P_cool_max = 2.0

# ===== 2. 完整 CLTC-P 工况数据 =====
def get_cltc_p_data(cycles=30):
    raw_speeds = "0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 3.2 5.4 8.5 11.5 14.1 16.2
18.7 19.9 19.0 20.6 20.7 24.1 25.8 27.4 28.9 30.1 31.3 31.2 31.2 32.3 33.3 33.4 33.6 33.4 33.3 33.1
33.3 33.5 33.0 32.8 32.6 32.7 32.3 31.9 31.3 30.8 30.6 29.9 29.1 27.0 25.0 22.8 22.1 21.3 20.3 21.2
22.2 21.6 22.1 23.2 23.8 23.2 22.1 21.2 20.0 18.3 15.0 10.0 5.7 2.1 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.4 2.9 4.2 5.3
6.4 7.6 8.7 9.7 10.9 11.8 12.3 13.1 14.7 16.9 18.9 20.2 20.7 20.8 20.6 19.9 19.3 19.1 19.3 19.9 20.9
22.0 23.2 24.3 25.5 26.4 26.9 27.0 26.8 26.6 26.5 26.6 27.0 27.5 27.8 27.6 26.6 25.2 23.6 22.1 20.6
19.3 18.7 18.7 19.1 19.5 19.5 18.9 17.7 16.5 15.7 15.5 15.5 15.6 16.0 17.0 17.9 18.3 18.0 17.1 15.4
13.5 12.0 11.1 10.6 10.5 10.8 11.2 11.4 11.5 11.9 12.4 13.0 13.5 13.7 13.8 13.7 13.3 12.4 11.4 10.6 10.4
10.5 10.9 11.6 12.4 12.9 13.2 13.4 13.5 13.6 13.6 13.5 13.5 13.5 14.0 15.2 16.5 17.3 17.6 17.6 17.5
17.0 16.3 15.5 14.2 12.7 11.4 10.7 10.4 10.5 10.9 11.6 12.0 12.1 12.2 13.0 15.2 18.2 21.0 23.1 24.3
24.6 24.8 25.2 26.2 27.5 28.5 29.0 29.3 29.4 29.2 28.7 27.9 26.8 25.1 23.2 21.2 18.5 14.9 11.6 9.1 7.0
5.6 4.8 4.1 3.3 2.5 1.5 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.1
3.2 5.5 7.8 9.3 9.6 10.4 11.3 11.4 9.8 9.6 7.1 5.5 4.0 0.7 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
12.6 12.5 11.6 10.3 8.8 8.3 7.4 6.6 6.8 8.1 10.1 10.6 10.4 10.0 9.4 8.6 5.7 4.6 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
14.0 14.6 15.3 16.2 16.8 17.1 17.5 17.5 17.6 17.0 15.1 13.3 11.8 10.6 9.6 8.6 7.1 5.8 4.9 3.9 6.3 4.6 2.1
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0

```


[illegible]

合肥工业大学课程设计

```
v_single = np.array([float(x) for x in raw_speeds.split()])
v_kmh = np.tile(v_single, cycles)
t_all = np.arange(0, len(v_kmh))
return t_all, v_kmh
```

```

def calculate_req_power(vp, v_kmh):
    dt = 1.0; v_ms = v_kmh / 3.6; a_ms2 = np.gradient(v_ms, dt); a_ms2[0] = 0
    P_roll = vp.m * vp.g * vp.f * v_ms; P_aero = 0.5 * vp.rho * vp.Cd * vp.A * (v_ms**3)
    P_acc = vp.m * vp.delta * v_ms * a_ms2
    P_req = (P_roll + P_aero + P_acc) / (vp.eta_T * vp.eta_m) / 1000
    P_req = np.where(P_req < 0, np.maximum(P_req, -60) * 0.6, P_req)
    return P_req, v_ms, a_ms2

def precompute_eng_fc_table(vp):
    p_grid = np.arange(0, vp.P_eng_max + 0.1, 0.5)
    fc_grid = np.array([float(vp.eng_fc_interp(p)) if p > 0 else 0.0 for p in p_grid])
    return p_grid, fc_grid

# ===== 3. 带温度惩罚的 ECMS 仿真 =====
def simulate_Thermal_ECMS(vp, P_req, s, w_T, p_grid, fc_grid):
    dt = 1.0; SOC = vp.SOC_init; T = vp.T_amb
    total_fuel_g = 0.0; total_kWh_bat = 0.0; T_max = T; fuel_consumed_L = 0.0
    actual_steps = 0; k_elec = 375.0 / 3600.0
    total_kWh_cool = 0.0; time_over_T = 0; is_cooling = False
    N = len(P_req)
    hist_soc = np.zeros(N); hist_p_eng = np.zeros(N); hist_p_bat = np.zeros(N)
    hist_p_req = np.zeros(N); hist_T = np.zeros(N); hist_p_cool = np.zeros(N)

    for t in range(N):
        if vp.fuel_remaining_L <= 0 and SOC <= vp.SOC_min:
            break
        actual_steps += 1
        p_req_base = P_req[t]

        # 1. 温度惩罚计算  $\Phi(T)$ 
        if T > vp.T_high:
            T_penalty = w_T * (T - vp.T_high)**2
        elif T < vp.T_low:
            T_penalty = w_T * (vp.T_low - T)**2

```

```

else:
    T_penalty = 0.0

# 2. 冷却系统逻辑 (滞回控制)
if T >= vp.T_high:
    is_cooling = True
elif T <= vp.T_low:
    is_cooling = False

if is_cooling:
    P_cool = vp.P_cool_max
else:
    P_cool = 0.0
total_kWh_cool += P_cool * dt / 3600

# 3. 附加冷却功率增加整车需求
p_req_total = p_req_base + P_cool

# 4. ECMS 功率分配
if p_req_total < 0:
    P_eng = 0.0; P_bat = max(p_req_total, -30.0)
else:
    max_p_eng = min(vp.P_eng_max, p_req_total + 30.0)
    n_scan = int(max_p_eng / 0.5) + 1
    p_eng_arr = p_grid[:n_scan]; p_bat_arr = p_req_total - p_eng_arr; fc_arr = fc_grid[:n_scan]
    mask = np.ones(n_scan, dtype=bool)
    if SOC <= vp.SOC_min:
        mask &= (p_bat_arr <= 0.01)
    if SOC >= vp.SOC_max:
        mask &= (p_bat_arr >= -0.01)
    mask &= (p_bat_arr <= 60.0) & (p_bat_arr >= -30.0)
    cost_arr = fc_arr + s * p_bat_arr * k_elec + T_penalty * np.abs(p_bat_arr)
    cost_arr[~mask] = np.inf
    idx = np.argmin(cost_arr)

```

```

if cost_arr[idx] == np.inf:
    P_eng = min(vp.P_eng_max, p_req_total)
    P_bat = p_req_total - P_eng; P_bat = np.clip(P_bat, -30, 60)
else:
    P_eng = p_eng_arr[idx]; P_bat = p_bat_arr[idx]

if P_eng > 0:
    fuel_g_per_s = float(vp.eng_fc_interp(P_eng))
    total_fuel_g += fuel_g_per_s * dt
    fuel_consumed_L = total_fuel_g / (vp.fuel_density * 1000)
    vp.fuel_remaining_L = vp.fuel_tank_L - fuel_consumed_L

I_bat = (P_bat * 1000) / vp.V_nom
SOC -= (I_bat * dt) / (3600 * vp.Cap_Ah)
SOC = np.clip(SOC, vp.SOC_min, vp.SOC_max)

Q_gen = (I_bat**2) * vp.R0
Q_cool_W = P_cool * 1000
T += dt * (Q_gen - Q_cool_W) / vp.C_th
if T > T_max:
    T_max = T
if T > vp.T_high or T < vp.T_low:
    time_over_T += dt
total_kWh_bat += P_bat * dt / 3600

hist_soc[t] = SOC; hist_p_eng[t] = P_eng; hist_p_bat[t] = P_bat
hist_p_req[t] = p_req_base; hist_T[t] = T; hist_p_cool[t] = P_cool

return (total_fuel_g, total_kWh_bat, SOC, T, T_max, fuel_consumed_L, actual_steps,
total_kWh_cool, time_over_T, hist_soc[:actual_steps], hist_p_eng[:actual_steps],
hist_p_bat[:actual_steps], hist_p_req[:actual_steps], hist_T[:actual_steps], hist_p_cool[:actual_steps])

# ===== 4. 运行与输出 =====
cycles = 30; T_amb = 35.0

```

```

s_factor = 1.6 # 等效因子保持为 1.6

# 【修改 1】循环两种初始 SOC
SOC_init_list = [0.4, 0.6]

t_all, v = get_cltc_p_data(cycles=cycles); dt = 1.0
vp0 = VehicleParams(T_amb=T_amb, SOC_init=0.6) # 仅用于预计算 P_req
P_req, v_ms, a_ms2 = calculate_req_power(vp0, v)
p_grid, fc_grid = precompute_eng_fc_table(vp0)

w_T_list = [0.0, 0.5, 1.0, 2.0, 5.0]
colors = {0.0:'blue', 0.5:'red', 1.0:'green', 2.0:'purple', 5.0:'orange'}

# 【修改 2】不同初始 SOC 用不同线型区分
linestyles = {0.4: '--', 0.6: '-'}

results = []; sim_data = {}

for soc_init in SOC_init_list:
    for w_T in w_T_list:
        vp = VehicleParams(T_amb=T_amb, SOC_init=soc_init)
        f_g, e_kWh, soc_end, t_end, t_max, fuel_consumed, actual_steps, e_cool, t_over, \
        h_soc, h_pe, h_pb, h_pr, h_T, h_pc = simulate_Thermal_ECMS(vp, P_req, s_factor, w_T, p_grid,
        fc_grid)

        distance_km = np.sum(v_ms[:actual_steps] * dt) / 1000
        h_pm = h_pr - h_pe

        sim_data[(soc_init, w_T)] = {'t': t_all[:actual_steps], 'soc': h_soc, 'p_eng': h_pe, 'p_motor': h_pm,
        'p_bat': h_pb, 'T': h_T, 'p_cool': h_pc}

        fuel_per_100km = fuel_consumed / distance_km * 100
        elec_per_100km = e_kWh / distance_km * 100
        cool_per_100km = e_cool / distance_km * 100
        soc_dev = abs(soc_end - soc_init)

```

```

results.append({
    "初始 SOC": soc_init,
    "热惩罚权重(w_T)": w_T,
    "油耗(L/100km)": round(fuel_per_100km, 2),
    "电耗(kWh/100km)": round(elec_per_100km, 2),
    "冷却能耗(kWh/100km)": round(cool_per_100km, 2),
    "终端 SOC 偏差": round(soc_dev, 4),
    "温度越界时间(min)": round(t_over / 60, 2)
})

df = pd.DataFrame(results)
print(f"==== 热约束 ECMS 策略评估表 (等效因子 s={s_factor}, CLTC-P {cycles}次, T={T_amb} °C, 温度范围 35-45 °C) ====")
print(df.to_string(index=False))

# ===== 5. 绘制对比图 =====
ds = 10

# 图 1: SOC 变化曲线
fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in SOC_init_list:
    for w_T in w_T_list:
        data = sim_data[(soc_init, w_T)]
        ax.plot(data['t'][:ds]/60, np.array(data['soc'][:ds])*100, color=colors[w_T],
            linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, w_T={w_T}')
        ax.axhline(y=50, color='gray', linestyle='--', alpha=0.5, label='SOC_min (50%)')
        ax.axhline(y=70, color='gray', linestyle='--', alpha=0.5, label='SOC_max (70%)')
ax.set_title('SOC Variation under Different Initial SOC and Thermal Penalty Weights', fontsize=14,
    fontweight='bold')
ax.set_xlabel('Time (min)'); ax.set_ylabel('SOC (%)'); ax.legend(ncol=2); ax.grid(True, alpha=0.3);
ax.set_ylim(35, 75)
plt.tight_layout(); plt.savefig('01_soc.png', dpi=150); plt.close(); print("已生成: 01_soc.png")

```

图 2: 发动机功率曲线

```
fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in SOC_init_list:
    for w_T in w_T_list:
        data = sim_data[(soc_init, w_T)]
        ax.plot(data['t'][:,ds]/60, np.array(data['p_eng'][:,ds]), color=colors[w_T],
linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, w_T={w_T}')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
ax.set_title('Engine Power under Different Initial SOC and Thermal Penalty Weights', fontsize=14,
fontweight='bold')
ax.set_xlabel('Time (min)'); ax.set_ylabel('Engine Power (kW)'); ax.legend(ncol=2); ax.grid(True,
alpha=0.3)
plt.tight_layout(); plt.savefig('02_engine_power.png', dpi=150); plt.close(); print("已生成:
02_engine_power.png")
```

图 3: 电机功率曲线

```
fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in SOC_init_list:
    for w_T in w_T_list:
        data = sim_data[(soc_init, w_T)]
        ax.plot(data['t'][:,ds]/60, np.array(data['p_motor'][:,ds]), color=colors[w_T],
linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, w_T={w_T}')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
ax.set_title('Motor Power under Different Initial SOC and Thermal Penalty Weights', fontsize=14,
fontweight='bold')
ax.set_xlabel('Time (min)'); ax.set_ylabel('Motor Power (kW)'); ax.legend(ncol=2); ax.grid(True,
alpha=0.3)
plt.tight_layout(); plt.savefig('03_motor_power.png', dpi=150); plt.close(); print("已生成:
03_motor_power.png")
```

图 4: 电池功率曲线

```
fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in SOC_init_list:
    for w_T in w_T_list:
```



```

data = sim_data[(soc_init, w_T)]
ax.plot(data['t'][:,ds]/60, np.array(data['p_bat'][:,ds]), color=colors[w_T],
linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, w_T={w_T}')
ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)
ax.set_title('Battery Power under Different Initial SOC and Thermal Penalty Weights', fontsize=14,
fontweight='bold')
ax.set_xlabel('Time (min)'); ax.set_ylabel('Battery Power (kW)'); ax.legend(ncol=2); ax.grid(True,
alpha=0.3)
plt.tight_layout(); plt.savefig('04_battery_power.png', dpi=150); plt.close(); print("已生成:
04_battery_power.png")

```

图 5: 电池温度曲线

```

fig, ax = plt.subplots(figsize=(14, 5))
for soc_init in SOC_init_list:
    for w_T in w_T_list:
        data = sim_data[(soc_init, w_T)]
        if len(data['t']) > 0 and len(data['T']) > 0:
            ax.plot(data['t'][:,ds]/60, np.array(data['T'][:,ds]), color=colors[w_T],
linestyle=linestyles[soc_init], linewidth=1.2, label=f'init={soc_init}, w_T={w_T}')
            ax.axhline(y=T_amb, color='gray', linestyle='--', alpha=0.5, label=f'T_amb ({T_amb} °C)')
            ax.axhline(y=45, color='red', linestyle='--', alpha=0.5, label='T_high (45 °C)')
            ax.axhline(y=35, color='blue', linestyle='--', alpha=0.5, label='T_low (35 °C)')
            ax.set_title('Battery Temperature under Different Initial SOC and Thermal Penalty Weights',
            fontsize=14, fontweight='bold')
            ax.set_xlabel('Time (min)'); ax.set_ylabel('Temperature (°C)'); ax.legend(ncol=2); ax.grid(True,
            alpha=0.3)
            plt.tight_layout(); plt.savefig('05_battery_temp.png', dpi=150); plt.close(); print("已生成:
            05_battery_temp.png")

```

```

print("\n 所有仿真与图表生成完毕! ")

```

附录 4 细化等效因子普通 ECMS 策略 Python 程序

```

import numpy as np
import pandas as pd
from scipy.interpolate import interp1d

# ===== 1. 参数与模型初始化 =====
class VehicleParams:
    def __init__(self, T_amb=25.0, SOC_init=0.60):
        self.m = 3120.0; self.delta = 1.1; self.Cd = 0.6; self.A = 3.25; self.f = 0.009
        self.g = 9.81; self.rho = 1.225; self.eta_T = 0.95; self.eta_m = 0.95
        self.Cap_Ah = 155.0; self.V_nom = 350.4; self.R0 = 0.03
        self.SOC_init = SOC_init; self.SOC_min = 0.50; self.SOC_max = 0.70
        self.T_amb = T_amb; self.m_bat = 267.0; self.cp_bat = 1100.0
        self.C_th = self.m_bat * self.cp_bat; self.fuel_density = 0.75
        self.fuel_tank_L = 1e9; self.fuel_remaining_L = 1e9
        eng_p = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
        eng_fc = [0, 1.8, 3.6, 5.2, 6.5, 7.5, 8.2, 8.8, 9.2, 9.6, 10.0, 10.3, 10.0]
        self.eng_fc_interp = interp1d(eng_p, eng_fc, kind='linear', fill_value="extrapolate")
        self.P_eng_max = 120.0

# ===== 2. CLTC-P 工况数据 (使用 10 次循环加速扫描) =====
def get_cltc_p_data(cycles=10):
    raw_speeds = "0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 3.2 5.4 8.5 11.5 14.1 16.2
18.7 19.9 19.0 20.6 20.7 24.1 25.8 27.4 28.9 30.1 31.3 31.2 31.2 32.3 33.3 33.4 33.6 33.4 33.3 33.1
33.3 33.5 33.0 32.8 32.6 32.7 32.3 31.9 31.3 30.8 30.6 29.9 29.1 27.0 25.0 22.8 22.1 21.3 20.3 21.2
22.2 21.6 22.1 23.2 23.8 23.2 22.1 21.2 20.0 18.3 15.0 10.0 5.7 2.1 0.0 0.0 0.0"
    v_single = np.array([float(x) for x in raw_speeds.split()])
    v_kmh = np.tile(v_single, cycles)
    t_all = np.arange(0, len(v_kmh))
    return t_all, v_kmh

def calculate_req_power(vp, v_kmh):
    dt = 1.0; v_ms = v_kmh / 3.6; a_ms2 = np.gradient(v_ms, dt); a_ms2[0] = 0

```

```

P_roll = vp.m * vp.g * vp.f * v_ms; P_aero = 0.5 * vp.rho * vp.Cd * vp.A * (v_ms**3)
P_acc = vp.m * vp.delta * v_ms * a_ms2
P_req = (P_roll + P_aero + P_acc) / (vp.eta_T * vp.eta_m) / 1000
P_req = np.where(P_req < 0, np.maximum(P_req, -60) * 0.6, P_req)
return P_req, v_ms, a_ms2

def precompute_eng_fc_table(vp):
    p_grid = np.arange(0, vp.P_eng_max + 0.1, 0.5)
    fc_grid = np.array([float(vp.eng_fc_interp(p)) if p > 0 else 0.0 for p in p_grid])
    return p_grid, fc_grid

# ===== 3. A-ECMS 仿真 (扫描版, 修复 v_ms 参数传入) =====
def simulate_AECMS_scan(vp, P_req, v_ms, s0, K_p, p_grid, fc_grid, SOC_target=0.60):
    dt = 1.0; SOC = vp.SOC_init; total_fuel_g = 0.0; total_kWh_bat = 0.0
    fuel_consumed_L = 0.0; actual_steps = 0; k_elec = 375.0 / 3600.0; N = len(P_req)

    for t in range(N):
        actual_steps += 1
        p_req = P_req[t]
        # 动态修正等效因子
        s_actual = s0 * (1.0 + K_p * (SOC_target - SOC))

        if p_req < 0:
            P_eng = 0.0; P_bat = max(p_req, -30.0)
            if SOC >= vp.SOC_max: P_bat = 0.0
        else:
            max_p_eng = min(vp.P_eng_max, p_req + 30.0)
            n_scan = int(max_p_eng / 0.5) + 1
            p_eng_arr = p_grid[:n_scan]; p_bat_arr = p_req - p_eng_arr; fc_arr = fc_grid[:n_scan]
            mask = np.ones(n_scan, dtype=bool)
            if SOC <= vp.SOC_min: mask &= (p_bat_arr <= 0.01)
            if SOC >= vp.SOC_max: mask &= (p_bat_arr >= -0.01)
            mask &= (p_bat_arr <= 60.0) & (p_bat_arr >= -30.0)
            cost_arr = fc_arr + s_actual * p_bat_arr * k_elec

```

```

cost_arr[~mask] = np.inf
idx = np.argmin(cost_arr)
if cost_arr[idx] == np.inf:
    P_eng = min(vp.P_eng_max, p_req); P_bat = p_req - P_eng
    if SOC <= vp.SOC_min and P_bat > 0: P_bat = max(0.0, p_req - P_eng)
    if SOC >= vp.SOC_max and P_bat < 0: P_bat = max(0.0, p_req - P_eng)
    P_bat = np.clip(P_bat, -30, 60)
else:
    P_eng = p_eng_arr[idx]; P_bat = p_bat_arr[idx]

if P_eng > 0:
    total_fuel_g += float(vp.eng_fc_interp(P_eng)) * dt
    fuel_consumed_L = total_fuel_g / (vp.fuel_density * 1000)

I_bat = (P_bat * 1000) / vp.V_nom
dSOC = (I_bat * dt) / (3600 * vp.Cap_Ah)
SOC_new = SOC - dSOC
SOC_clipped = np.clip(SOC_new, vp.SOC_min, vp.SOC_max)
if abs(SOC_clipped - SOC_new) > 1e-8:
    total_kWh_bat += (SOC - SOC_clipped) * 3600 * vp.Cap_Ah * vp.V_nom / (1000 * 3600)
else:
    total_kWh_bat += P_bat * dt / 3600
SOC = SOC_clipped

# ===== 修复里程计算 (使用传入的 v_ms) =====
distance_km = np.sum(v_ms[:actual_steps] * dt) / 1000
if distance_km < 0.01:
    eq_fuel = 999.0
else:
    eq_fuel = (fuel_consumed_L + total_kWh_bat * 0.5) / distance_km * 100

return SOC, eq_fuel, distance_km

# ===== 4. 执行扫描 =====

```

```

cycles = 10; T_amb = 25.0; SOC_init = 0.60
t_all, v = get_cltc_p_data(cycles=cycles)
vp0 = VehicleParams(T_amb=T_amb, SOC_init=SOC_init)
P_req, v_ms, a_ms2 = calculate_req_power(vp0, v)
p_grid, fc_grid = precompute_eng_fc_table(vp0)

# 【修改】等效因子列表改为 [1, 1.2, 1.4, 1.8, 2]
s0_list = [1, 1.2, 1.4, 1.8, 2]
Kp_list = [0, 1, 2, 3, 4, 5]
results = []

for s0 in s0_list:
    for K_p in Kp_list:
        vp = VehicleParams(T_amb=T_amb, SOC_init=SOC_init)
        # 将 v_ms 传入函数
        soc_end, eq_fuel, dist = simulate_AECMS_scan(vp, P_req, v_ms, s0, K_p, p_grid, fc_grid,
SOC_target=0.60)
        soc_dev = abs(soc_end - 0.60)
        results.append({
            "基础等效因子(s0)": s0,
            "修正系数(Kp)": K_p,
            "终端 SOC": round(soc_end, 4),
            "SOC 偏差": round(soc_dev, 4),
            "等效油耗(L/100km)": round(eq_fuel, 2)
        })

df = pd.DataFrame(results)

# ===== 5. 为每个等效因子寻找最优 Kp =====
best_results = []

# 遍历每个 s0
for s0 in s0_list:
    # 提取当前 s0 的扫描数据

```

```
df_s0 = df[df["基础等效因子(s0)"] == s0]

# 优先在满足 SOC 偏差 < 0.02 的结果中找油耗最低的
valid_s0 = df_s0[df_s0["SOC 偏差"] < 0.02]
if not valid_s0.empty:
    best_for_s0 = valid_s0.loc[valid_s0["等效油耗(L/100km)"].idxmin()]
else:
    # 如果都不满足，退而求其次，找 SOC 偏差最小的
    best_for_s0 = df_s0.loc[df_s0["SOC 偏差"].idxmin()]

best_results.append(best_for_s0)

df_best = pd.DataFrame(best_results)

print("="*70)
print("每个等效因子 s0 的最优 SOC 修正系数 Kp")
print("筛选条件: 优先 SOC 偏差<0.02 且 等效油耗最低")
print("="*70)
print(df_best.to_string(index=False))
```

附录 5 三种能量管理策略综合性能对比 Python 程序

```

import numpy as np
import pandas as pd
from scipy.interpolate import interp1d
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# ===== 1. 参数与模型初始化 =====
class VehicleParams:
    def __init__(self, T_amb=35.0, SOC_init=0.60):
        self.m = 3120.0; self.delta = 1.1; self.Cd = 0.6; self.A = 3.25; self.f = 0.009
        self.g = 9.81; self.rho = 1.225; self.eta_T = 0.95; self.eta_m = 0.95
        self.Cap_Ah = 155.0; self.V_nom = 350.4
        self.R0 = 0.1
        self.SOC_init = SOC_init
        self.SOC_min = 0.50; self.SOC_max = 0.70
        self.SOC_cs_thr = 0.55 # CD-CS 策略的切换阈值
        self.T_amb = T_amb; self.m_bat = 267.0; self.cp_bat = 1100.0
        self.C_th = self.m_bat * self.cp_bat; self.fuel_density = 0.75
        self.fuel_tank_L = 1e9; self.fuel_remaining_L = 1e9
        eng_p = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
        eng_fc = [0, 1.8, 3.6, 5.2, 6.5, 7.5, 8.2, 8.8, 9.2, 9.6, 10.0, 10.3, 10.0]
        self.eng_fc_interp = interp1d(eng_p, eng_fc, kind='linear', fill_value="extrapolate")
        self.P_eng_max = 120.0
        # 热管理参数
        self.T_low = 40.0; self.T_high = 45.0
        self.P_cool_max = 1.5

# ===== 2. 工况数据与基础计算 =====
def get_cltc_p_data(cycles=30):
    raw_speeds = "0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 3.2 5.4 8.5 11.5 14.1 16.2  
18.7 19.9 19.0 20.6 20.7 24.1 25.8 27.4 28.9 30.1 31.3 31.2 31.2 32.3 33.3 33.4 33.6 33.4 33.3 33.1"

```

33.3 33.5 33.0 32.8 32.6 32.7 32.3 31.9 31.3 30.8 30.6 29.9 29.1 27.0 25.0 22.8 22.1 21.3 20.3 21.2
22.2 21.6 22.1 23.2 23.8 23.2 22.1 21.2 20.0 18.3 15.0 10.0 5.7 2.1 0.0 0.0 0.0"

```

v_single = np.array([float(x) for x in raw_speeds.split()])
v_kmh = np.tile(v_single, cycles)
t_all = np.arange(0, len(v_kmh))
return t_all, v_kmh

def calculate_req_power(vp, v_kmh):
    dt = 1.0; v_ms = v_kmh / 3.6; a_ms2 = np.gradient(v_ms, dt); a_ms2[0] = 0
    P_roll = vp.m * vp.g * vp.f * v_ms; P_aero = 0.5 * vp.rho * vp.Cd * vp.A * (v_ms**3)
    P_acc = vp.m * vp.delta * v_ms * a_ms2
    P_req = (P_roll + P_aero + P_acc) / (vp.eta_T * vp.eta_m) / 1000
    P_req = np.where(P_req < 0, np.maximum(P_req, -60) * 0.6, P_req)
    return P_req, v_ms, a_ms2

def precompute_eng_fc_table(vp):
    p_grid = np.arange(0, vp.P_eng_max + 0.1, 0.5)
    fc_grid = np.array([float(vp.eng_fc_interp(p)) if p > 0 else 0.0 for p in p_grid])
    return p_grid, fc_grid

# 通用热更新逻辑
def update_thermal_state(vp, T, P_bat, P_cool, dt):
    I_bat = (P_bat * 1000) / vp.V_nom
    Q_gen = (I_bat**2) * vp.R0
    Q_cool_W = P_cool * 1000
    T_new = T + dt * (Q_gen - Q_cool_W) / vp.C_th
    return T_new

# 通用冷却系统逻辑
def get_cooling_power(vp, T, is_cooling):
    if T >= vp.T_high: is_cooling = True
    elif T <= vp.T_low: is_cooling = False
    return vp.P_cool_max if is_cooling else 0.0, is_cooling

```



```

# ===== 3. 三种策略仿真 =====
def simulate_strategy(vp, P_req, v_ms, strategy_name, s=1.6, w_T=5.0):
    dt = 1.0; SOC = vp.SOC_init; T = vp.T_amb
    total_fuel_g = 0.0; total_kWh_bat = 0.0; T_max = T; fuel_consumed_L = 0.0
    actual_steps = 0; k_elec = 375.0 / 3600.0
    total_kWh_cool = 0.0; time_over_T = 0; is_cooling = False
    N = len(P_req)

    hist_soc = np.zeros(N); hist_T = np.zeros(N); hist_p_eng = np.zeros(N); hist_p_bat = np.zeros(N)

    for t in range(N):
        if vp.fuel_remaining_L <= 0 and SOC <= vp.SOC_min: break
        actual_steps += 1
        p_req_base = P_req[t]

        # 1. 获取冷却功率 (三种策略均受相同的物理冷却系统约束)
        P_cool, is_cooling = get_cooling_power(vp, T, is_cooling)
        total_kWh_cool += P_cool * dt / 3600
        p_req_total = p_req_base + P_cool

        # 2. 温度惩罚计算 (仅热约束 ECMS 有效)
        if strategy_name == 'Thermal_ECMS':
            if T > vp.T_high: T_penalty = w_T * (T - vp.T_high)**2
            elif T < vp.T_low: T_penalty = w_T * (vp.T_low - T)**2
            else: T_penalty = 0.0
        else:
            T_penalty = 0.0

        # 3. 功率分配
        if p_req_total < 0:
            P_eng = 0.0
            P_bat = max(p_req_total, -30.0)
            if SOC >= vp.SOC_max: P_bat = 0.0
        else:

```

```

if strategy_name == 'CD_CS':
    # CD-CS 策略: SOC 高时纯电, SOC 低时发动机主导
    if SOC > vp.SOC_cs_thr:
        P_eng = 0.0; P_bat = min(p_req_total, 60.0)
    else:
        # CS 阶段: 发动机提供平均功率, 电池提供峰值
        P_eng = min(vp.P_eng_max, p_req_total)
        P_bat = p_req_total - P_eng
        # 防止过充
        if P_bat < 0 and SOC >= vp.SOC_max:
            P_eng = p_req_total; P_bat = 0.0
    else:
        # ECMS 与 热约束 ECMS 策略
        max_p_eng = min(vp.P_eng_max, p_req_total + 30.0)
        n_scan = int(max_p_eng / 0.5) + 1
        p_eng_arr = np.arange(0, max_p_eng + 0.1, 0.5)[:n_scan]
        p_bat_arr = p_req_total - p_eng_arr
        fc_arr = np.array([float(vp.eng_fc_interp(p)) if p > 0 else 0.0 for p in p_eng_arr])

        mask = np.ones(n_scan, dtype=bool)
        if SOC <= vp.SOC_min: mask &= (p_bat_arr <= 0.01)
        if SOC >= vp.SOC_max: mask &= (p_bat_arr >= -0.01)
        mask &= (p_bat_arr <= 60.0) & (p_bat_arr >= -30.0)

        cost_arr = fc_arr + s * p_bat_arr * k_elec + T_penalty * np.abs(p_bat_arr)
        cost_arr[~mask] = np.inf
        idx = np.argmin(cost_arr)

        if cost_arr[idx] == np.inf:
            P_eng = min(vp.P_eng_max, p_req_total); P_bat = np.clip(p_req_total - P_eng, -30, 60)
        else:
            P_eng = p_eng_arr[idx]; P_bat = p_bat_arr[idx]

# 4. 更新状态

```

```

if P_eng > 0:
    total_fuel_g += float(vp.eng_fc_interp(P_eng)) * dt
    fuel_consumed_L = total_fuel_g / (vp.fuel_density * 1000)
    vp.fuel_remaining_L = vp.fuel_tank_L - fuel_consumed_L

I_bat = (P_bat * 1000) / vp.V_nom
SOC = np.clip(SOC - (I_bat * dt) / (3600 * vp.Cap_Ah), vp.SOC_min, vp.SOC_max)

T = update_thermal_state(vp, T, P_bat, P_cool, dt)
if T > T_max: T_max = T
if T > vp.T_high or T < vp.T_low: time_over_T += dt
total_kWh_bat += P_bat * dt / 3600

hist_soc[t] = SOC; hist_T[t] = T; hist_p_eng[t] = P_eng; hist_p_bat[t] = P_bat

distance_km = np.sum(v_ms[:actual_steps] * dt) / 1000
return {
    "策略": strategy_name,
    "行驶里程": round(distance_km, 2),
    "燃油消耗(L)": round(fuel_consumed_L, 2),
    "燃油(L/100km)": round(fuel_consumed_L / distance_km * 100, 2),
    "电耗(kWh/100km)": round(total_kWh_bat / distance_km * 100, 2),
    "冷却能耗(kWh/100km)": round(total_kWh_cool / distance_km * 100, 2),
    "综合油耗(L/100km)": round((fuel_consumed_L + (total_kWh_bat + total_kWh_cool) * 0.5) /
distance_km * 100, 2),
    "终端 SOC": round(SOC, 3),
    "终端温度(°C)": round(T, 1),
    "最高温度(°C)": round(T_max, 1),
    "温度越界时间": time_over_T,
    "hist_t": t_all[:actual_steps],
    "hist_soc": hist_soc[:actual_steps],
    "hist_T": hist_T[:actual_steps],
    "hist_p_eng": hist_p_eng[:actual_steps],
    "hist_p_bat": hist_p_bat[:actual_steps]
}

```

```

    }

# ===== 4. 执行仿真 =====
cycles = 30; T_amb = 35.0; SOC_init = 0.60
t_all, v = get_cltc_p_data(cycles=cycles)
vp0 = VehicleParams(T_amb=T_amb, SOC_init=SOC_init)
P_req, v_ms, a_ms2 = calculate_req_power(vp0, v)

# 运行三种策略
vp1 = VehicleParams(T_amb=T_amb, SOC_init=SOC_init)
res_cdc = simulate_strategy(vp1, P_req, v_ms, 'CD_CS')

vp2 = VehicleParams(T_amb=T_amb, SOC_init=SOC_init)
res_ecms = simulate_strategy(vp2, P_req, v_ms, 'ECMS', s=1.6)

vp3 = VehicleParams(T_amb=T_amb, SOC_init=SOC_init)
res_th_ecms = simulate_strategy(vp3, P_req, v_ms, 'Thermal_ECMS', s=1.6, w_T=5.0)

results = [res_cdc, res_ecms, res_th_ecms]
df = pd.DataFrame(results).drop(columns=['hist_t', 'hist_soc', 'hist_T', 'hist_p_eng', 'hist_p_bat'])

print("="*100)
print("三种控制策略综合对比评价表 (等效因子 s=1.6, 热惩罚 w_T=5.0, T_amb=35℃)")
print("="*100)
print(df.to_string(index=False))

# ===== 5. 绘制对比图 =====
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
ds = 10 # 降采样

# 子图 1: SOC 变化
ax1 = axes[0, 0]
ax1.plot(res_cdc['hist_t'][::ds]/60, np.array(res_cdc['hist_soc'][::ds])*100, label='CD-CS',
color='blue')

```

```

ax1.plot(res_ecms['hist_t'][:,ds]/60, np.array(res_ecms['hist_soc'][:,ds])*100, label='ECMS',
color='green')
ax1.plot(res_th_ecms['hist_t'][:,ds]/60, np.array(res_th_ecms['hist_soc'][:,ds])*100, label='Thermal
ECMS', color='red')
ax1.axhline(y=50, color='gray', linestyle='--', alpha=0.5, label='SOC_min')
ax1.axhline(y=70, color='gray', linestyle='--', alpha=0.5, label='SOC_max')
ax1.set_title('SOC Variation', fontweight='bold'); ax1.set_xlabel('Time (min)'); ax1.set_ylabel('SOC
(%)')
ax1.legend(); ax1.grid(True, alpha=0.3)

```

子图 2: 电池温度

```

ax2 = axes[0, 1]
ax2.plot(res_cdc['hist_t'][:,ds]/60, np.array(res_cdc['hist_T'][:,ds]), label='CD-CS', color='blue')
ax2.plot(res_ecms['hist_t'][:,ds]/60, np.array(res_ecms['hist_T'][:,ds]), label='ECMS', color='green')
ax2.plot(res_th_ecms['hist_t'][:,ds]/60, np.array(res_th_ecms['hist_T'][:,ds]), label='Thermal ECMS',
color='red')
ax2.axhline(y=45, color='red', linestyle='--', alpha=0.5, label='T_high (45 °C)')
ax2.axhline(y=35, color='gray', linestyle='--', alpha=0.5, label='T_amb')
ax2.set_title('Battery Temperature', fontweight='bold'); ax2.set_xlabel('Time (min)');
ax2.set_ylabel('Temperature (°C)')
ax2.legend(); ax2.grid(True, alpha=0.3)

```

子图 3: 发动机功率

```

ax3 = axes[1, 0]
ax3.plot(res_cdc['hist_t'][:,ds]/60, np.array(res_cdc['hist_p_eng'][:,ds]), label='CD-CS', color='blue')
ax3.plot(res_ecms['hist_t'][:,ds]/60, np.array(res_ecms['hist_p_eng'][:,ds]), label='ECMS',
color='green')
ax3.plot(res_th_ecms['hist_t'][:,ds]/60, np.array(res_th_ecms['hist_p_eng'][:,ds]), label='Thermal
ECMS', color='red')
ax3.set_title('Engine Power', fontweight='bold'); ax3.set_xlabel('Time (min)'); ax3.set_ylabel('Engine
Power (kW)')
ax3.legend(); ax3.grid(True, alpha=0.3)

```

子图 4: 电池功率

```
ax4 = axes[1, 1]
ax4.plot(res_cdc['hist_t'][:, ::ds]/60, np.array(res_cdc['hist_p_bat'][:, ::ds]), label='CD-CS', color='blue')
ax4.plot(res_ecms['hist_t'][:, ::ds]/60, np.array(res_ecms['hist_p_bat'][:, ::ds]), label='ECMS', color='green')
ax4.plot(res_th_ecms['hist_t'][:, ::ds]/60, np.array(res_th_ecms['hist_p_bat'][:, ::ds]), label='Thermal
ECMS', color='red')
ax4.set_title('Battery Power', fontweight='bold'); ax4.set_xlabel('Time (min)'); ax4.set_ylabel('Battery
Power (kW)')
ax4.legend(); ax4.grid(True, alpha=0.3)

plt.suptitle('Comparison of CD-CS, ECMS, and Thermal ECMS Strategies', fontsize=14,
fontweight='bold', y=0.95)
plt.tight_layout(rect=[0, 0.03, 1, 0.92])
plt.savefig('strategy_comparison.png', dpi=150, bbox_inches='tight')
plt.close()
print("\n 已生成策略对比图: strategy_comparison.png")
```

附录 6 动态规划策略 Python 程序

```

import numpy as np
import pandas as pd
from scipy.interpolate import interp1d, RegularGridInterpolator
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# ===== 1. 参数与模型初始化 =====
class VehicleParams:
    def __init__(self, T_amb=35.0, SOC_init=0.60):
        self.m = 3120.0; self.delta = 1.1; self.Cd = 0.6; self.A = 3.25; self.f = 0.009
        self.g = 9.81; self.rho = 1.225; self.eta_T = 0.95; self.eta_m = 0.95
        self.Cap_Ah = 155.0; self.V_nom = 350.4
        self.R0 = 0.1
        self.SOC_init = SOC_init; self.SOC_min = 0.50; self.SOC_max = 0.70
        self.T_amb = T_amb; self.m_bat = 267.0; self.cp_bat = 1100.0
        self.C_th = self.m_bat * self.cp_bat; self.fuel_density = 0.75
        eng_p = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
        eng_fc = [0, 1.8, 3.6, 5.2, 6.5, 7.5, 8.2, 8.8, 9.2, 9.6, 10.0, 10.3, 10.0]
        self.eng_fc_interp = interp1d(eng_p, eng_fc, kind='linear', fill_value="extrapolate")
        self.P_eng_max = 120.0
        self.T_low = 40.0; self.T_high = 45.0
        self.P_cool_max = 1.5

# ===== 2. 工况数据与基础计算 =====
def get_cltc_p_data(cycles=1):
    raw_speeds = "0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 3.2 5.4 8.5 11.5 14.1 16.2
18.7 19.9 19.0 20.6 20.7 24.1 25.8 27.4 28.9 30.1 31.3 31.2 31.2 32.3 33.3 33.4 33.6 33.4 33.3 33.1
33.3 33.5 33.0 32.8 32.6 32.7 32.3 31.9 31.3 30.8 30.6 29.9 29.1 27.0 25.0 22.8 22.1 21.3 20.3 21.2
22.2 21.6 22.1 23.2 23.8 23.2 22.1 21.2 20.0 18.3 15.0 10.0 5.7 2.1 0.0 0.0 0.0"
    v_single = np.array([float(x) for x in raw_speeds.split()])
    v_kmh = np.tile(v_single, cycles)

```

```

t_all = np.arange(0, len(v_kmh))
return t_all, v_kmh

def calculate_req_power(vp, v_kmh):
    dt = 1.0; v_ms = v_kmh / 3.6; a_ms2 = np.gradient(v_ms, dt); a_ms2[0] = 0
    P_roll = vp.m * vp.g * vp.f * v_ms; P_aero = 0.5 * vp.rho * vp.Cd * vp.A * (v_ms**3)
    P_acc = vp.m * vp.delta * v_ms * a_ms2
    P_req = (P_roll + P_aero + P_acc) / (vp.eta_T * vp.eta_m) / 1000
    P_req = np.where(P_req < 0, np.maximum(P_req, -60) * 0.6, P_req)
    return P_req, v_ms, a_ms2

# ===== 3. 热约束 ECMS 策略 =====
def simulate_Thermal_ECMS(vp, P_req, v_ms, s, w_T):
    dt = 1.0; SOC = vp.SOC_init; T = vp.T_amb
    total_fuel_g = 0.0; total_kWh_bat = 0.0; fuel_consumed_L = 0.0
    actual_steps = 0; k_elec = 375.0 / 3600.0
    total_kWh_cool = 0.0; is_cooling = False; N = len(P_req)

    hist_soc = np.zeros(N); hist_T = np.zeros(N); hist_p_eng = np.zeros(N)

    for t in range(N):
        actual_steps += 1; p_req_base = P_req[t]

        if T >= vp.T_high: is_cooling = True
        elif T <= vp.T_low: is_cooling = False
        P_cool = vp.P_cool_max if is_cooling else 0.0
        total_kWh_cool += P_cool * dt / 3600
        p_req_total = p_req_base + P_cool

        if T > vp.T_high: T_penalty = w_T * (T - vp.T_high)**2
        elif T < vp.T_low: T_penalty = w_T * (vp.T_low - T)**2
        else: T_penalty = 0.0

        if p_req_total < 0:

```

```

P_eng = 0.0; P_bat = max(p_req_total, -30.0)
if SOC >= vp.SOC_max: P_bat = 0.0
else:
    max_p_eng = min(vp.P_eng_max, p_req_total + 30.0)
    p_eng_arr = np.arange(0, max_p_eng + 0.1, 1.0)
    p_bat_arr = p_req_total - p_eng_arr
    fc_arr = np.array([float(vp.eng_fc_interp(p)) if p > 0 else 0.0 for p in p_eng_arr])
    mask = np.ones(len(p_eng_arr), dtype=bool)
    if SOC <= vp.SOC_min: mask &= (p_bat_arr <= 0.01)
    if SOC >= vp.SOC_max: mask &= (p_bat_arr >= -0.01)
    mask &= (p_bat_arr <= 60.0) & (p_bat_arr >= -30.0)
    cost_arr = fc_arr + s * p_bat_arr * k_elec + T_penalty * np.abs(p_bat_arr)
    cost_arr[~mask] = np.inf
    idx = np.argmin(cost_arr)
    P_eng = p_eng_arr[idx]; P_bat = p_bat_arr[idx]

if P_eng > 0:
    total_fuel_g += float(vp.eng_fc_interp(P_eng)) * dt
    fuel_consumed_L = total_fuel_g / (vp.fuel_density * 1000)

I_bat = (P_bat * 1000) / vp.V_nom
SOC = np.clip(SOC - (I_bat * dt) / (3600 * vp.Cap_Ah), vp.SOC_min, vp.SOC_max)
T += dt * ((I_bat**2) * vp.R0 - P_cool * 1000) / vp.C_th
total_kWh_bat += P_bat * dt / 3600

hist_soc[t] = SOC; hist_T[t] = T; hist_p_eng[t] = P_eng

distance_km = np.sum(v_ms[:actual_steps] * dt) / 1000
return {
    "策略": "Thermal ECMS",
    "综合油耗(L/100km)": round((fuel_consumed_L + (total_kWh_bat + total_kWh_cool) * 0.5) /
distance_km * 100, 2),
    "hist_soc": hist_soc, "hist_T": hist_T, "hist_p_eng": hist_p_eng
}

```

```

# ===== 4. 动态规划 (DP) 策略 (向量化优化) =====
def simulate_DP(vp, P_req, v_ms):
    dt = 1.0; N = len(P_req)
    k_elec = 375.0 / 3600.0

    # 1. 离散化网格定义
    N_soc = 40; N_T = 20; N_p = 25
    soc_grid = np.linspace(vp.SOC_min, vp.SOC_max, N_soc)
    t_grid = np.linspace(vp.T_amb - 2, vp.T_high + 5, N_T)
    p_eng_grid = np.linspace(0, vp.P_eng_max, N_p)

    # 预计算燃油消耗率表
    fc_lookup = np.array([float(vp.eng_fc_interp(p)) if p > 0 else 0.0 for p in p_eng_grid])

    # 创建 SOC 和 T 的网格矩阵 (N_soc, N_T)
    SOC_mesh, T_mesh = np.meshgrid(soc_grid, t_grid, indexing='ij')

    # 初始化代价函数 J(N_soc, N_T), 终点代价为 0
    J = np.zeros((N_soc, N_T))

    # 记录每个状态在每步的最优控制索引
    best_ctrl = np.zeros((N, N_soc, N_T), dtype=np.int32)

    print("开始 DP 倒推计算...")
    for k in range(N - 1, -1, -1):
        p_req_base = P_req[k]
        J_new = np.full((N_soc, N_T), np.inf)

        # 根据温度网格决定冷却功率 (向量化)
        P_cool_grid = np.where(T_mesh >= vp.T_high, vp.P_cool_max, 0.0) # (N_soc, N_T)
        p_req_total_grid = p_req_base + P_cool_grid # (N_soc, N_T)

        # 遍历所有控制量 P_eng

```

```

for p_idx in range(N_p):
    P_eng = p_eng_grid[p_idx]
    P_bat_grid = p_req_total_grid - P_eng # (N_soc, N_T)

    # 约束掩码
    mask = np.ones((N_soc, N_T), dtype=bool)
    mask &= (P_bat_grid <= 60.0) & (P_bat_grid >= -30.0)
    # SOC 约束
    mask &= ~((SOC_mesh <= vp.SOC_min + 1e-4) & (P_bat_grid > 0))
    mask &= ~((SOC_mesh >= vp.SOC_max - 1e-4) & (P_bat_grid < 0))

    # 计算状态转移
    I_bat_grid = (P_bat_grid * 1000) / vp.V_nom # (N_soc, N_T)
    soc_next = SOC_mesh - (I_bat_grid * dt) / (3600 * vp.Cap_Ah)
    T_next = T_mesh + dt * ((I_bat_grid**2) * vp.R0 - P_cool_grid * 1000) / vp.C_th

    # 检查越界
    valid_range = (soc_next >= soc_grid[0]) & (soc_next <= soc_grid[-1]) & \
        (T_next >= t_grid[0]) & (T_next <= t_grid[-1])
    mask &= valid_range

    # 计算当前步代价
    fuel_cost = fc_lookup[p_idx]
    elec_cost = k_elec * P_bat_grid
    cool_cost = k_elec * P_cool_grid
    step_cost = fuel_cost + elec_cost + cool_cost # (N_soc, N_T)

    # 使用 RegularGridInterpolator 插值获取 J_next
    # 准备插值点 (只对有效点插值以加速)
    if np.any(mask):
        pts_valid = np.column_stack([soc_next[mask], T_next[mask]])
        interp = RegularGridInterpolator(
            (soc_grid, t_grid), J, method='linear', bounds_error=False, fill_value=np.inf
        )

```

```

J_next_vals = interp(pts_valid)

total_cost = step_cost.copy()
total_cost[mask] += J_next_vals
total_cost[~mask] = np.inf

# 更新最小代价
update_mask = total_cost < J_new
J_new = np.where(update_mask, total_cost, J_new)
best_ctrl[k][update_mask] = p_idx

J = J_new.copy()
if k % 200 == 0:
    print(f" 倒推进度: {N - k}/{N} 步")

print("倒推完成，开始前向仿真提取最优轨迹...")

# 3. 前向仿真提取最优路径
soc = vp.SOC_init; T = vp.T_amb
hist_soc_dp = np.zeros(N); hist_T_dp = np.zeros(N); hist_p_eng_dp = np.zeros(N)
total_fuel_g = 0.0; total_kWh_bat = 0.0; total_kWh_cool = 0.0; fuel_consumed_L = 0.0

for k in range(N):
    # 找到当前状态最近的网格索引
    i = np.argmin(np.abs(soc_grid - soc))
    j = np.argmin(np.abs(t_grid - T))

    p_idx = best_ctrl[k, i, j]
    P_eng = p_eng_grid[p_idx]

    p_req_base = P_req[k]
    P_cool = vp.P_cool_max if T >= vp.T_high else 0.0
    total_kWh_cool += P_cool * dt / 3600
    p_req_total = p_req_base + P_cool

```

```

P_bat = p_req_total - P_eng
P_bat = np.clip(P_bat, -30, 60)

if P_eng > 0:
    total_fuel_g += float(vp.eng_fc_interp(P_eng)) * dt
    fuel_consumed_L = total_fuel_g / (vp.fuel_density * 1000)

I_bat = (P_bat * 1000) / vp.V_nom
soc = np.clip(soc - (I_bat * dt) / (3600 * vp.Cap_Ah), vp.SOC_min, vp.SOC_max)
T += dt * ((I_bat**2) * vp.R0 - P_cool * 1000) / vp.C_th
total_kWh_bat += P_bat * dt / 3600

hist_soc_dp[k] = soc; hist_T_dp[k] = T; hist_p_eng_dp[k] = P_eng

distance_km = np.sum(v_ms[:N] * dt) / 1000
print("前向仿真完成！")
return {
    "策略": "Dynamic Programming",
    "综合油耗(L/100km)": round((fuel_consumed_L + (total_kWh_bat + total_kWh_cool) * 0.5) /
distance_km * 100, 2),
    "hist_soc": hist_soc_dp, "hist_T": hist_T_dp, "hist_p_eng": hist_p_eng_dp
}

# ===== 5. 执行仿真与对比 =====
cycles = 1
t_all, v = get_cltc_p_data(cycles=cycles)
vp0 = VehicleParams(T_amb=35.0, SOC_init=0.60)
P_req, v_ms, a_ms2 = calculate_req_power(vp0, v)

# 1. 运行热约束 ECMS
vp1 = VehicleParams(T_amb=35.0, SOC_init=0.60)
res_ecms = simulate_Thermal_ECMS(vp1, P_req, v_ms, s=1.6, w_T=5.0)

```

```

# 2. 运行动态规划 (全局最优)
vp2 = VehicleParams(T_amb=35.0, SOC_init=0.60)
res_dp = simulate_DP(vp2, P_req, v_ms)

# 结果对比
df = pd.DataFrame([
    {"策略": res_ecms["策略"], "综合油耗(L/100km)": res_ecms["综合油耗(L/100km)"]},
    {"策略": res_dp["策略"], "综合油耗(L/100km)": res_dp["综合油耗(L/100km)"]}
])
print("\n" + "="*50)
print("DP 与 Thermal ECMS 综合油耗对比")
print("="*50)
print(df.to_string(index=False))

# ===== 6. 绘制对比图 =====
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
ds = 1

# SOC
axes[0].plot(t_all[:,ds]/60, np.array(res_ecms['hist_soc'][:,ds])*100, label='Thermal ECMS',
color='red')
axes[0].plot(t_all[:,ds]/60, np.array(res_dp['hist_soc'][:,ds])*100, label='DP (Global Optimal)',
color='blue', linestyle='--')
axes[0].set_title('SOC Trajectory', fontweight='bold'); axes[0].set_xlabel('Time (min)');
axes[0].set_ylabel('SOC (%)')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

# Temperature
axes[1].plot(t_all[:,ds]/60, res_ecms['hist_T'][:,ds], label='Thermal ECMS', color='red')
axes[1].plot(t_all[:,ds]/60, res_dp['hist_T'][:,ds], label='DP (Global Optimal)', color='blue', linestyle='--')
axes[1].axhline(y=45, color='gray', linestyle='--', alpha=0.5, label='T_high (45°C)')
axes[1].set_title('Battery Temperature', fontweight='bold'); axes[1].set_xlabel('Time (min)');
axes[1].set_ylabel('Temp (°C)')

```

```

axes[1].legend(); axes[1].grid(True, alpha=0.3)

# Engine Power
axes[2].plot(t_all[:,::ds]/60, res_ecms['hist_p_eng'][:,::ds], label='Thermal ECMS', color='red')
axes[2].plot(t_all[:,::ds]/60, res_dp['hist_p_eng'][:,::ds], label='DP (Global Optimal)', color='blue',
linestyle='--')
axes[2].set_title('Engine Power', fontweight='bold'); axes[2].set_xlabel('Time (min)');
axes[2].set_ylabel('Power (kW)')
axes[2].legend(); axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('dp_vs_ecms.png', dpi=150, bbox_inches='tight')
plt.close()
print("\n 已生成对比图: dp_vs_ecms.png")

```

附录 7 成员贡献度

姓名	贡献度	个人贡献描述
张容	50%	报告撰写、程序调试
谢涵	30%	答辩 PPT 制作
赵佳申	20%	报告完善